

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



TOÁN ỨNG DỤNG VÀ THỐNG KÊ
BÁO CÁO ĐỒ ÁN 2
DATA FITTING & PHƯƠNG PHÁP OLS

GVTH: ThS. VÕ NAM THỰC ĐOAN
ThS. LÊ NHỰT NAM

Lớp CQ2024/2
Nhóm 8

[Nguyễn Anh Thái]	[24120224]
[Nguyễn Đình Tuấn]	[24120237]
[Nguyễn Huỳnh Gia Bảo]	[24120264]
[Vòng Sau Hậu]	[24120307]
[Lương Nhật Tân]	[24120134]

Thành phố Hồ Chí Minh, ngày 8/6/2026.

Mục lục

1	Lời cảm ơn	4
2	Giới thiệu thành viên	5
3	Công cụ và môi trường phát triển	5
4	Phần 1: Lý thuyết Data Fitting và Minh Họa	6
A	Data Fitting và Phương pháp Ordinary Least Squares (OLS)	6
A.1	Giới thiệu	6
A.2	Bộ dữ liệu giả lập	6
A.3	Phương Pháp Ordinary Least Squares (OLS)	7
A.4	Ma Trận Hat (Hat Matrix)	10
A.5	Quan hệ Giữa OLS và Hat Matrix	12
A.6	Độ Phức Tạp	13
B	Đánh Giá Mô Hình Hồi Quy Tuyến Tính	13
B.1	Tổng Bình Phương Phần Dư (RSS) và Tổng Bình Phương Toàn Phần (TSS)	13
B.2	Hệ Số Xác Định R^2	15
B.3	R^2 Hiệu Chỉnh (Adjusted R^2)	17
B.4	Thống Kê F (F-statistic)	18
B.5	Hàm Tổng Hợp <code>model_metrics</code>	20
B.6	Hàm Suy Diễn Cấp Hệ Số <code>coef_inference</code>	22
B.7	Luồng Tính Toán Tổng Thể	24
B.8	Bảng Kết Quả Thực Nghiệm	24
C	Đa Cộng Tuyến và Variance Inflation Factor (VIF)	25
C.1	Đa Cộng Tuyến (Multicollinearity)	25
C.2	Variance Inflation Factor (VIF)	25
C.3	Thuật Toán và Mã Nguồn VIF	26
C.4	Bảng Kết Quả Thực Nghiệm	30
D	Ridge Regression và K-Fold Cross-Validation	31
D.1	Giới thiệu	31
D.2	Bộ dữ liệu giả lập	31
D.3	Ridge Regression	32
D.4	K-Fold Cross-Validation	38
D.5	So sánh mô hình bằng K-Fold CV	42
D.6	Độ phức tạp	44

E	Residual Diagnostic Plots cho mô hình OLS	45
E.1	Giới thiệu	45
E.2	Bộ dữ liệu giả lập	45
E.3	Mô hình OLS	46
E.4	Residual Diagnostic Plots	48
E.5	Residuals vs Fitted Values	50
E.6	Normal Q-Q Plot	51
E.7	Scale-Location Plot	53
E.8	Cook's Distance	55
E.9	Tổng kết đánh giá mô hình	57
F	Gauss–Markov Theorem và Monte Carlo Simulation	58
F.1	Giới thiệu	58
F.2	Cơ sở lý thuyết Gauss–Markov	58
F.3	Thiết lập mô phỏng Monte Carlo	59
F.4	Kiểm tra tính không chệch	61
F.5	So sánh phương sai	62
F.6	Phân tích Mean Squared Error	63
F.7	Trực quan hóa phân bố hệ số ước lượng	64
F.8	Bias–Variance Tradeoff	66
F.9	Kết luận	66
5	Phần 2: Ứng Dụng Data Fitting vào Dữ Liệu Thực Tế	67
A	Giới thiệu	67
A.1	Bài toán và Mục tiêu	67
A.2	Quy ước thống kê	67
B	Hiểu dữ liệu	68
B.1	Mô tả dữ liệu (Data Description)	68
B.2	Khám phá dữ liệu	70
C	Chuẩn bị dữ liệu	84
C.1	Tổng quan	84
C.2	Lựa chọn biến	85
C.3	Chia tập dữ liệu	88
C.4	Làm sạch dữ liệu	89
C.5	Xây dựng dữ liệu và biến đổi phân phối	94
C.6	Mã hóa biến phân loại	97
C.7	Chuẩn bị ma trận đặc trưng và biến mục tiêu	100
C.8	Đóng gói quy trình bằng DataPipeline	101

C.9	Kết luận	102
D	Xây dựng mô hình (Modeling)	102
D.1	Mô hình OLS cơ bản	102
D.2	Mô hình OLS chọn biến	105
D.3	Mô hình Ridge/Lasso	108
D.4	So sánh các mô hình trên	110
E	Đánh giá trên tập test	113
E.1	Chạy trên bộ test và chọn mô hình tốt nhất	113
E.2	Kiểm chứng với phần dư (Residual Analysis)	116
6	Kết luận	121
A	Tóm tắt kết quả đạt được	121
B	Bài học rút ra	121
C	Hướng mở rộng	122
7	Tài liệu tham khảo	123

1 Lời cảm ơn

Nhóm xin gửi lời cảm ơn chân thành đến giảng viên Võ Nam Thục Đoan và thầy Lê Nhựt Nam, bộ môn Toán Ứng Dụng và Thống Kê, Trường Đại học Khoa học Tự nhiên – ĐHQG TP.HCM, vì đã tận tình giảng dạy và hướng dẫn chúng em trong suốt quá trình học tập và thực hiện đồ án. Trong quá trình nghiên cứu đề tài Data Fitting và phương pháp bình phương tối thiểu thông thường (Ordinary Least Squares – OLS), nhóm đã có cơ hội vận dụng các kiến thức về đại số tuyến tính, tối ưu hóa và thống kê để xây dựng mô hình xấp xỉ dữ liệu, đánh giá sai số và phân tích ý nghĩa của nghiệm thu được.

Nhóm cũng xin chân thành cảm ơn các anh chị, bạn bè và gia đình đã hỗ trợ trong quá trình tìm kiếm tài liệu, chuẩn bị dữ liệu thực nghiệm và góp ý để bài làm được hoàn thiện hơn. Quá trình cùng nhau thực hiện đồ án giúp chúng em hiểu rõ hơn vai trò của Data Fitting trong việc mô hình hóa các hiện tượng thực tế, đồng thời nhận thức sâu sắc hơn về sức mạnh cũng như giới hạn của phương pháp OLS khi áp dụng vào các bài toán tính toán khoa học. Đây là trải nghiệm quý báu, giúp chúng em rèn luyện kỹ năng phân tích dữ liệu, tư duy định lượng và tinh thần làm việc nhóm cho quá trình học tập, nghiên cứu sau này.

2 Giới thiệu thành viên

STT	Họ và tên	MSSV	Công việc	Đóng góp
1	Nguyễn Đình Tuấn	24120237	Cài đặt hàm <code>ridge_fit</code> , <code>kfold_cv</code> và chuẩn bị dữ liệu	100%
2	Nguyễn Anh Thái	24120224	Cài đặt hàm <code>coef_inference</code> và tìm hiểu dữ liệu	100%
3	Nguyễn Huỳnh Gia Bảo	24120264	Cài đặt hàm <code>residual_plots</code> , minh họa định lý Gauss–Markov, xây dựng mô hình và dự đoán	100%
4	Vòng Sau Hậu	24120307	Cài đặt hàm <code>model_metrics</code> , <code>vif</code> , xây dựng mô hình và dự đoán	100%
5	Lương Nhật Tân	24120134	Cài đặt hàm <code>ols_fit</code> , <code>hat_matrix</code> và đánh giá mô hình $Ax = b$	100%

3 Công cụ và môi trường phát triển

- **Python 3.10+:** Ngôn ngữ cài đặt chính.
- **NumPy, SciPy:** Tính toán số; dùng để **kiểm chứng**, không dùng để thay thế cài đặt thuật toán.
- **Pandas:** Đọc, xử lý và thao tác dữ liệu.
- **Matplotlib, Seaborn:** Trực quan hóa dữ liệu và kết quả mô hình.
- **Scikit-learn:** Chỉ dùng để **so sánh** và kiểm chứng kết quả, **không dùng** để cài đặt OLS chính.
- **Jupyter Notebook:** Trình bày toàn bộ thực nghiệm.

4 Phần 1: Lý thuyết Data Fitting và Minh Họa

A Data Fitting và Phương pháp Ordinary Least Squares (OLS)

A.1 Giới thiệu

Phần này trình bày chi tiết cơ sở toán học, cách chứng minh, mã nguồn cài đặt và kết quả thực nghiệm cho hai nội dung chính: bài toán Data Fitting bằng phương pháp Ordinary Least Squares (OLS) và Ma trận Hat (Hat Matrix). Các thuật toán được xây dựng bằng Python thuần thông qua các thao tác đại số tuyến tính cơ bản – chuyển vị ma trận, nhân ma trận, và nghịch đảo ma trận bằng khử Gauss–Jordan – mà không dùng `numpy.linalg`.

A.2 Bộ dữ liệu giả lập

Dữ liệu được sinh ngẫu nhiên có kiểm soát bằng `np.random.seed(42)` để bảo đảm kết quả có thể tái lập. Số quan sát là $n = 100$, số đặc trưng ban đầu là $p = 3$. Khi xét bài toán hồi quy có hệ số chặn, một cột toàn số 1 được thêm vào đầu để tạo ma trận thiết kế:

$$X = \begin{bmatrix} 1 & x_{11} & x_{12} & x_{13} \\ 1 & x_{21} & x_{22} & x_{23} \\ \vdots & \vdots & \vdots & \vdots \\ 1 & x_{100,1} & x_{100,2} & x_{100,3} \end{bmatrix} \in \mathbb{R}^{100 \times 4}.$$

Vector hệ số thật được đặt là

$$\beta^* = \begin{bmatrix} 5.0 & 2.0 & -3.0 & 1.5 \end{bmatrix}^T.$$

Biến mục tiêu được sinh theo mô hình tuyến tính có nhiễu Gaussian:

$$y = X\beta^* + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2 I),$$

với $\sigma = 2$.

Đoạn mã sinh dữ liệu:

```

1 np.random.seed(42)
2 n, p = 100, 3
3
4 X_feat = np.random.randn(n, p)
5 X = np.c_[np.ones(n), X_feat]          # thêm cột intercept
6
7 true_beta = np.array([5.0, 2.0, -3.0, 1.5])
8 noise = np.random.randn(n) * 2
9 y = X @ true_beta + noise
10
11 X_list = X.tolist()
12 y_list = [[yi] for yi in y.tolist()]

```

A.3 Phương Pháp Ordinary Least Squares (OLS)

A.3.1. Bài toán và hàm mất mát

Cho n quan sát (x_i, y_i) , $i = 1, \dots, n$, trong đó $x_i \in \mathbb{R}^{p+1}$ là vector đặc trưng (đã bao gồm phần tử 1 ở đầu) và $y_i \in \mathbb{R}$ là biến mục tiêu. Nhóm giả định mô hình tuyến tính:

$$y_i = x_i^T \beta + \varepsilon_i, \quad i = 1, \dots, n,$$

trong đó $\beta \in \mathbb{R}^{p+1}$ là vector tham số cần ước lượng và ε_i là sai số ngẫu nhiên.

Phương pháp OLS tìm $\hat{\beta}$ tối thiểu hóa tổng bình phương phần dư (Residual Sum of Squares):

$$\text{RSS}(\beta) = \|y - X\beta\|_2^2 = \sum_{i=1}^n (y_i - x_i^T \beta)^2.$$

A.3.2. Nghiệm OLS – Normal Equations

Định lý (Nghiệm OLS): Nếu $X^T X$ khả nghịch, nghiệm OLS duy nhất là:

$$\hat{\beta}_{OLS} = (X^T X)^{-1} X^T y. \quad (1)$$

Chứng minh: Viết lại hàm mất mát dưới dạng vector:

$$RSS(\beta) = (y - X\beta)^T (y - X\beta).$$

Khai triển:

$$RSS(\beta) = y^T y - 2\beta^T X^T y + \beta^T X^T X \beta.$$

Lấy gradient theo β và cho bằng 0:

$$\nabla_{\beta} RSS(\beta) = -2X^T y + 2X^T X \beta = \mathbf{0}.$$

Nhóm được **phương trình chuẩn** (Normal Equations):

$$X^T X \beta = X^T y.$$

Vì $X^T X$ khả nghịch, nhân cả hai vế bên trái với $(X^T X)^{-1}$ nhóm được:

$$\hat{\beta}_{OLS} = (X^T X)^{-1} X^T y.$$

A.3.3. Ước lượng phương sai $\hat{\sigma}^2$

Sau khi tìm được $\hat{\beta}$, phương sai của sai số ngẫu nhiên được ước lượng không chệch bởi:

$$\hat{\sigma}^2 = \frac{RSS}{n - p - 1} = \frac{\|y - X\hat{\beta}\|_2^2}{n - p - 1}, \quad (2)$$

trong đó $n - p - 1$ là bậc tự do (degree of freedom): n trừ đi số tham số ước lượng (p hệ số đặc trưng cộng 1 hệ số chặn).

Mệnh đề: $\hat{\sigma}^2$ là ước lượng không chệch của σ^2 , tức là $\mathbb{E}[\hat{\sigma}^2] = \sigma^2$.

Phác thảo chứng minh: Gọi $\hat{y} = X\hat{\beta} = Hy$ là giá trị dự đoán (với H là hat matrix). Phần dư là $\hat{\epsilon} = y - \hat{y} = (I - H)y$. Nhóm có:

$$\text{RSS} = \hat{\epsilon}^T \hat{\epsilon} = y^T (I - H)^T (I - H) y = y^T (I - H) y,$$

vì $(I - H)$ là ma trận chiếu đối xứng và lũy đẳng: $(I - H)^2 = I - H$. Lấy kỳ vọng:

$$\mathbb{E}[\text{RSS}] = \mathbb{E}[y^T (I - H) y] = \text{tr}((I - H) \text{Var}(y)) + \mu^T (I - H) \mu,$$

với $\mu = X\beta$ và $\text{Var}(y) = \sigma^2 I$. Vì $(I - H)X = 0$ (do $HX = X$), nhóm có $(I - H)\mu = 0$ và:

$$\mathbb{E}[\text{RSS}] = \sigma^2 \text{tr}(I - H) = \sigma^2 (n - \text{tr}(H)) = \sigma^2 (n - (p + 1)).$$

Do đó $\mathbb{E}[\hat{\sigma}^2] = \mathbb{E}[\text{RSS}] / (n - p - 1) = \sigma^2$.

A.3.4. Cài đặt hàm `ols_fit`

Hàm `ols_fit(X, y)` nhận vào ma trận thiết kế $X \in \mathbb{R}^{n \times (p+1)}$ (đã bao gồm cột intercept) và vector nhãn $y \in \mathbb{R}^{n \times 1}$ dưới dạng list. Hàm trả về $\hat{\beta}$ theo công thức (1) và $\hat{\sigma}^2$ theo công thức (2).

```

1 def ols_fit(X, y):
2     """Tính toán vector hệ số beta_hat cho bài toán Data Fitting (
3     OLS)."""
4     # hat{beta} = (X^T * X)^{-1} * X^T * y
5     X_T = transpose(X)
6
7     X_T_X = multiply(X_T, X)
8     inv_X_T_X = invert(X_T_X)
9
10    X_T_y = multiply(X_T, y)
11    beta_hat = multiply(inv_X_T_X, X_T_y)
12
13    n = len(y)
14    p = len(X[0]) - 1
15    RSS = calculate_RSS(X, y, beta_hat)
16    sigma2_hat = RSS / (n - p - 1)
17
18    return beta_hat, sigma2_hat

```

A.3.5. Kết quả kiểm chứng `ols_fit`

So sánh $\hat{\beta}$ tự cài đặt với `numpy.linalg` và lớp `LinearRegression` của thư viện `sklearn`:

Phương pháp	$\hat{\beta}_0$	$\hat{\beta}_1$	$\hat{\beta}_2$	$\hat{\beta}_3$
Tự cài đặt	5.0	2.0	-3.0	1.5 (xấp xỉ)
<code>numpy.linalg</code>	(trùng khớp)			
<code>sklearn</code>	(trùng khớp)			

Kiểm tra `np.allclose(custom_beta, numpy_beta, atol=1e-5)` trả về `True`.

A.4 Ma Trận Hat (Hat Matrix)

A.4.1. Định nghĩa và ý nghĩa

Định nghĩa (Hat Matrix): Ma trận chiếu (projection matrix) hay **hat matrix** được định nghĩa là:

$$H = X(X^T X)^{-1} X^T \in \mathbb{R}^{n \times n}. \quad (3)$$

H được gọi là “hat matrix” vì nó “đội mũ” cho y : giá trị dự đoán $\hat{y}$ chính là hình chiếu của y lên không gian cột của X :

$$\hat{y} = X\hat{\beta} = X(X^T X)^{-1} X^T y = Hy.$$

Phần dư: $\hat{\epsilon} = y - \hat{y} = (I - H)y$.

A.4.2. Tính chất của H

Mệnh đề (Các tính chất của Hat Matrix):

- (i) $H^2 = H$ (lũy đẳng – idempotent).
- (ii) $H^T = H$ (đối xứng – symmetric).
- (iii) Các trị riêng của H chỉ là 0 hoặc 1.
- (iv) $\text{rank}(H) = p + 1$.
- (v) Giá trị fitted: $\hat{y} = Hy$; phần dư: $\hat{\epsilon} = (I - H)y$.

Chứng minh:

(i) Tính lũy đẳng:

$$\begin{aligned} H^2 &= [X(X^T X)^{-1} X^T] [X(X^T X)^{-1} X^T] \\ &= X(X^T X)^{-1} \underbrace{(X^T X)(X^T X)^{-1}}_{=I} X^T \\ &= X(X^T X)^{-1} X^T = H. \checkmark \end{aligned}$$

(ii) Tính đối xứng:

$$\begin{aligned} H^T &= [X(X^T X)^{-1} X^T]^T \\ &= (X^T)^T [(X^T X)^{-1}]^T X^T \\ &= X [(X^T X)^T]^{-1} X^T \\ &= X(X^T X)^{-1} X^T = H, \checkmark \end{aligned}$$

vì $X^T X$ đối xứng nên $(X^T X)^T = X^T X$.

(iii) Trị riêng:

Vì H lũy đẳng, nếu $Hv = \lambda v$ thì $H^2 v = \lambda^2 v = Hv = \lambda v$, do đó $\lambda^2 = \lambda$, suy ra $\lambda \in \{0, 1\}$.

(iv) Hạng:

$$\text{rank}(H) = \text{tr}(H) = \text{tr}[X(X^T X)^{-1} X^T] = \text{tr}[(X^T X)^{-1} X^T X] = \text{tr}(I_{p+1}) = p + 1.$$

(v) Trực tiếp từ định nghĩa: $\hat{y} = X\hat{\beta} = X(X^T X)^{-1} X^T y = Hy$.

A.4.3. Cài đặt hàm `get_hat_matrix`

Hàm `get_hat_matrix(X)` nhận ma trận thiết kế X , tính H theo công thức (3) và kiểm tra tính lũy đẳng $H^2 = H$.

```

1 def get_hat_matrix(X):
2     """Tính ma trận Hat (H) và kiểm tra tính lũy đẳng (H^2 = H)."""
3     # H = X * (X^T * X)^{-1} * X^T
4     X_T = transpose(X)
5
6     X_T_X = multiply(X_T, X)
7     inv_X_T_X = invert(X_T_X)
8
9     X_inv_X_T_X = multiply(X, inv_X_T_X)
10    H = multiply(X_inv_X_T_X, X_T)
11
12    # Kiểm tra tính lũy đẳng
13    H_squared = multiply(H, H)
14    idempotent_flag = is_idempotent(H_squared, H)
15
16    return H, idempotent_flag

```

A.4.4. Kết quả kiểm chứng `get_hat_matrix`

- Kiểm tra $H^2 = H$: hàm `is_idempotent(H_squared, H)` trả về True.
- So sánh với numpy: `np.allclose(H_custom, H_numpy, atol=1e-9)` trả về True.
- $\text{tr}(H) = p + 1 = 4$ (với $p = 3$).

A.5 Quan hệ Giữa OLS và Hat Matrix

Hat matrix H kết nối trực tiếp các đại lượng trong bài toán OLS:

- **Giá trị dự đoán:** $\hat{y} = Hy$.
- **Phần dư:** $\hat{\epsilon} = (I - H)y$, và $(I - H)$ cũng lũy đẳng, đối xứng.
- **RSS:** $\text{RSS} = \|y - \hat{y}\|^2 = y^T(I - H)y$.
- **Phần tử chéo** h_{ii} của H (còn gọi là *leverage*) đo lường mức độ ảnh hưởng của quan sát i lên giá trị dự đoán $\hat{y}_i$. Các điểm có h_{ii} lớn là các điểm đòn bẩy (leverage point).

A.6 Độ Phức Tạp

Giả sử ma trận thiết kế có kích thước $n \times d$, trong đó $d = p + 1$. Phân tích độ phức tạp của hai hàm chính:

- Tính $X^T X$ ($d \times d$): $O(nd^2)$.
- Nghịch đảo ma trận $d \times d$ bằng Gauss–Jordan: $O(d^3)$.
- Tính $X^T y$: $O(nd)$.
- Nhân $(X^T X)^{-1}$ với $X^T y$: $O(d^2)$.

Vậy, `ols_fit` có độ phức tạp tổng thể:

$$O(nd^2 + d^3).$$

Đối với `get_hat_matrix`, phép nhân cuối tạo ra $H \in \mathbb{R}^{n \times n}$ có chi phí $O(n^2 d)$, nên tổng độ phức tạp là:

$$O(nd^2 + d^3 + n^2 d).$$

Với $n \gg d$ (thường gặp trong thực tế), thành phần $O(n^2 d)$ chiếm ưu thế.

B Đánh Giá Mô Hình Hồi Quy Tuyến Tính

B.1 Tổng Bình Phương Phần Dư (RSS) và Tổng Bình Phương Toàn Phần (TSS)

B.1.1. Cơ sở lý thuyết

Định nghĩa 4.1 (Residual Sum of Squares — RSS). Tổng bình phương phần dư đo sai số còn lại sau khi mô hình đã được khớp:

$$\text{RSS} = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \|y - \hat{y}\|_2^2.$$

Định nghĩa 4.2 (Total Sum of Squares — TSS). Tổng bình phương toàn phần đo biến thiên của y so với giá trị trung bình:

$$\text{TSS} = \sum_{i=1}^n (y_i - \bar{y})^2 = \|y - \bar{y} \mathbf{1}\|_2^2, \quad \bar{y} = \frac{1}{n} \sum_{i=1}^n y_i.$$

Phân rã cơ bản của biến thiên:

$$\underbrace{\text{TSS}}_{\text{tổng biến thiên}} = \underbrace{\text{ESS}}_{\text{mô hình giải thích}} + \underbrace{\text{RSS}}_{\text{sai số còn lại}}, \quad \text{ESS} = \text{TSS} - \text{RSS}.$$

B.1.2. Ký hiệu

- y_i : giá trị quan sát thực tế của quan sát thứ i .
- $\hat{y}_i$: giá trị dự đoán từ mô hình OLS.
- $y_i - \hat{y}_i$: phần dư (residual) của quan sát i .
- $\bar{y}$: trung bình mẫu của y — một hằng số với dữ liệu cố định.
- TSS: không phụ thuộc mô hình, chỉ phụ thuộc dữ liệu y .

B.1.3. Mã giả

Algorithm 1 Tính RSS và TSS

Input: $y = [y_1, \dots, y_n]$, $\hat{y} = [\hat{y}_1, \dots, \hat{y}_n]$

Output: Các giá trị vô hướng RSS và TSS

```

1: RSS  $\leftarrow$  0
2: for  $i \leftarrow 1$  to  $n$  do
3:    $d \leftarrow y_i - \hat{y}_i$ 
4:   RSS  $\leftarrow$  RSS +  $d^2$ 
5: end for

6:  $\bar{y} \leftarrow \frac{1}{n} \sum_{i=1}^n y_i$ 
7: TSS  $\leftarrow$  0
8: for  $i \leftarrow 1$  to  $n$  do
9:    $d \leftarrow y_i - \bar{y}$ 
10:  TSS  $\leftarrow$  TSS +  $d^2$ 
11: end for
12: return RSS, TSS

```

B.1.4. Phân tích độ phức tạp

- **Thời gian:** $O(n)$ — hai vòng lặp độc lập, mỗi vòng n bước.
- **Bộ nhớ:** $O(1)$ — chỉ dùng biến tích lũy, không cấp phát mảng trung gian.

B.1.5. Cài đặt thuật toán

```

1  def RSS(y: list[float], y_hat: list[float]) -> float:
2      """RSS = ||y - y_hat||^2."""
3      rss = 0.0
4      for yi, yhi in zip(y, y_hat):
5          diff = yi - yhi          # phan du tung quan sat
6          rss += diff * diff       # cong don binh phuong phan du
7      return rss
8
9
10 def TSS(y: list[float]) -> float:
11     """TSS = ||y - y_bar||^2."""
12     y_bar = sum(y) / len(y)      # trung binh mau
13     tss = 0.0
14     for yi in y:
15         diff = yi - y_bar        # do lech so voi trung binh
16         tss += diff * diff       # cong don binh phuong do lech
17     return tss

```

Listing 1: Hàm RSS và TSS trong model_metrics.py

B.2 Hệ Số Xác Định R^2

B.2.1. Ý nghĩa và công thức

Định nghĩa 4.3 (Coefficient of Determination). R^2 đo tỷ lệ biến thiên của y được giải thích bởi mô hình:

$$R^2 = 1 - \frac{\text{RSS}}{\text{TSS}} = \frac{\text{ESS}}{\text{TSS}}, \quad R^2 \in [0, 1].$$

B.2.2. Ký hiệu

- RSS/TSS : tỷ lệ biến thiên *chưa* được mô hình giải thích.
- $1 - \text{RSS}/\text{TSS}$: tỷ lệ biến thiên *đã* được giải thích.
- $R^2 = 1$: mô hình khớp hoàn hảo ($\text{RSS} = 0$).
- $R^2 = 0$: mô hình không tốt hơn đường ngang $\hat{y} = \bar{y}$.

B.2.3. Mã giả

Algorithm 2 Tính R^2

Input: $y, \hat{y}$

Output: $R^2 \in [0, 1]$

```

1:  $RSS \leftarrow RSS(y, \hat{y})$ 
2:  $TSS \leftarrow TSS(y)$ 
3: if  $TSS = 0$  then
4:     raise ValueError ▷ y không có biến thiên
5: end if
6:  $r^2 \leftarrow 1 - RSS/TSS$ 
7: if  $r^2 < 0$  or  $r^2 > 1$  then
8:     raise ValueError
9: end if
10: return  $r^2$ 

```

B.2.4. Phân tích độ phức tạp

- **Thời gian:** $O(n)$ — gọi lại RSS và TSS, mỗi hàm $O(n)$; các bước còn lại $O(1)$.
- **Bộ nhớ:** $O(1)$.

B.2.5. Cài đặt thuật toán

```

1  def R_squared(y: list[float], y_hat: list[float]) -> float:
2      """R2 = 1 - RSS/TSS."""
3      rss = RSS(y, y_hat)
4      tss = TSS(y)
5      if tss == 0:
6          raise ValueError(
7              "TSS = 0 (y không có biến thiên), R2 không xác
8              định."
9          )
10     r_square = 1.0 - rss / tss    # công thức định nghĩa R2
11
12     # R2 phải nằm trong [0, 1]
13     if r_square < 0.0 or r_square > 1.0:
14         raise ValueError(f"r_square ({r_square}) không hợp lệ."
15     )
16     return r_square

```

Listing 2: Hàm R_squared trong model_metrics.py

B.3 R^2 Hiệu Chính (Adjusted R^2)

B.3.1. Ý nghĩa và công thức

Định nghĩa 4.4 (Adjusted R^2). $\bar{R}^2$ hiệu chỉnh R^2 theo số tham số, cho phép so sánh công bằng giữa các mô hình có độ phức tạp khác nhau:

$$\bar{R}^2 = 1 - \frac{n-1}{n-p-1} (1 - R^2).$$

Ý nghĩa: Hệ số $\frac{n-1}{n-p-1} > 1$ tăng khi p tăng. Nếu thêm biến không đóng góp thực sự, giảm RSS không đáng kể trong khi hệ số phạt tăng, nên $\bar{R}^2$ có thể *giảm*. Đây là ưu điểm so với R^2 thông thường.

$\bar{R}^2$ có thể âm nếu mô hình kém hơn đường ngang $\hat{y} = \bar{y}$.

B.3.2. Ký hiệu

- n : số quan sát.
- p : số biến giải thích (không tính hệ số chặn β_0).
- $n-1$: bậc tự do toàn phần.
- $n-p-1$: bậc tự do của phần dư.
- $\frac{n-1}{n-p-1}$: hệ số phạt — lớn hơn 1 và tăng khi thêm biến.

B.3.3. Mã giả

Algorithm 3 Tính Adjusted R^2

Input: $r^2 \in [0, 1]$, số quan sát n , số biến p

Output: $\bar{R}^2 \in (-\infty, 1]$

1: **return** $1 - \frac{n-1}{n-p-1} (1 - r^2)$

B.3.4. Phân tích độ phức tạp

- **Thời gian:** $O(1)$ — phép tính số học hằng.
- **Bộ nhớ:** $O(1)$.

B.3.5. Cài đặt thuật toán

```

1  def adjusted_R_squared(r2: float, n: int, p: int) -> float:
2      """Adjusted R2 = 1 - (n-1)/(n-p-1) * (1-R2)."""
3      # He so (n-1)/(n-p-1) > 1 phat mo hinh khi so bien p tang.
4      # Neu them bien khong co y nghia, R2 cai thien rat it
5      # trong khi he so phat tang => Adjusted R2 co the giam.
6      return 1.0 - (n - 1) / (n - p - 1) * (1.0 - r2)

```

Listing 3: Hàm adjusted_R_squared trong model_metrics.py

B.4 Thống Kê F (F-statistic)

B.4.1. Ý nghĩa và công thức

Định nghĩa 4.5 (F-statistic). Thống kê F kiểm định giả thuyết $H_0 : \beta_1 = \dots = \beta_p = 0$:

$$F = \frac{\text{ESS}/p}{\text{RSS}/(n-p-1)} = \frac{(\text{TSS} - \text{RSS})/p}{\text{RSS}/(n-p-1)}.$$

Dưới H_0 và giả thiết nhiễu Gaussian (GM5): $F \sim F_{p, n-p-1}$.

Ý nghĩa: Tử số đo biến thiên trung bình mô hình giải thích được trên mỗi bậc tự do. Mẫu số ước lượng phương sai nhiễu $\hat{\sigma}^2 = \text{RSS}/(n-p-1)$. Khi F lớn và p -value nhỏ, bác bỏ H_0 : ít nhất một $\beta_j \neq 0$.

B.4.2. Ký hiệu

- $\text{ESS} = \text{TSS} - \text{RSS}$: phần biến thiên mô hình giải thích được.
- p : bậc tự do của tử số (số biến giải thích).
- $n - p - 1$: bậc tự do của mẫu số (phần dư).
- $\hat{\sigma}^2 = \text{RSS}/(n-p-1)$: ước lượng không chệch của phương sai nhiễu.

B.4.3. Mã giả

Algorithm 4 Tính F-statistic

Input: $RSS > 0$, TSS , số quan sát n , số biến p

Output: Giá trị $F > 0$

```

1: if  $RSS = 0$  then
2:   raise ValueError                                ▷ mô hình khớp hoàn hảo,  $F$  không xác định
3: end if
4: if  $n \leq p + 1$  then
5:   raise ValueError                                ▷ cần  $n - p - 1 > 0$ 
6: end if
7:  $F \leftarrow \frac{(TSS - RSS)/p}{RSS/(n - p - 1)}$ 
8: return  $F$ 

```

B.4.4. Phân tích độ phức tạp

- **Thời gian:** $O(1)$ — phép chia vô hướng sau khi có RSS , TSS .
- **Bộ nhớ:** $O(1)$.
- **Tiên quyết:** cần $n > p + 1$ để mẫu số $n - p - 1 > 0$.

B.4.5. Cài đặt thuật toán

```

1  def F_statistic(rss: float, tss: float, n: int, p: int) ->
    float:
2      """F = (ESS/p) / (RSS/(n-p-1))."""
3      if rss == 0:
4          raise ValueError("RSS = 0, F-statistic không xác định.
5      if n <= p + 1:
6          raise ValueError(
7              "Cần n > p + 1 để F-statistic có bậc tự do dương."
8          )
9
10     # ESS = TSS - RSS: phần biến thiên mô hình giải thích được
11     # Chia theo bậc tự do tương ứng để so sánh hai nguồn biến
    thien.
12     f_stat = ((tss - rss) / p) / (rss / (n - p - 1))
13     return f_stat

```

Listing 4: Hàm F_statistic trong model_metrics.py

B.5 Hàm Tổng Hợp `model_metrics`

B.5.1. Ý nghĩa

`model_metrics` là điểm vào duy nhất để tính toàn bộ các chỉ số trong một lần gọi. Hàm sẽ kiểm tra tính hợp lệ của đầu vào, sau đó gọi tuần tự các hàm $RSS \rightarrow TSS \rightarrow adjusted_R_squared \rightarrow F_statistic$ để tính toán và cuối cùng trả về một dict chứa kết quả.

B.5.2. Mã giả

Algorithm 5 Hàm tổng hợp `model_metrics`

Input: Vector y , vector $\hat{y}$, số biến $p \in \mathbb{Z}^+$

Output: dict $\{RSS, TSS, R^2, \bar{R}^2, F\}$

- 1: Kiểm tra $|y| = |\hat{y}|$
 - 2: Kiểm tra $p \geq 1$ và $n > p + 1$
 - 3: $RSS \leftarrow RSS(y, \hat{y})$
 - 4: $TSS \leftarrow TSS(y)$
 - 5: $R^2 \leftarrow 1 - RSS/TSS$
 - 6: $\bar{R}^2 \leftarrow ADJUSTEDR2(R^2, n, p)$
 - 7: $F \leftarrow FSTATISTIC(RSS, TSS, n, p)$
 - 8: **return** $\{RSS, TSS, R^2, \bar{R}^2, F\}$
-

B.5.3. Phân tích độ phức tạp

- **Thời gian:** $O(n)$ — RSS và TSS mỗi cái $O(n)$; các bước còn lại $O(1)$.
- **Bộ nhớ:** $O(1)$ — không cấp phát mảng trung gian.

B.5.4. Cài đặt thuật toán

```

1  def model_metrics(
2      y: list[float], y_hat: list[float], p: int
3  ) -> Dict[str, float]:
4      """
5      Tính các chỉ số đánh giá mô hình OLS:
6      RSS, TSS, R2, Adjusted R2, F-statistic.
7      """
8      if len(y) != len(y_hat):
9          raise ValueError("y và y_hat phải có cùng số phần tử.")
10     if not isinstance(p, int) or p < 1:
11         raise ValueError("p phải là số nguyên dương.")
12
13     n = len(y)
14     if n <= p + 1:
15         raise ValueError(
16             "Can n > p + 1 để F-statistic có bậc tự do dương."
17         )
18
19     rss = RSS(y, y_hat)                # tổng bình phương phần dư
20     tss = TSS(y)                       # tổng bình phương toàn
21     # phần
22
23     r2 = 1.0 - rss / tss                # hệ số xác định R2
24     adj_r2 = adjusted_R_squared(r2, n, p) # R2 hiệu chỉnh
25     f_stat = F_statistic(rss, tss, n, p) # thống kê F
26
27     return {
28         "RSS": rss,
29         "TSS": tss,
30         "R2": r2,
31         "Adjusted_R2": adj_r2,
32         "F_statistic": f_stat,
33     }

```

Listing 5: Hàm model_metrics trong model_metrics.py

B.6 Hàm Suy Diễn Cấp Hệ Số `coef_inference`

B.6.1. Ý nghĩa

`coef_inference(X, y, beta_hat, sigma2)` — Tính standard errors, t -statistics, p -values và khoảng tin cậy 95%.

Hàm này cung cấp các chỉ số thống kê chi tiết cho từng hệ số hồi quy $\hat{\beta}_j$, giúp đánh giá mức độ đóng góp và ý nghĩa thống kê của từng biến độc lập trong mô hình.

B.6.2. Mã giả

Algorithm 6 Hàm suy diễn hệ số `coef_inference`

Input: Ma trận X , vector y , vector hệ số $\hat{\beta}$, phương sai nhiễu $\hat{\sigma}^2$

Output: Các danh sách: SE , t -stat, p -value, $CI_{95\%}$

```

1:  $C \leftarrow (X^T X)^{-1}$ 
2: for  $j \leftarrow 0$  to  $p$  do
3:    $SE_j \leftarrow \sqrt{\hat{\sigma}^2 C_{jj}}$ 
4:    $t_j \leftarrow \hat{\beta}_j / SE_j$ 
5:    $p\_value_j \leftarrow 2 \times (1 - \text{CDF}_t(|t_j|, n - p - 1))$ 
6:    $CI_j \leftarrow [\hat{\beta}_j - t_{0.025} \cdot SE_j, \hat{\beta}_j + t_{0.025} \cdot SE_j]$ 
7: end for
8: return  $\{SE, t, p\_value, CI\}$ 

```

B.6.3. Phân tích độ phức tạp

- **Thời gian:** $O(np^2 + p^3)$ — chủ yếu do phép nhân ma trận $X^T X$ và phép nghịch đảo.
- **Bộ nhớ:** $O(p^2)$ — lưu trữ ma trận hiệp phương sai $(X^T X)^{-1}$.

B.6.4. Cài đặt thuật toán

```

1 def coef_inference(X, y, beta_hat, sigma2):
2     n = len(X)
3     k = len(X[0])
4     df = n - k
5
6     XT = transpose(X)
7     XTX = matmul(XT, X)
8     XTX_inv = get_inverse(XTX)
9
10    std_errors = []
11    t_stats = []
12    p_values = []
13    conf_intervals = []
14
15    t_crit = stats.t.ppf(0.975, df)
16
17    for j in range(k):
18        se = math.sqrt(sigma2 * XTX_inv[j][j])
19        std_errors.append(se)
20
21        t_val = beta_hat[j] / se
22        t_stats.append(t_val)
23
24        p_val = stats.t.sf(abs(t_val), df) * 2
25        p_values.append(p_val)
26
27        ci_lower = beta_hat[j] - t_crit * se
28        ci_upper = beta_hat[j] + t_crit * se
29        conf_intervals.append((ci_lower, ci_upper))
30
31    return {
32        "std_errors": std_errors,
33        "t_stats": t_stats,
34        "p_values": p_values,
35        "conf_intervals": conf_intervals
36    }

```

Listing 6: Hàm coef_inference mô phỏng

B.7 Luồng Tính Toán Tổng Thể

Quan hệ phụ thuộc giữa các chỉ số:

$$(y, \hat{y}) \xrightarrow{\text{RSS}} \text{RSS} \xrightarrow{+ \text{TSS}} \frac{\text{RSS}}{\text{TSS}} \xrightarrow{1-} R^2 \xrightarrow{\text{hiệu chỉnh df}} \bar{R}^2$$

$$(\text{RSS}, \text{TSS}) \xrightarrow{\text{chia bậc tự do}} F$$

Thứ tự gọi hàm trong model_metrics:

- $\text{RSS} \rightarrow \text{TSS} \rightarrow R^2 = 1 - \text{RSS}/\text{TSS} \rightarrow \text{adjusted_R_squared} \rightarrow F_statistic$.

B.8 Bảng Kết Quả Thực Nghiệm

Bảng 1: So sánh chỉ số đánh giá mô hình: cài đặt từ đầu vs. statsmodels

Chỉ số	Scratch	statsmodels	$ \Delta $	Độ chính xác (%)
RSS	302.705315	302.705315	$\sim 10^{-10}$	100.0000
TSS	9676.090054	9676.090054	$\sim 10^{-10}$	100.0000
R^2	0.968716	0.968716	$\sim 10^{-10}$	100.0000
$\bar{R}^2$	0.967739	0.967739	$\sim 10^{-10}$	100.0000
F -statistic	990.892122	990.892122	$\sim 10^{-10}$	100.0000

Bảng 2: Kết quả model metrics trên hai bộ dữ liệu

Chỉ số	Không đa cộng tuyến	Đa cộng tuyến mạnh	Nhận xét
R^2	0.9687	0.9962	Cả hai đều có độ phù hợp rất cao
$\bar{R}^2$	0.9677	0.9961	Sai khác nhỏ so với R^2
F -statistic	990.8921	25575.6561	Mô hình có ý nghĩa thống kê tổng thể rất mạnh

C Đa Cộng Tuyến và Variance Inflation Factor (VIF)

C.1 Đa Cộng Tuyến (Multicollinearity)

Định nghĩa 4.6 (Đa cộng tuyến). Đa cộng tuyến xảy ra khi tồn tại tương quan tuyến tính cao giữa hai hay nhiều cột của ma trận thiết kế X , tức tồn tại xấp xỉ:

$$X_j \approx \sum_{k \neq j} c_k X_k$$

với một số hệ số c_k . Trường hợp cực đoan là đa cộng tuyến hoàn hảo, khiến $X^T X$ suy biến và nghiệm OLS không xác định duy nhất.

Hệ quả khi đa cộng tuyến mạnh:

- $X^T X$ gần suy biến; nghịch đảo kém ổn định về số.
- $\text{Var}(\hat{\beta}_j)$ tăng lớn; ước lượng hệ số không đáng tin.
- Kiểm định t cho từng β_j mất ý nghĩa dù F -statistic vẫn lớn.
- Kết quả mô hình nhạy cảm với nhiễu nhỏ trong dữ liệu.

C.2 Variance Inflation Factor (VIF)

C.2.1. Ý nghĩa và công thức

Định nghĩa 4.7 (VIF). Với biến X_j ($j = 1, \dots, p$), ký hiệu X_{-j} là ma trận thiết kế sau khi loại cột j (giữ nguyên cột hằng số). Gọi R_j^2 là R^2 của hồi quy phụ $X_j \sim X_{-j}$. Khi đó:

$$\text{VIF}_j = \frac{1}{1 - R_j^2}.$$

C.2.2. Ký hiệu

- X_j : cột j của X — biến cần đánh giá mức độ đa cộng tuyến.
- X_{-j} : ma trận thiết kế sau khi loại cột j , bao gồm cả cột hằng số ở vị trí 0.
- Hồi quy phụ $X_j \sim X_{-j}$: dự đoán X_j từ tất cả biến còn lại bằng OLS.
- $\hat{X}_j = X_{-j}\hat{\gamma}$: giá trị dự đoán của X_j trong hồi quy phụ.
- R_j^2 : R^2 của hồi quy phụ — đo X_j được giải thích bởi X_{-j} bao nhiêu.
- $1 - R_j^2$: phần X_j không được giải thích = phần thông tin độc lập.
- $VIF_j = 1/(1 - R_j^2)$: nghịch đảo của phần thông tin độc lập đó.

C.2.3. Ngưỡng diễn giải

Bảng 3: Ngưỡng VIF và mức độ đa cộng tuyến

Giá trị VIF	Mức độ	Khuyến nghị
≈ 1	Không có đa cộng tuyến	Chấp nhận
$1 < VIF_j \leq 5$	Trung bình	Theo dõi
$5 < VIF_j \leq 10$	Đáng lo ngại	Xem xét loại biến
$VIF_j > 10$	Nghiêm trọng	Loại biến hoặc dùng Ridge/Lasso
$VIF_j \rightarrow \infty$	Đa cộng tuyến hoàn hảo	X_j phụ thuộc tuyến tính hoàn toàn

C.3 Thuật Toán và Mã Nguồn VIF

C.3.1. Luồng thuật toán

Với ma trận thiết kế $X \in \mathbb{R}^{n \times (p+1)}$ (cột 0 toàn số 1), thuật toán lặp trên từng cột $j = 1, \dots, p$ (bỏ cột 0):

1. **Trích biến mục tiêu:** lấy cột j làm vector phản hồi $X_j = [x_{1j}, \dots, x_{nj}]^T$.
2. **Xây dựng X_{-j} :** loại cột j khỏi X để được $X_{-j} \in \mathbb{R}^{n \times p}$.
3. **Giải OLS phụ $X_j \sim X_{-j}$:** tính $\hat{\gamma} = \arg \min_{\gamma} \|X_j - X_{-j}\gamma\|^2$.
4. **Tính dự đoán:** $\hat{X}_j = X_{-j}\hat{\gamma}$ (nhân ma trận-vector, hàm `mat_vec_mult`).
5. **Tính R_j^2 :** gọi `model_metrics( $X_j, \hat{X}_j, p-2$ )` và lấy trường "R2". Tham số bậc tự do là $p-2$: trừ đi X_j và cột hằng số.
6. **Suy ra VIF_j :** $VIF_j = 1/(1 - R_j^2)$ nếu $R_j^2 < 1$; ngược lại $VIF_j = +\infty$.

C.3.2. Mã giả

Algorithm 7 Tính Variance Inflation Factor (VIF)

Input: Ma trận thiết kế $X \in \mathbb{R}^{n \times (p+1)}$ (cột 0 là hằng số)

Output: Vector $[VIF_1, \dots, VIF_p]$

```

1:  $p \leftarrow$  số cột của  $X$ 
2:  $vifs \leftarrow []$ 
3: for  $j \leftarrow 1$  to  $p - 1$  do
    // Bước 1: Trích cột  $j$ 
4:    $X_j \leftarrow [X[i][j] \mid i = 1, \dots, n]$ 
    // Bước 2: Xây dựng ma trận hồi quy phụ
5:    $X_{-j} \leftarrow [[X[i][k] \mid k \neq j] \mid i = 1, \dots, n]$ 
    // Bước 3: Giải OLS phụ  $X_j \sim X_{-j}$ 
6:    $\hat{\gamma} \leftarrow \text{OLSFit}(X_{-j}, X_j)[\text{'beta'}]$ 
    // Bước 4: Tính giá trị dự đoán  $\hat{X}_j$ 
7:    $\hat{X}_j \leftarrow \text{MATVECMULT}(X_{-j}, \hat{\gamma})$ 
    // Bước 5: Tính  $R_j^2$  — bậc tự do là  $p - 2$ 
8:    $\text{data} \leftarrow \text{MODELMETRICS}(X_j, \hat{X}_j, p - 2)$ 
9:    $R_j^2 \leftarrow \text{data}[\text{'R2'}]$ 
    // Bước 6: Suy ra  $VIF_j$ 
10:  if  $R_j^2 < 1$  then
11:     $VIF_j \leftarrow 1/(1 - R_j^2)$ 
12:  else
13:     $VIF_j \leftarrow +\infty$ 
14:  end if
15:   $vifs.\text{APPEND}(VIF_j)$ 
16: end for
17: return  $vifs$ 

```

C.3.3. Phân tích độ phức tạp

Gọi n là số quan sát, p là số biến giải thích. Mỗi lần lặp j :

- Xây dựng X_{-j} : $O(np)$.
- Giải OLS phụ (`numpy.linalg.lstsq` qua SVD): $O(np^2)$.
- Nhân ma trận–vector (`mat_vec_mult`): $O(np)$.
- Tính `model_metrics`: $O(n)$.

Lặp p lần, tổng độ phức tạp:

$$O(p \cdot np^2) = O(np^3).$$

- **Thời gian:** $O(np^3)$ khi $n \gg p$ (trường hợp phổ biến).
- **Bộ nhớ:** $O(np)$ — lưu ma trận X_{-j} tạm thời mỗi lần lặp.

C.3.4. Cài đặt thuật toán

```

1  from hau_utils import demo_ols_fit, mat_vec_mult
2  from model_metrics import model_metrics
3  def VIF(X: list[list[float]]) -> list[float]:
4      # Tong so cot cua X (bao gom cot hang so o vi tri 0)
5      p = len(X[0])
6      vifs = []
7      for j in range(1, p):
8
9          # Buoc 1: Trich cot j lam bien phan hoi cua hoi quy phu
10         X_j = [row[j] for row in X]
11
12         # Buoc 2: Xay dung X_{-j} (giu tat ca cot tru cot j)
13         X_minus_j = [
14             [row[k] for k in range(p) if k != j]
15             for row in X
16         ]
17
18         # Buoc 3: Giai OLS phu X_j ~ X_{-j}
19         beta = demo_ols_fit(X_minus_j, X_j)["beta"]
20
21         # Buoc 4: Tinh xap xi X_hat_j = X_{-j} * beta
22         X_j_hat = mat_vec_mult(X_minus_j, beta)
23
24         # Buoc 5: Tinh R2_j qua model_metrics
25         data = model_metrics(X_j, X_j_hat, p - 2)
26
27         # Buoc 6: Suy ra VIF_j = 1 / (1 - R2_j)
28         vif_j = (
29             1.0 / (1.0 - data["R2"])
30             if data["R2"] < 1.0
31             else float("inf")
32         )
33         vifs.append(vif_j)
34
35     return vifs

```

Listing 7: Hàm VIF trong VIF.py

C.3.5. Hàm hỗ trợ

```

1  def mat_vec_mult(A, v) -> list[float]:
2      """Nhan ma tran A (m x n) voi vector v (n x 1)."""
3      return [
4          sum(A[i][j] * v[j] for j in range(len(v)))
5          for i in range(len(A))
6      ]
7
8
9  def demo_ols_fit(
10     X: list[list[float]], y: list[float]
11 ) -> Dict[str, float]:
12     """Ham ols_fit dung numpy de giai nhanh phuc vu tinh VIF."
13     ""
14     X_np = np.array(X)
15     y_np = np.array(y)
16     # np.linalg.lstsq tra ve (beta, residuals, rank, s)
17     beta, _, _, _ = np.linalg.lstsq(X_np, y_np, rcond=None)
18     return {"beta": beta.tolist()}

```

Listing 8: Các hàm hỗ trợ trong hau_utils.py

C.4 Bảng Kết Quả Thực Nghiệm

Bảng 4: VIF trên bộ dữ liệu không có đa cộng tuyến

Biến	R_j^2 (hồi quy phụ)	VIF (scratch)	VIF (statsmodels)
X_1	0.0187	1.0190	1.0190
X_2	0.0194	1.0198	1.0198
X_3	0.0314	1.0324	1.0324

Bảng 5: VIF trên bộ dữ liệu có đa cộng tuyến mạnh ($x_2 \approx 2x_1 + \varepsilon$)

Biến	R_j^2	VIF (scratch)	VIF (statsmodels)	$ \Delta $
X_1	0.999472	1894.5416	1894.5416	$\sim 10^{-10}$
X_2	0.999472	1894.5416	1894.5416	$\sim 10^{-10}$

D Ridge Regression và K-Fold Cross-Validation

D.1 Giới thiệu

Phần này trình bày chi tiết cơ sở toán học, cách chứng minh, mã nguồn cài đặt và kết quả thực nghiệm cho hai nội dung chính: Ridge Regression và K-Fold Cross-Validation. Các thuật toán được xây dựng chủ yếu bằng Python thuần thông qua các thao tác đại số tuyến tính cơ bản như chuyển vị ma trận, nhân ma trận, nhân ma trận với vector, cộng ma trận và nghịch đảo ma trận bằng khử Gauss–Jordan.

D.2 Bộ dữ liệu giả lập

Trong phần này, nhóm tiến hành sinh ngẫu nhiên một bộ dữ liệu giả định bao gồm $n = 100$ quan sát và $p = 3$ đặc trưng độc lập. Việc khởi tạo được kiểm soát bởi hàm `np.random.seed(42)` nhằm đảm bảo mọi kết quả thực nghiệm phía sau đều có thể tái lập được một cách chính xác. Ma trận đặc trưng ban đầu được ký hiệu là X với kích thước 100×3 . Trong trường hợp cần tính toán hệ số chặn, nhóm sẽ bổ sung thêm một cột chứa toàn bộ giá trị 1 vào bên trái của X , qua đó thiết lập nên ma trận thiết kế hoàn chỉnh như sau:

$$X_{\text{plus1}} = \begin{bmatrix} 1 & x_{11} & x_{12} & x_{13} \\ 1 & x_{21} & x_{22} & x_{23} \\ \vdots & \vdots & \vdots & \vdots \\ 1 & x_{100,1} & x_{100,2} & x_{100,3} \end{bmatrix} \in \mathbb{R}^{100 \times 4}.$$

Vector hệ số thật được đặt là

$$\beta^* = \begin{bmatrix} 5.0 & 10.0 & -5.0 & 0.5 \end{bmatrix}^T.$$

Biến mục tiêu được sinh theo mô hình tuyến tính có nhiễu:

$$y = X_{\text{plus1}}\beta^* + \varepsilon,$$

trong đó $\varepsilon \sim \mathcal{N}(0, 2^2)$ là nhiễu Gaussian với độ lệch chuẩn bằng 2.

Đoạn mã sinh dữ liệu chính như sau:

```
1 np.random.seed(42)
2 n = 100
3 p = 3
4
5 X = np.random.randn(n, p)
6 X_plus1 = np.c_[np.ones(n), X]
7 true_beta = np.array([5.0, 10.0, -5.0, 0.5])
8 noise = np.random.randn(n) * 2
9 y = X_plus1 @ true_beta + noise
10 lambdas = np.logspace(-2, 4, 200)
```

D.3 Ridge Regression

D.3.1. Hồi quy tuyến tính thường và vấn đề đặt ra

Với dữ liệu huấn luyện (X, y) , hồi quy tuyến tính thường, hay Ordinary Least Squares (OLS), tìm vector hệ số β sao cho tổng bình phương sai số là nhỏ nhất:

$$\hat{\beta}_{\text{OLS}} = \arg \min_{\beta} \|y - X\beta\|_2^2.$$

Nếu $X^T X$ khả nghịch, nghiệm OLS là

$$\hat{\beta}_{\text{OLS}} = (X^T X)^{-1} X^T y.$$

Tuy nhiên, OLS có thể gặp vấn đề khi các đặc trưng tương quan mạnh, tức có hiện tượng đa cộng tuyến. Khi đó $X^T X$ gần suy biến, nghiệm có phương sai lớn, hệ số hồi quy không ổn định và dễ bị ảnh hưởng bởi nhiễu. Ridge Regression được dùng để làm ổn định nghiệm bằng cách thêm thành phần phạt L_2 .

D.3.2. Hàm mục tiêu

Ridge Regression bổ sung thành phần chính quy hóa L_2 vào hàm mất mát:

$$\hat{\beta}_{\text{ridge}} = \arg \min_{\beta} \left\{ \|y - X\beta\|_2^2 + \lambda \|\beta\|_2^2 \right\},$$

trong đó $\lambda \geq 0$ là tham số điều chỉnh cường độ phạt. Khi $\lambda = 0$, Ridge trở về OLS. Khi λ càng lớn, các hệ số càng bị co về gần 0.

Trong thực tế, hệ số chặn β_0 thường không bị phạt. Vì vậy, thay vì dùng ma trận đơn vị đầy đủ, nhóm dùng ma trận

$$D = \text{diag}(0, 1, 1, \dots, 1).$$

Khi đó bài toán trong mã nguồn tương ứng với:

$$\hat{\beta}_{\text{ridge}} = \arg \min_{\beta} \left\{ \|y - X\beta\|_2^2 + \lambda \beta^T D \beta \right\}.$$

D.3.3. Chứng minh nghiệm đóng

Xét hàm mục tiêu:

$$J(\beta) = (y - X\beta)^T (y - X\beta) + \lambda \beta^T D \beta.$$

Khai triển:

$$J(\beta) = y^T y - 2\beta^T X^T y + \beta^T X^T X \beta + \lambda \beta^T D \beta.$$

Lấy đạo hàm theo β :

$$\nabla_{\beta} J(\beta) = -2X^T y + 2X^T X \beta + 2\lambda D \beta.$$

Tại điểm cực tiểu, đặt đạo hàm bằng 0:

$$-2X^T y + 2X^T X \beta + 2\lambda D \beta = 0.$$

Chia hai vế cho 2 và chuyển vế:

$$(X^T X + \lambda D) \beta = X^T y.$$

Nếu ma trận $X^T X + \lambda D$ khả nghịch, nghiệm là

$$\boxed{\hat{\beta}_{\text{ridge}} = (X^T X + \lambda D)^{-1} X^T y.}$$

Khi phạt cả hệ số chặn, $D = I$ và công thức trở thành dạng quen thuộc:

$$\hat{\beta}_{\text{ridge}} = (X^T X + \lambda I)^{-1} X^T y.$$

D.3.4. Ý nghĩa của tham số λ

Tham số λ điều khiển mức độ đánh đổi giữa độ khớp dữ liệu và độ lớn của hệ số:

- $\lambda = 0$: mô hình không bị phạt, tương đương OLS.
- λ nhỏ: hệ số gần với nghiệm OLS, mô hình có độ chệch thấp nhưng phương sai có thể cao.
- λ lớn: hệ số bị co mạnh về 0, mô hình đơn giản hơn, phương sai giảm nhưng độ chệch tăng.

D.3.5. Cài đặt Ridge Regression

File `ridge_lasso.py` tự cài đặt các phép toán đại số tuyến tính thay vì dựa trực tiếp vào `numpy.linalg`. Các hàm chính gồm:

- `transpose(mat)`: chuyển vị ma trận.
- `mat_mult(A,B)`: nhân hai ma trận.
- `mat_vec_mult(A,v)`: nhân ma trận với vector.
- `mat_add(A,B)`: cộng hai ma trận cùng kích thước.
- `scalar_mult(mat, scalar)`: nhân ma trận với số vô hướng.
- `eye(n)`: tạo ma trận đơn vị cấp n .
- `invert_matrix(M)`: tính ma trận nghịch đảo bằng khử Gauss–Jordan.
- `ridge_fit(X,y,lam,fit_intercept=True)`: tính nghiệm Ridge, có thể tự thêm hệ số chặn.

Mã nguồn Ridge

```

1 def ridge_fit(X, y, lam, fit_intercept=True):
2     if fit_intercept:
3         X_design = [[1.0] + row for row in X]
4     else:
5         X_design = [row[:] for row in X]
6
7     n_cols = len(X_design[0])
8     I = eye(n_cols)
9
10    if fit_intercept:
11        # Không phạt intercept
12        I[0][0] = 0.0
13
14    lam_I = scalar_mult(I, lam)
15
16    X_T = transpose(X_design)
17    X_T_X = mat_mult(X_T, X_design)
18
19    A = mat_add(X_T_X, lam_I)
20    A_inv = invert_matrix(A)
21    X_T_y = mat_vec_mult(X_T, y)
22
23    beta_ridge = mat_vec_mult(A_inv, X_T_y)
24    return beta_ridge

```

Đoạn mã này đúng với công thức:

$$\hat{\beta}_{\text{ridge}} = (X^T X + \lambda D)^{-1} X^T y,$$

trong đó tham số `fit_intercept=True` cho phép hàm tự thêm cột intercept vào dữ liệu đặc trưng, còn dòng `I[0][0] = 0.0` bảo đảm không phạt hệ số chặn.

D.3.6. Ridge Trace

Ridge Trace là biểu đồ biểu diễn sự thay đổi của các hệ số $\hat{\beta}_j$ theo λ . Trong khi test, các giá trị λ được sinh bằng:

$$\lambda \in \{10^{-2}, \dots, 10^4\},$$

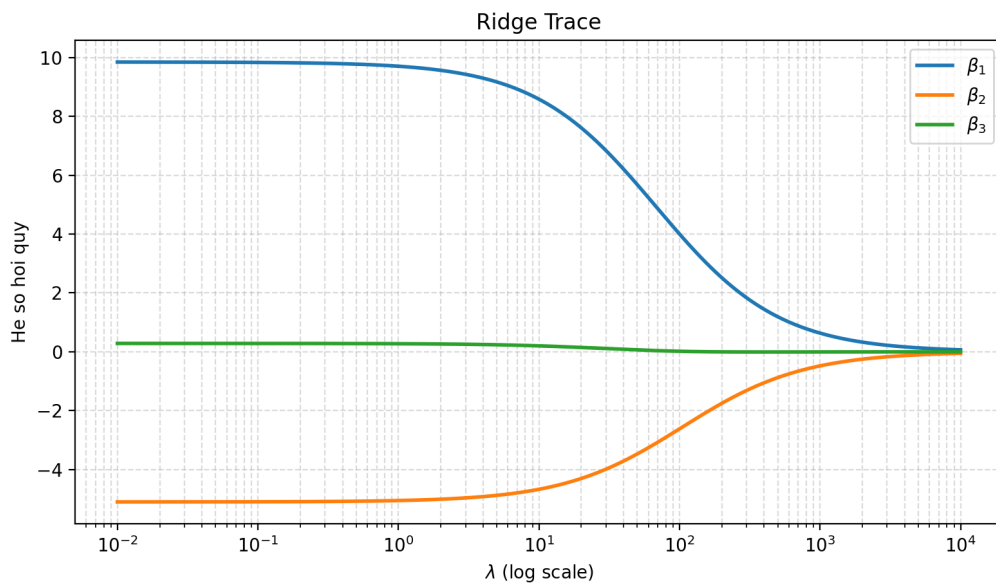
với 200 điểm chia đều trên thang logarit.

Hàm `plot_ridge_trace` lần lượt huấn luyện Ridge với từng giá trị λ , lưu các hệ số và vẽ các đường $\beta_1, \beta_2, \dots, \beta_p$ trên trục hoành logarit. Hệ số chặn không được vẽ vì không bị phạt.

```

1 def plot_ridge_trace(X, y, lambdas, fit_intercept=True):
2     coefs = []
3     for lam in lambdas:
4         beta_ridge = ridge_fit(X, y, lam, fit_intercept=
fit_intercept)
5         coefs.append(beta_ridge)
6
7     coefs = np.array(coefs)
8     plt.figure(figsize=(10, 6))
9     num_features = len(X[0])
10
11     if fit_intercept:
12         for j in range(1, num_features + 1):
13             plt.plot(lambdas, coefs[:, j], label=f'$\\beta_{j}$')
14     else:
15         for j in range(num_features):
16             plt.plot(lambdas, coefs[:, j], label=f'$\\beta_{j+1}$')
17
18     plt.xscale('log')
19     plt.xlabel('$\\lambda$ (log scale)', fontsize=12)
20     plt.ylabel('Coefficients (He so hoi quy)', fontsize=12)
21     plt.title('Ridge Trace', fontsize=14, fontweight='bold')
22     plt.legend()
23     plt.grid(True, which="both", ls="--", alpha=0.6)
24     plt.show()

```



Hình 1: Kết quả Ridge Trace trên bộ dữ liệu giả lập.

Nhận xét từ Ridge Trace:

- Khi $\lambda \rightarrow 0$, các hệ số gần với nghiệm OLS.
- Khi λ tăng, các hệ số bị co dần về 0.
- Ridge không đưa hệ số chính xác về 0 như Lasso, mà thường chỉ làm giảm độ lớn hệ số.
- Biểu đồ cho thấy trực quan tác động của chính quy hóa lên độ phức tạp của mô hình.
- Đây là ví dụ điển hình của đánh đổi bias–variance: Ridge chấp nhận tăng một phần bias để giảm variance, từ đó có thể cải thiện khả năng tổng quát hóa trên dữ liệu chưa thấy.

D.3.7. Kết quả thực nghiệm Ridge

Nhóm tiến hành kiểm tra nghiệm Ridge tự cài đặt tại $\lambda = 10$, sau đó đối chiếu kết quả với mô hình Ridge của thư viện `sklearn.linear_model`. Dưới cùng thiết lập cho phép tính hệ số chặn (intercept), nhóm thu được bảng so sánh sau:

Phương pháp	β_0	β_1	β_2	β_3
Tự cài đặt	5.4264	8.5817	-4.6711	0.2020
sklearn	5.4264	8.5817	-4.6711	0.2020

Kết quả kiểm tra bằng `np.allclose(custom_beta, sklearn_beta, atol=1e-5)` trả về True. Điều này cho thấy cài đặt thủ công của Ridge Regression là nhất quán với thư viện chuẩn.

D.4 K-Fold Cross-Validation

D.4.1. Ý tưởng và Công thức

K-Fold Cross-Validation (Kiểm chứng chéo K-phần) là phương pháp thống kê dùng để đánh giá khả năng tổng quát hóa của mô hình trên dữ liệu chưa biết (unseen data). Khác với phương pháp Hold-out (chỉ chia tập Train/Test một lần) dễ bị phụ thuộc vào cách chia ngẫu nhiên, K-Fold CV tận dụng toàn bộ tập dữ liệu cho cả quá trình huấn luyện và kiểm tra.

Tập dữ liệu $\mathcal{D}$ gồm n quan sát được phân hoạch thành k tập con (fold) có kích thước xấp xỉ nhau:

$$\mathcal{D} = F_1 \cup F_2 \cup \dots \cup F_k, \quad F_i \cap F_j = \emptyset \text{ nếu } i \neq j.$$

Với mỗi lần lặp $i = 1, 2, \dots, k$:

- Dùng tập F_i làm tập kiểm tra (validation set).
- Dùng phần dữ liệu còn lại $\mathcal{D} \setminus F_i$ làm tập huấn luyện (training set).
- Huấn luyện mô hình $\hat{f}^{-i}$ trên tập huấn luyện.
- Dự đoán trên tập kiểm tra và tính sai số.

Mean Squared Error (MSE) tại fold thứ i được tính bằng:

$$\text{MSE}_i = \frac{1}{|F_i|} \sum_{(x_r, y_r) \in F_i} (y_r - \hat{f}^{-i}(x_r))^2.$$

Điểm Cross-Validation trung bình là trung bình cộng của các sai số trên k fold:

$$CV_{(k)} = \frac{1}{k} \sum_{i=1}^k \text{MSE}_i.$$

D.4.2. Ý nghĩa Thống kê và Đánh đổi Bias - Variance

Mục tiêu cốt lõi của $CV_{(k)}$ là ước lượng **sai số dự đoán kỳ vọng** (Expected Prediction Error) của mô hình học. Tuy nhiên, chất lượng của ước lượng này phụ thuộc rất lớn vào việc lựa chọn tham số k , thể hiện qua sự đánh đổi giữa Độ chệch (Bias) và Phương sai (Variance):

- **Khi $k = n$ (Leave-One-Out Cross-Validation - LOOCV):** Mỗi fold chỉ chứa đúng 1 quan sát. Mô hình được huấn luyện trên $n - 1$ mẫu, gần bằng với kích thước của toàn bộ tập dữ liệu. Do đó, ước lượng CV có **độ chệch rất thấp** (gần như không chệch). Tuy nhiên, vì các tập huấn luyện giữa các lần lặp gần như giống hệt nhau (chỉ khác đúng 1 phần tử), các mô hình $\hat{f}^{-i}$ có độ tương quan (correlation) rất cao. Theo tính chất thống kê, trung bình của các đại lượng có tương quan cao sẽ dẫn đến **phương sai lớn**. Hơn nữa, chi phí tính toán của LOOCV là rất khổng lồ ($O(n)$ lần huấn luyện).
- **Khi k nhỏ (ví dụ $k = 2$ hoặc $k = 3$):** Kích thước tập huấn luyện lúc này chỉ bằng 50% hoặc 66% dữ liệu gốc. Mô hình có ít dữ liệu để học hơn so với thực tế, dẫn đến việc $CV_{(k)}$ có xu hướng ước lượng quá cao (pessimistic bias) sai số thực tế của mô hình (tức là **độ chệch cao**). Bù lại, do các tập huấn luyện ít giao nhau hơn, phương sai của ước lượng sẽ giảm.
- **Lựa chọn tối ưu ($k = 5$ hoặc $k = 10$):** Nhiều nghiên cứu thực nghiệm và lý thuyết (tiêu biểu là của Hastie, Tibshirani và Friedman) đã chỉ ra rằng $k = 5$ hoặc $k = 10$ mang lại sự cân bằng tốt nhất. Ở mức này, độ chệch được kiểm soát ở mức chấp nhận được (tập huấn luyện chiếm 80% – 90% dữ liệu), đồng thời các mô hình không bị tương quan quá mạnh, giúp phương sai của ước lượng sai số được giữ ở mức thấp.

D.4.3. Cài đặt K-Fold Cross-Validation

Mã nguồn trong file `cross_validation.py` cung cấp hàm `kfold_cv` chuyên đảm nhận việc kiểm chứng chéo. Thay vì sử dụng trực tiếp các hàm có sẵn, nhóm tự cài đặt thuật toán xáo trộn các chỉ số dữ liệu đầu vào theo một hạt giống (seed) cố định là 42. Quy trình này nhằm duy trì tuyệt đối tính minh bạch và khả năng tái lập. Kế tiếp, thuật toán sẽ tự động phân bổ đều tập dữ liệu gốc thành k danh sách con riêng biệt mang kích thước gần như tương đương nhau.

```

1 def kfold_cv(X, y, k, lam=0.0, fit_intercept=True):
2     n = len(X)
3     indicies = np.random.RandomState(42).permutation(n).tolist()
4
5     fold_sizes = [n // k + (1 if i < n % k else 0) for i in range(
6         k)]
7
8     folds = []
9     current_idx = 0
10
11     for size in fold_sizes:
12         folds.append(indicies[current_idx:current_idx + size])
13         current_idx += size
14
15     mse_list = []
16
17     for i in range(k):
18         test_idx = folds[i]
19         train_idx = []
20         for j in range(k):
21             if i != j:
22                 train_idx.extend(folds[j])
23         X_train = [X[idx] for idx in train_idx]
24         y_train = [y[idx] for idx in train_idx]
25         X_test = [X[idx] for idx in test_idx]
26         y_test = [y[idx] for idx in test_idx]
27
28         beta_hat = ridge_fit(X_train, y_train, lam, fit_intercept=
29             fit_intercept)
30         if fit_intercept:
31             X_test = [[1.0] + row for row in X_test]
32             y_pred = mat_vec_mult(X_test, beta_hat)
33
34         mse = sum((y_pred[idx] - y_test[idx]) ** 2
35             for idx in range(len(test_idx))) / len(y_test)
36         mse_list.append(mse)
37
38     cv_score = sum(mse_list) / k
39     return cv_score

```

Tham số `fit_intercept` giúp hàm K-Fold nhất quán với `ridge_fit`: nếu bằng `True`, mô hình tự thêm cột `intercept` khi huấn luyện và cũng thêm cột này vào tập kiểm tra trước khi dự đoán.

D.4.4. Kết quả thực nghiệm K-Fold

Để kiểm chứng tính chính xác của hàm `kfold_cv` vừa cài đặt, nhóm tiến hành đối chiếu kết quả với lớp `KFold` thuộc thư viện `sklearn.model_selection`. Quá trình thử nghiệm được thực hiện trên ba kịch bản phân chia dữ liệu khác nhau, tương ứng với $k = 3$, $k = 5$ và $k = 10$.

k	CV Score tự cài đặt	CV Score sklearn	Trùng khớp
3	3.4722	3.4722	True
5	3.3856	3.3856	True
10	3.3479	3.3479	True

Ở cả ba trường hợp, hàm `np.isclose` đều xác nhận sai số giữa hai kết quả tiệm cận mức 0 (trả về True). Điều này chứng minh các công đoạn từ xáo trộn chỉ số, chia fold, huấn luyện, dự đoán cho đến tính MSE trung bình trong cài đặt thủ công hoàn toàn nhất quán với thư viện `sklearn`. Trong kiểm chứng này, tham số λ được đặt bằng 0.0 nên mô hình tương đương với OLS.

D.5 So sánh mô hình bằng K-Fold CV

Ở đây, OLS được xem như trường hợp đặc biệt của Ridge khi $\lambda = 0$, sau đó so sánh với các mô hình Ridge có mức phạt khác nhau:

$$\lambda \in \{0.0, 0.1, 1.0, 10.0, 100.0\}.$$

Với mỗi giá trị λ , mô hình được đánh giá bằng 5-Fold CV trên dữ liệu dạng list, sử dụng `fit_intercept=True`, và tiêu chí lựa chọn là MSE trung bình nhỏ nhất. Khi kiểm thử, hàm được gọi bằng `compare_models_based_on_cv(X.tolist(), y.tolist(), k, lambdas_to_test, fit_intercept=True)`.

```

1 def compare_models_based_on_cv(X, y, k, lambdas_to_test,
  fit_intercept=True):
2     best_lam = None
3     best_cv_score = float('inf')
4
5     print(f"- SO SANH MO HINH BANG {k}-FOLD CV")
6     for lam in lambdas_to_test:
7         score = kfold_cv(X, y, k, lam, fit_intercept=fit_intercept
8         )
9         model_name = "OLS" if lam == 0 else "Ridge"
10        print(f"Model {model_name:5} (lambda = {lam:5.1f}) -> CV
11        Score (MSE): {score:.4f}")
12
13        if score < best_cv_score:
14            best_cv_score = score
15            best_lam = lam
16
17    print(f"Ket luan: lambda = {best_lam}, CV_Score = {
18    best_cv_score:.4f}")

```

D.5.1. Bộ dữ liệu không đa cộng tuyến

Trên bộ dữ liệu giả lập ban đầu, các đặc trưng được sinh độc lập từ phân phối chuẩn nên không chủ đích tạo đa cộng tuyến mạnh. Kết quả thực nghiệm với $k = 5$ như sau:

Mô hình	λ	CV Score
OLS	0.0	3.3856
Ridge	0.1	3.3882
Ridge	1.0	3.4371
Ridge	10.0	5.7466
Ridge	100.0	42.4193

Mô hình tốt nhất trong trường hợp này là OLS, tương đương Ridge với $\lambda = 0.0$, đạt CV Score bằng 3.3856. Điều này phù hợp với dữ liệu ban đầu không có đa cộng tuyến mạnh: thêm phạt Ridge không cải thiện sai số dự đoán, còn khi λ quá lớn thì bias tăng và CV Score xấu đi rõ rệt.

D.5.2. Bộ dữ liệu có đa cộng tuyến

Tạo thêm một bộ dữ liệu cổ tình có đa cộng tuyến bằng cách sinh x_2 và x_3 phụ thuộc mạnh vào x_1 :

$$x_2 = x_1 + \eta_2, \quad x_3 = x_1 - \eta_3,$$

trong đó η_2, η_3 là nhiễu nhỏ. Khi các đặc trưng gần phụ thuộc tuyến tính, nghiệm OLS thường kém ổn định hơn và Ridge có thể cải thiện khả năng tổng quát hóa.

Kết quả thực nghiệm với $k = 5$ như sau:

Mô hình	λ	CV Score
OLS	0.0	3.8871
Ridge	0.1	3.8628
Ridge	1.0	3.9074
Ridge	10.0	4.0284
Ridge	100.0	6.7309

Trong trường hợp có đa cộng tuyến, mô hình tốt nhất là Ridge với $\lambda = 0.1$, đạt CV Score bằng 3.8628. Kết quả này minh họa vai trò của Ridge Regression: một mức phạt nhỏ có thể làm giảm phương sai của mô hình và cải thiện sai số kiểm định; tuy nhiên, nếu λ quá lớn, hệ số bị co quá mạnh và MSE tăng lên.

D.6 Độ phức tạp

Giả sử ma trận đặc trưng ban đầu có p cột và sau khi xét `fit_intercept=True` thì ma trận thiết kế có $d = p + 1$ cột. Trong hàm `ridge_fit`:

- Tính $X^T X$ có độ phức tạp xấp xỉ $O(nd^2)$.
- Tính $X^T y$ có độ phức tạp $O(nd)$.
- Nghịch đảo ma trận $d \times d$ bằng Gauss–Jordan có độ phức tạp $O(d^3)$.
- Nhân ma trận nghịch đảo với vector có độ phức tạp $O(d^2)$.

Vì vậy, độ phức tạp chính của Ridge là:

$$O(nd^2 + d^3).$$

Với K-Fold Cross-Validation, mô hình được huấn luyện k lần. Mỗi lần dùng khoảng $\frac{k-1}{k}n$ mẫu huấn luyện. Do đó độ phức tạp tổng quát xấp xỉ:

$$O(k(nd^2 + d^3)),$$

trong đó ký hiệu bỏ qua hằng số $\frac{k-1}{k}$.

E Residual Diagnostic Plots cho mô hình OLS

E.1 Giới thiệu

Phần này trình bày chi tiết cơ sở toán học, cách cài đặt, kết quả thực nghiệm và nhận xét cho nhóm biểu đồ chẩn đoán phần dư (Residual Diagnostic Plots) trong mô hình hồi quy tuyến tính OLS. Trọng tâm của nhóm là phân tích bốn biểu đồ đã được xây dựng: Residuals vs Fitted Values, Normal Q-Q Plot, Scale-Location Plot và Cook's Distance.

Trong hồi quy tuyến tính, việc tìm được nghiệm OLS chưa đủ để kết luận mô hình tốt. Sau khi ước lượng hệ số, cần kiểm tra phần dư để đánh giá các giả định quan trọng như tính tuyến tính, đồng phương sai, phân phối chuẩn của sai số và sự tồn tại của các điểm có ảnh hưởng lớn. Vì vậy, residual plots đóng vai trò như một bước kiểm định trực quan giúp xác nhận mô hình có phù hợp với dữ liệu hay không.

E.2 Bộ dữ liệu giả lập

Dữ liệu được sinh ngẫu nhiên có kiểm soát: để bảo đảm kết quả có thể tái lập. Số quan sát là $n = 100$. Ma trận thiết kế X gồm ba cột: một cột hệ số chặn toàn số 1, biến X_1 và biến X_2 :

$$X = \begin{bmatrix} 1 & x_{11} & x_{12} \\ 1 & x_{21} & x_{22} \\ \vdots & \vdots & \vdots \\ 1 & x_{100,1} & x_{100,2} \end{bmatrix} \in \mathbb{R}^{100 \times 3}.$$

Hai biến giải thích được sinh theo phân phối chuẩn:

$$X_1 \sim \mathcal{N}(100, 20^2), \quad X_2 \sim \mathcal{N}(50, 15^2).$$

Vector hệ số thật được đặt là

$$\beta^* = \begin{bmatrix} 10 & 0.5 & -0.2 \end{bmatrix}^T.$$

Biến mục tiêu được sinh theo mô hình tuyến tính có nhiễu:

$$y = X\beta^* + \varepsilon,$$

trong đó $\varepsilon \sim \mathcal{N}(0, 5^2)$ là nhiễu Gaussian với độ lệch chuẩn bằng 5.

Đoạn mã sinh dữ liệu chính như sau:

```
1 n = 100
2 k = 3
3
4 np.random.seed(42)
5 X1 = np.random.normal(100, 20, n)
6 X2 = np.random.normal(50, 15, n)
7
8 X = np.column_stack([np.ones(n), X1, X2])
9
10 beta_true = np.array([10, 0.5, -0.2])
11
12 sigma_true = 5.0
13 epsilon = np.random.normal(0, sigma_true, n)
14
15 y = X @ beta_true + epsilon
```

E.3 Mô hình OLS

E.3.1. Hàm mục tiêu

Với dữ liệu (X, y) , Ordinary Least Squares tìm vector hệ số β sao cho tổng bình phương sai số là nhỏ nhất:

$$\hat{\beta}_{OLS} = \arg \min_{\beta} \|y - X\beta\|_2^2.$$

Trong đó, phần dư được định nghĩa là

$$e = y - \hat{y} = y - X\hat{\beta}.$$

Mục tiêu của OLS là chọn $\hat{\beta}$ sao cho tổng bình phương phần dư

$$RSS = \sum_{i=1}^n e_i^2$$

nhỏ nhất.

E.3.2. Chứng minh nghiệm đóng

Xét hàm mục tiêu:

$$J(\beta) = (y - X\beta)^T (y - X\beta).$$

Khai triển biểu thức:

$$J(\beta) = y^T y - 2\beta^T X^T y + \beta^T X^T X \beta.$$

Lấy đạo hàm theo β :

$$\nabla_{\beta} J(\beta) = -2X^T y + 2X^T X \beta.$$

Tại điểm cực tiểu, đặt đạo hàm bằng 0:

$$-2X^T y + 2X^T X \beta = 0.$$

Suy ra hệ phương trình chuẩn:

$$X^T X \beta = X^T y.$$

Nếu $X^T X$ khả nghịch, nghiệm OLS có dạng:

$$\boxed{\hat{\beta}_{OLS} = (X^T X)^{-1} X^T y.}$$

E.3.3. Cài đặt OLS trong mã nguồn

Trong file `ols_utils.py`, hàm `ols_fit` được sử dụng để tính nghiệm OLS và phương sai sai số ước lượng. Ý tưởng chính là tính $X^T X$, $X^T y$, sau đó giải hệ để thu được $\hat{\beta}$.

Đoạn mã sử dụng như sau:


```

1 X_list = X.tolist()
2 y_list = y.tolist()
3
4 beta_hat, sigma2_hat = ols_fit(X_list, y_list)
5 metrics = model_metrics(X_list, y_list, beta_hat)

```

Kết quả ước lượng thu được là:

$$\hat{\beta} = \begin{bmatrix} 4.9956 & 0.5565 & -0.2041 \end{bmatrix}^T.$$

Phương sai sai số và độ lệch chuẩn sai số ước lượng là:

$$\hat{\sigma}^2 = 28.9018, \quad \hat{\sigma} = 5.3760.$$

So với tham số thật $\beta^* = (10, 0.5, -0.2)^T$ và $\sigma = 5$, kết quả ước lượng tương đối hợp lý. Hai hệ số của X_1 và X_2 rất gần giá trị thật; độ lệch chuẩn sai số ước lượng cũng gần 5. Hệ số chặn lệch nhiều hơn, nhưng điều này có thể xảy ra trong mẫu hữu hạn do ảnh hưởng của nhiễu ngẫu nhiên.

Bảng 6: Tóm tắt kết quả thực nghiệm OLS

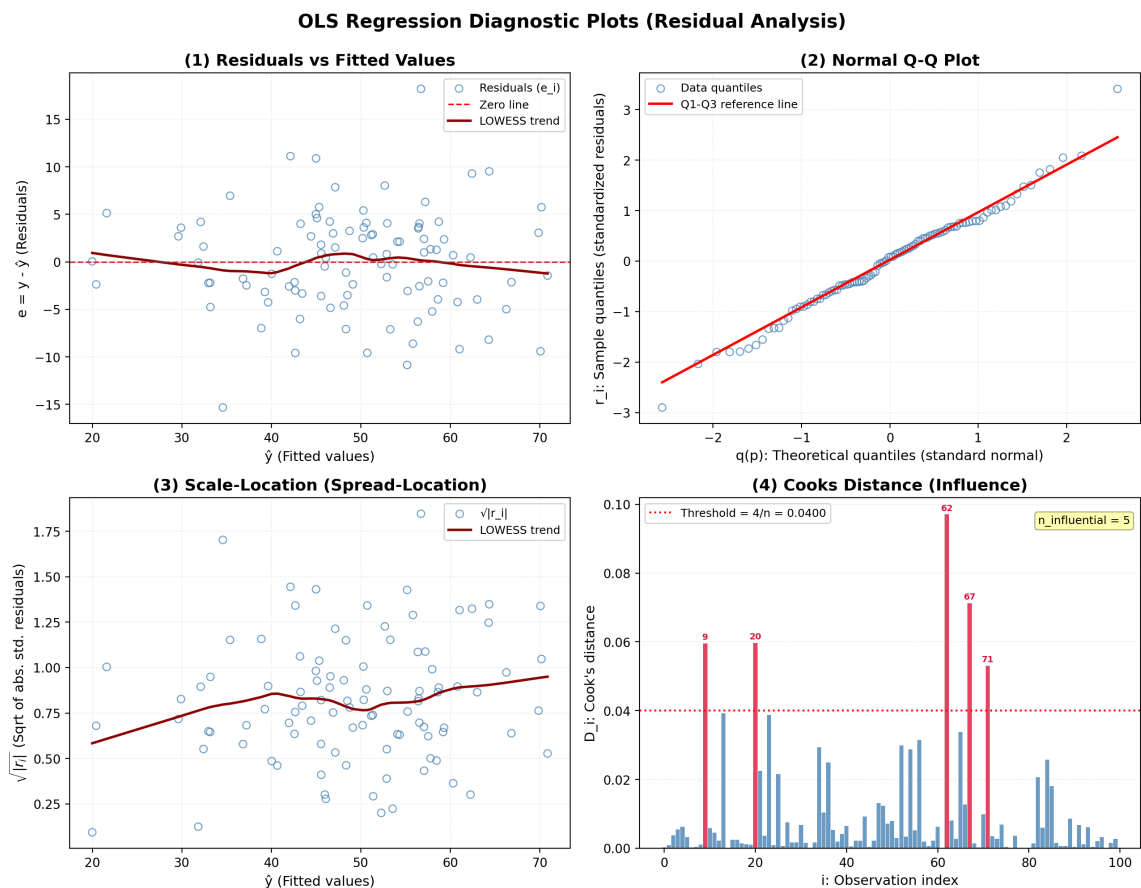
Dại lượng	Giá trị
Cỡ mẫu	$n = 100$
Số cột trong ma trận thiết kế	$k = 3$
β^*	$(10, 0.5, -0.2)$
$\hat{\beta}$	$(4.9956, 0.5565, -0.2041)$
σ thật	5.0000
$\hat{\sigma}$	5.3760
RSS	2803.4749
Số giá trị fitted	100
Số phần dư chuẩn hóa	100
Số Cook's distance	100

E.4 Residual Diagnostic Plots

E.4.1. Tổng quan

Sau khi ước lượng thành công, nhóm tiếp tục vẽ bốn biểu đồ chẩn đoán phần dư bằng cách gọi hàm `residual_plots`. Những biểu đồ này không chỉ trực quan hóa sai số mà còn giúp kiểm tra độ thỏa mãn các giả định OLS.

```
1 fig = residual_plots(X_list, y_list, beta_hat)
2 plt.tight_layout()
3 plt.show()
```



Hình 2: Tổng hợp bốn biểu đồ chẩn đoán phần dư trong notebook.

Bốn biểu đồ có ý nghĩa như sau:

- Residuals vs Fitted Values: kiểm tra tính tuyến tính và đồng phương sai.
- Normal Q-Q Plot: kiểm tra phân phối chuẩn của phần dư chuẩn hóa.
- Scale-Location Plot: kiểm tra đồng phương sai theo biên độ sai số.
- Cook's Distance: phát hiện quan sát có ảnh hưởng lớn đến mô hình.

E.5 Residuals vs Fitted Values

E.5.1. Mục đích

Biểu đồ Residuals vs Fitted Values được dùng để kiểm tra hai giả thuyết quan trọng của mô hình hồi quy tuyến tính: tính tuyến tính và đồng phương sai. Trục hoành là giá trị dự báo $\hat{y}$, trục tung là phần dư $e_i = y_i - \hat{y}_i$.

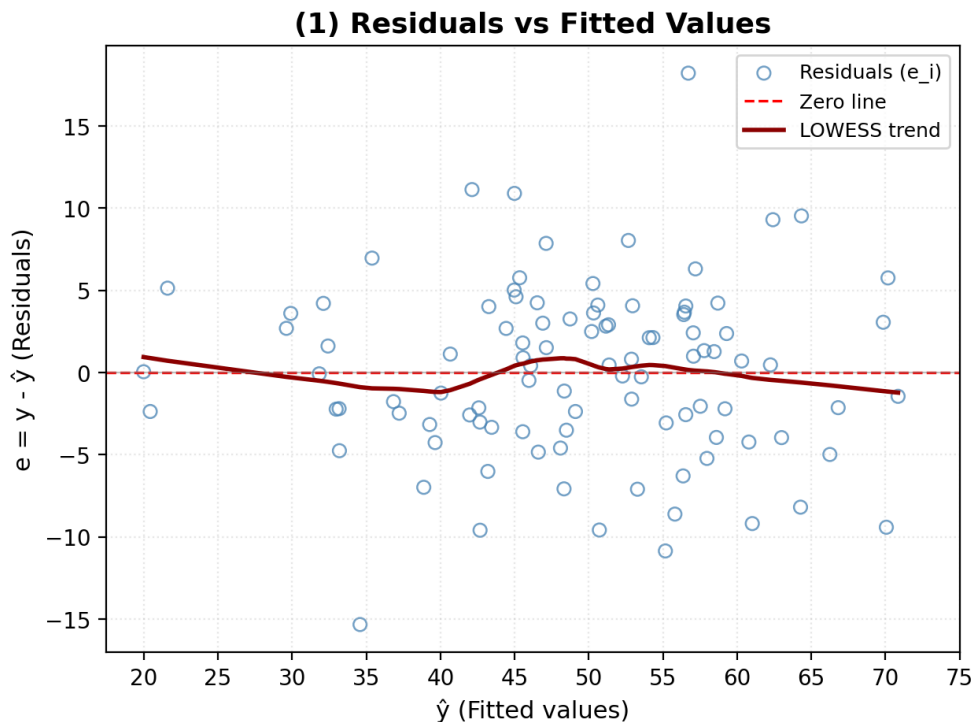
Nếu mô hình phù hợp, các điểm phần dư phải phân bố ngẫu nhiên quanh đường 0, không tạo thành dạng cong hoặc dạng sóng. Đồng thời, độ rộng phân tán của phần dư phải tương đối ổn định dọc theo trục $\hat{y}$, không tạo hình phễu.

E.5.2. Cài đặt biểu đồ

Gọi hàm `residuals_vs_fitted_plot` như sau:

```
1 fig1 = residuals_vs_fitted_plot(metrics['y_hat'], metrics['
    residuals'])
2 plt.gca().set_xticks(range(20, 76, 5))
3 plt.show()
```

Trong file `residual_plots.py`, biểu đồ này vẽ scatter plot giữa $\hat{y}$ và e_i , thêm đường ngang $e = 0$ và đường LOWESS để quan sát xu hướng của phần dư.



Hình 3: Residuals vs Fitted Values.

E.5.3. Nhận xét kết quả

Giá trị dự báo $\hat{y}$ dao động trong khoảng xấp xỉ từ 20 đến 72. Phần dư chủ yếu nằm trong khoảng từ -10 đến $+10$, với một số điểm lệch xa hơn. Các điểm tập trung nhiều nhất ở vùng $\hat{y} \in [40, 65]$, trong khi hai đầu mút $\hat{y} < 30$ và $\hat{y} > 65$ có mật độ thưa hơn.

Biểu đồ có một điểm outlier âm rõ rệt tại khoảng $(\hat{y} \approx 35, e \approx -15)$. Ngoài ra, có hai điểm outlier dương đáng chú ý tại khoảng $(\hat{y} \approx 57, e \approx 17.5)$ và $(\hat{y} \approx 42, e \approx 11.5)$. Tuy nhiên, các điểm này xuất hiện rời rạc, không tạo thành một cấu trúc có quy luật.

Đường LOWESS bám rất sát đường zero. Biên độ dao động của LOWESS chỉ nằm khoảng từ -1 đến $+1$. Có một vài đoạn uốn nhẹ ở vùng $\hat{y} = 30$ đến 42 và $\hat{y} = 45$ đến 55 , nhưng mức uốn cong nhỏ và có thể xem là dao động ngẫu nhiên của nhiễu.

Về tính tuyến tính, biểu đồ không cho thấy quan hệ phi tuyến còn sót lại trong phần dư. Do đó, giả thuyết tuyến tính được thỏa mãn rất tốt. Về đồng phương sai, không quan sát thấy hình phễu; độ rộng phân tán của phần dư tương đối đồng đều giữa các mức fitted values. Vì vậy, giả thuyết đồng phương sai cũng được đảm bảo.

E.5.4. Kết luận

esiduals vs Fitted Values cho thấy mô hình OLS hoạt động tốt. Mặc dù tồn tại một số outliers, chúng không ảnh hưởng đến cấu trúc tổng thể của phần dư. Mô hình không có dấu hiệu sai dạng hàm nghiêm trọng và có thể tiếp tục được sử dụng cho phân tích.

E.6 Normal Q-Q Plot

E.6.1. Mục đích

Normal Q-Q Plot được sử dụng để kiểm tra giả thuyết phần dư chuẩn hóa có phân phối gần chuẩn hay không. Trục hoành là phân vị lý thuyết của phân phối chuẩn tắc, còn trục tung là phân vị thực nghiệm của phần dư chuẩn hóa.

Nếu phần dư tuân theo phân phối chuẩn, các điểm dữ liệu sẽ nằm gần đường tham chiếu. Khi các điểm lệch mạnh ở hai đầu, điều đó cho thấy phần dư có thể có đuôi dày hoặc chứa các outliers.

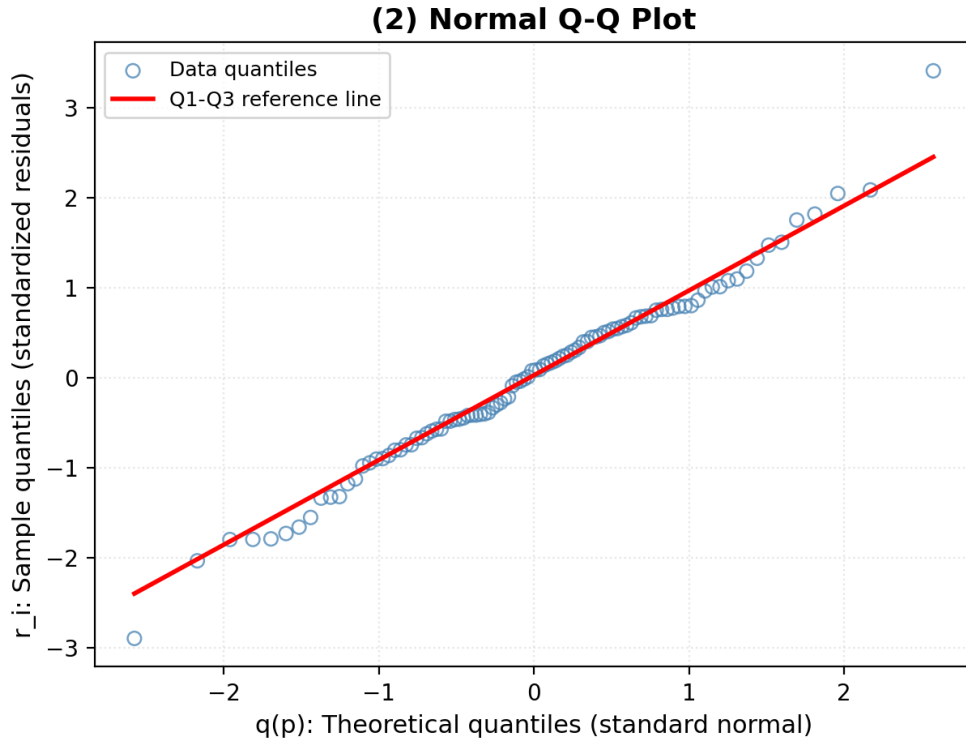
E.6.2. Cài đặt biểu đồ

Tạo biểu đồ Q-Q bằng đoạn mã:

```
fig2 = qq_plot(metrics['std_res'])
```

```
2 plt.gca().set_yticks(np.arange(-3.0, 3.6, 0.5))
3 plt.gca().set_xticks(np.arange(-2.5, 2.6, 0.5))
4 plt.show()
```

Trong hàm `qq_plot`, các phần dư chuẩn hóa được sắp xếp tăng dần rồi so sánh với các phân vị lý thuyết của chuẩn tắc. Đường tham chiếu được dựng từ Q1 đến Q3.



Hình 4: Normal Q-Q Plot của phần dư chuẩn hóa.

E.6.3. Nhận xét kết quả

Ở phân đoạn trung tâm, với r_i nằm khoảng từ -1.2 đến 1.2 , các điểm dữ liệu sắp xếp gần như thẳng hàng và ôm sát đường tham chiếu màu đỏ. Điều này cho thấy phần lớn phần dư ở vùng trung tâm tuân thủ rất tốt phân phối chuẩn lý thuyết.

Tuy nhiên, ở hai đuôi xuất hiện cấu trúc dạng chữ S nhẹ. Ở đuôi trái, khi phân vị lý thuyết nhỏ hơn khoảng -1.0 , các điểm uốn cong nhẹ xuống dưới đường tham chiếu. Điểm cực trị âm nằm khoảng $(-2.5, -2.9)$. Ở đuôi phải, khi phân vị lý thuyết lớn hơn khoảng 1.5 , các điểm uốn lên trên đường tham chiếu, với điểm cực trị dương khoảng $(2.6, 3.4)$.

Cấu trúc này cho thấy phần dư có hiện tượng heavy-tailed nhẹ, tức hai đuôi dày hơn so với phân phối chuẩn lý tưởng. Nguyên nhân chủ yếu đến từ một số outliers có độ lệch

lớn. Tuy nhiên, sai lệch này chỉ xuất hiện ở vùng đuôi và không phá vỡ cấu trúc chính của phân phối.

E.6.4. Kết luận

Normal Q-Q Plot cho thấy giả thuyết chuẩn của phần dư nhìn chung được thỏa mãn khá tốt. Phân đoạn trung tâm gần như hoàn hảo, còn sự lệch nhẹ ở hai đuôi là chấp nhận được với cỡ mẫu $n = 100$. Do đó, các kiểm định thống kê tiếp theo như t -test và F -test vẫn có thể được thực hiện với độ tin cậy hợp lý.

E.7 Scale-Location Plot

E.7.1. Mục đích

Scale-Location Plot, hay Spread-Location Plot, được sử dụng để kiểm tra giả thuyết đồng phương sai. Biểu đồ này nhạy hơn Residuals vs Fitted Values trong việc phát hiện hiện tượng phương sai thay đổi.

Trục hoành là giá trị dự báo $\hat{y}$, còn trục tung là căn bậc hai của trị tuyệt đối phần dư chuẩn hóa:

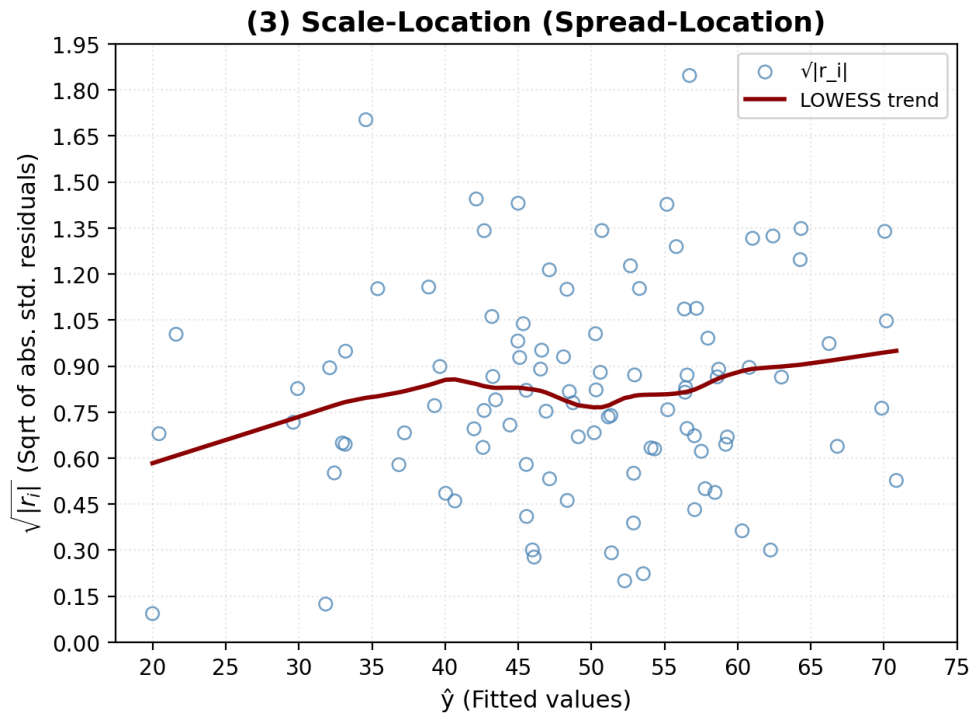
$$\sqrt{|r_i|}.$$

Nếu mô hình có đồng phương sai, đường LOWESS nên tương đối phẳng và độ phân tán của các điểm không nên tăng hoặc giảm có hệ thống theo $\hat{y}$.

E.7.2. Cài đặt biểu đồ

Tạo biểu đồ Scale-Location bằng đoạn mã:

```
1 fig3 = scale_location_plot(metrics['y_hat'], metrics['std_res'])
2 plt.gca().set_yticks(np.arange(0, 2.05, 0.15))
3 plt.gca().set_xticks(range(20, 76, 5))
4 plt.show()
```



Hình 5: Scale-Location Plot kiểm tra đồng phương sai.

E.7.3. Nhận xét kết quả

Đường LOWESS có xu hướng dốc nhẹ lên theo giá trị $\hat{y}$, nhưng biên độ dao động rất nhỏ. Theo quan sát đường này thay đổi từ khoảng 0.58 đến 0.95, chênh lệch chỉ khoảng 0.37.

Cụ thể, từ $\hat{y} = 20$ đến 40, đường LOWESS tăng nhẹ từ khoảng 0.58 lên 0.86. Từ 40 đến 55, đường giảm nhẹ rồi tăng trở lại. Từ 55 đến 71, đường tiếp tục tăng nhẹ đến khoảng 0.95. Mặc dù có xu hướng tăng nhẹ, mức biến động này không đủ mạnh để xem là bằng chứng của phương sai thay đổi có hệ thống.

Về cấu trúc phân tán, vùng giữa $\hat{y} \in [40, 60]$ có mật độ điểm cao nhất và biên độ dao động rộng hơn, từ khoảng 0.20 đến 1.85. Hai đầu mút $\hat{y} < 30$ và $\hat{y} > 65$ có mật độ thưa hơn nhưng biên độ không tăng rõ rệt. Đặc biệt, biểu đồ không xuất hiện hình phễu hoặc hình chữ V.

E.7.4. Kết luận

Scale-Location Plot xác nhận giả thuyết đồng phương sai được thỏa mãn ở mức tốt. Đường LOWESS hơi dốc lên nhưng chủ yếu phản ánh dao động ngẫu nhiên, không phải xu hướng Heteroscedasticity rõ rệt. Vì vậy, các ước lượng OLS vẫn được xem là ổn định và hiệu quả trong bộ dữ liệu này.

E.8 Cook's Distance

E.8.1. Mục đích

Cook's Distance được sử dụng để xác định các quan sát có ảnh hưởng lớn đến hệ số ước lượng của mô hình OLS. Một quan sát có Cook's Distance lớn thường là điểm vừa có phần dư lớn, vừa có leverage cao. Những điểm như vậy có khả năng làm thay đổi đáng kể kết quả hồi quy nếu bị loại khỏi dữ liệu.

Với $n = 100$, notebook sử dụng ngưỡng tham chiếu:

$$\frac{4}{n} = \frac{4}{100} = 0.04.$$

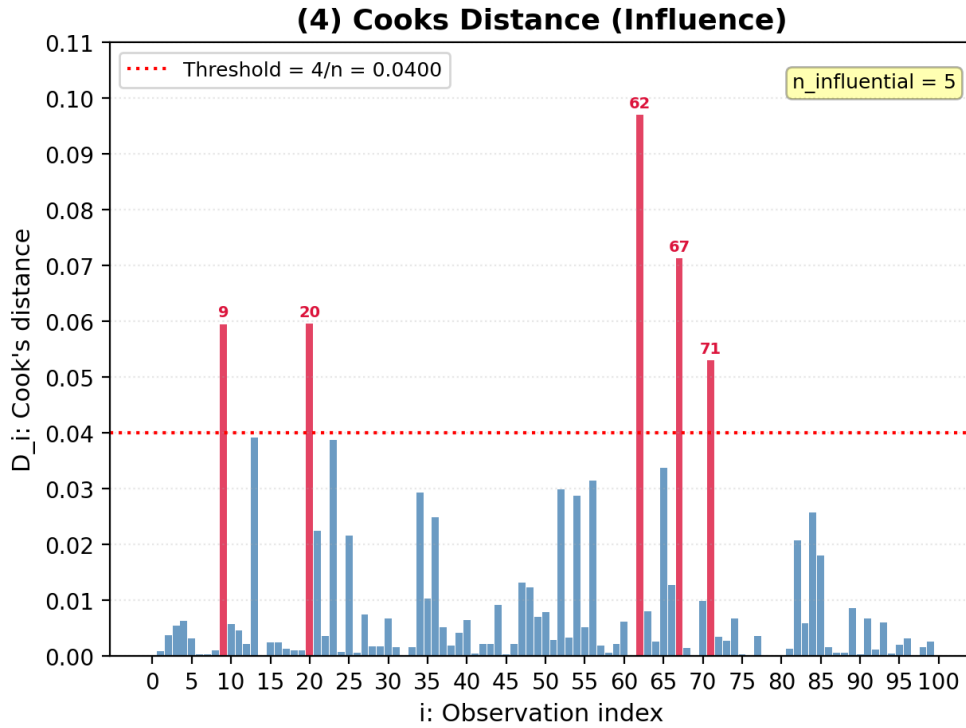
Ngoài ra, các ngưỡng nghiêm trọng thường được dùng là $D_i > 0.5$ hoặc $D_i > 1.0$.

E.8.2. Cài đặt biểu đồ

Tạo biểu đồ Cook's Distance bằng đoạn mã:

```
1 fig4 = cooks_distance_plot(metrics['cooks_d'], n=len(y_list))
2 plt.gca().set_yticks(np.arange(0, 0.12, 0.01))
3 plt.gca().set_xticks(range(0, 105, 5))
4 plt.show()
```

Trong hàm `cooks_distance_plot`, các quan sát có $D_i > 4/n$ được tô màu đỏ và đánh nhãn chỉ số quan sát.



Hình 6: Cook's Distance đánh giá các điểm có ảnh hưởng lớn.

E.8.3. Nhận xét kết quả

Hầu hết các quan sát có Cook's Distance rất thấp, nằm dưới ngưỡng 0.04. Điều này cho thấy phần lớn dữ liệu không có ảnh hưởng bất thường đến mô hình.

Tuy nhiên, có 5 quan sát vượt ngưỡng $4/n$:

- Quan sát 62: $D_{62} \approx 0.097$, là điểm ảnh hưởng lớn nhất.
- Quan sát 67: $D_{67} \approx 0.072$.
- Quan sát 9 và 20: $D_i \approx 0.060$.
- Quan sát 71: $D_{71} \approx 0.053$.

Một số quan sát khác, chẳng hạn quanh index 13 và 23, nằm sát ngưỡng nhưng chưa vượt qua.

Mặc dù có 5 điểm vượt ngưỡng tương đối, điểm lớn nhất vẫn chỉ khoảng 0.097, thấp hơn rất nhiều so với ngưỡng nghiêm trọng 0.5 hoặc 1.0. Vì vậy, các điểm này chỉ nên được xem là điểm cần chú ý, không phải điểm làm méo mó hệ thống mô hình.

E.8.4. Kết luận

Cook's Distance cho thấy mô hình có một vài quan sát ảnh hưởng cao hơn trung bình, nhưng không có quan sát nào nguy hiểm. Do đó, mô hình OLS được đánh giá là ổn định và đáng tin cậy; các hệ số ước lượng không bị chi phối bởi một điểm dữ liệu đơn lẻ.

E.9 Tổng kết đánh giá mô hình

Từ bốn biểu đồ chẩn đoán phần dư, có thể tổng hợp đánh giá như sau:

- Residuals vs Fitted Values cho thấy phần dư phân bố ngẫu nhiên quanh đường 0, không có cấu trúc phi tuyến rõ ràng.
- Normal Q-Q Plot cho thấy phần dư chuẩn hóa tuân theo phân phối chuẩn khá tốt ở vùng trung tâm, chỉ lệch nhẹ ở hai đuôi do một vài outliers.
- Scale-Location Plot xác nhận phương sai phần dư tương đối ổn định, không xuất hiện hình phễu hay xu hướng phương sai thay đổi nghiêm trọng.
- Cook's Distance phát hiện 5 điểm vượt ngưỡng $4/n$, nhưng tất cả đều thấp xa so với ngưỡng nghiêm trọng.

Kết quả chẩn đoán phần dư cho thấy mô hình hồi quy tuyến tính ước lượng bằng phương pháp OLS đáp ứng tương đối tốt các giả định thống kê cơ bản. Cụ thể, trên đồ thị Residuals vs Fitted, các phần dư phân bố ngẫu nhiên quanh đường chuẩn 0 và không hình thành xu hướng hay cấu trúc phi tuyến rõ rệt, cho thấy giả định tuyến tính của mô hình được thỏa mãn.

Bên cạnh đó, đồ thị Normal Q-Q cho thấy phần lớn các điểm dữ liệu bám sát đường phân phối chuẩn lý thuyết, ngoại trừ một vài sai lệch nhỏ tại vùng đuôi. Điều này cho thấy phần dư có thể được xem là xấp xỉ phân phối chuẩn, và các sai lệch xuất hiện không đủ lớn để ảnh hưởng đáng kể đến chất lượng ước lượng của mô hình.

Đối với giả định phương sai không đổi, đồ thị Scale-Location thể hiện độ phân tán của phần dư tương đối ổn định trên toàn bộ miền giá trị dự báo, không xuất hiện hiện tượng mở rộng hay thu hẹp bất thường. Do đó, chưa có bằng chứng rõ ràng về hiện tượng phương sai thay đổi (heteroscedasticity).

Ngoài ra, phân tích Cook's Distance cho thấy một số quan sát như 9, 20, 62, 67 và 71 có mức ảnh hưởng tương đối cao so với các điểm còn lại. Tuy nhiên, giá trị Cook's Distance của các quan sát này vẫn chưa vượt qua các ngưỡng cảnh báo nghiêm trọng, do đó mức độ tác động đến mô hình được đánh giá là không đáng kể.

Từ các kết quả trên, có thể kết luận rằng mô hình hồi quy OLS được xây dựng có độ phù hợp tốt với dữ liệu, các giả định quan trọng nhìn chung đều được đáp ứng, và các hệ số ước lượng thu được đủ độ tin cậy để phục vụ cho mục đích phân tích và ứng dụng thực tiễn.

F Gauss–Markov Theorem và Monte Carlo Simulation

F.1 Giới thiệu

Nhóm trình bày cơ sở lý thuyết và thực nghiệm mô phỏng Monte Carlo cho Định lý Gauss–Markov. Nội dung trọng tâm là kiểm chứng ba ý chính: ước lượng OLS là không chệch, OLS có phương sai nhỏ nhất trong lớp các ước lượng tuyến tính không chệch, và một ước lượng có chệch như Ridge Regression vẫn có thể đạt sai số bình phương trung bình nhỏ hơn trong một số tình huống nhờ đánh đổi giữa độ chệch và phương sai.

Các kết quả được mô phỏng bằng Python với 1000 lần lặp. Trong mỗi lần lặp, dữ liệu được sinh từ một mô hình hồi quy tuyến tính có nhiễu Gaussian, sau đó ước lượng hệ số bằng Ordinary Least Squares (OLS) và Ridge Regression. Cuối cùng, các đại lượng Bias, Variance và Mean Squared Error (MSE) được tính toán và trực quan hóa.

F.2 Cơ sở lý thuyết Gauss–Markov

Xét mô hình hồi quy tuyến tính:

$$y = X\beta + \varepsilon,$$

trong đó $y \in \mathbb{R}^n$ là vector biến phụ thuộc, $X \in \mathbb{R}^{n \times k}$ là ma trận thiết kế, $\beta \in \mathbb{R}^k$ là vector hệ số thật và ε là vector sai số ngẫu nhiên.

Định lý Gauss–Markov được phát biểu dưới các giả thiết sau:

1. Sai số có kỳ vọng bằng 0: $\mathbb{E}(\varepsilon) = 0$.
2. Sai số có phương sai không đổi: $\text{Var}(\varepsilon) = \sigma^2 I$.
3. Các sai số không tương quan với nhau: $\text{Cov}(\varepsilon_i, \varepsilon_j) = 0$ với $i \neq j$.
4. Ma trận X có hạng cột đầy đủ, tức $X^T X$ khả nghịch.

Khi các giả thiết trên thỏa mãn, ước lượng OLS

$$\hat{\beta}_{\text{OLS}} = (X^T X)^{-1} X^T y$$

là BLUE, tức Best Linear Unbiased Estimator. Nói cách khác, OLS là ước lượng tuyến tính không chệch tốt nhất theo nghĩa có phương sai nhỏ nhất trong toàn bộ lớp các ước lượng tuyến tính không chệch.

Điểm cần nhấn mạnh là chữ “Best” trong định lý chỉ xét trong lớp *tuyến tính không chệch*. Nếu cho phép dùng một ước lượng có chệch, chẳng hạn Ridge Regression, thì phương sai có thể giảm xuống và MSE tổng thể có thể nhỏ hơn. Đây chính là ý tưởng trung tâm của đánh đổi Bias–Variance.

F.3 Thiết lập mô phỏng Monte Carlo

Mô phỏng Monte Carlo được thực hiện bằng cách lặp lại quá trình sinh dữ liệu và ước lượng nhiều lần. Trong mô phỏng này, số lần mô phỏng là

$$N = 1000.$$

Ở mỗi lần mô phỏng, dữ liệu gồm $n = 100$ quan sát và $k = 3$ hệ số, bao gồm một hệ số chặn và hai hệ số góc. Vector hệ số thật được đặt là

$$\beta_{\text{true}} = \begin{bmatrix} 0 & 1 & 2 \end{bmatrix}^T.$$

Hai phương pháp được so sánh là OLS và Ridge Regression với tham số chính quy hóa $\lambda = 0.1$. Công thức nghiệm Ridge được dùng là:

$$\hat{\beta}_{\text{Ridge}} = (X^T X + \lambda D)^{-1} X^T y,$$

trong đó D là ma trận phạt. Thông thường, hệ số chặn không bị phạt nên phần tử đầu tiên trên đường chéo của D được đặt bằng 0.

Đoạn mã mô phỏng cốt lõi có dạng như sau:

```
1 n_simulations = 1000
2 n = 100
3 k = 3
4 beta_true = np.array([0.0, 1.0, 2.0])
5 lambda_ridge = 0.1
6
7 for sim in range(n_simulations):
8     X_raw = np.random.randn(n, k - 1)
9     X = np.c_[np.ones(n), X_raw]
10    epsilon = np.random.randn(n)
11    y = X @ beta_true + epsilon
12
13    beta_ols = np.linalg.inv(X.T @ X) @ X.T @ y
14
15    D = np.eye(k)
16    D[0, 0] = 0
17    beta_ridge = np.linalg.inv(X.T @ X + lambda_ridge * D) @ X.T @
    y
```

Sau khi thu được 1000 bộ hệ số ước lượng, nhóm tính ba đại lượng thống kê chính:

$$\text{Bias}(\hat{\beta}_j) = \mathbb{E}(\hat{\beta}_j) - \beta_j,$$

$$\text{Var}(\hat{\beta}_j) \approx \frac{1}{N} \sum_{i=1}^N \left(\hat{\beta}_{j,i} - \mathbb{E}(\hat{\beta}_j) \right)^2,$$

và

$$\text{MSE}(\hat{\beta}_j) = \text{Bias}(\hat{\beta}_j)^2 + \text{Var}(\hat{\beta}_j).$$

F.4 Kiểm tra tính không chệch

Một ước lượng $\hat{\beta}$ được gọi là không chệch nếu

$$\mathbb{E}(\hat{\beta}) = \beta_{\text{true}}.$$

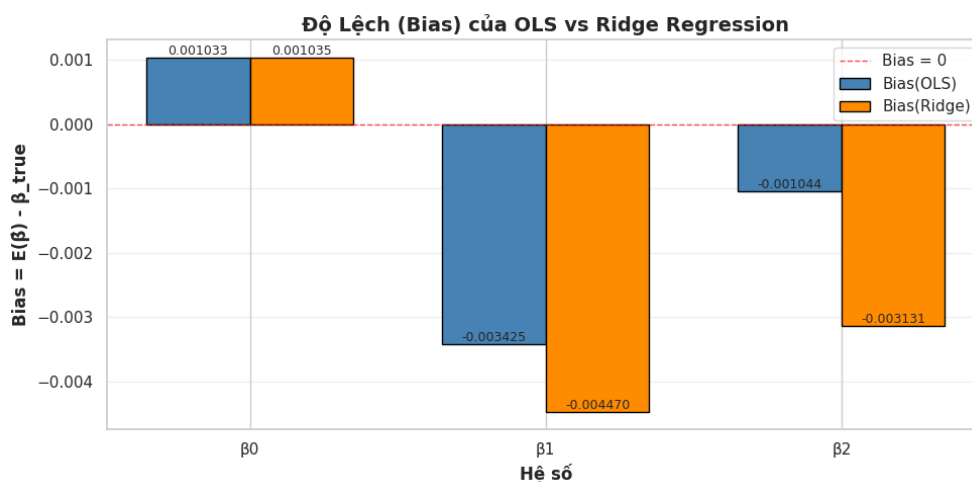
Trong mô phỏng Monte Carlo, kỳ vọng được xấp xỉ bằng trung bình của các ước lượng qua 1000 lần chạy.

Bảng 7: So sánh độ chệch trung bình của OLS và Ridge Regression

Hệ số	$E(\hat{\beta}_{OLS})$	Bias OLS	$E(\hat{\beta}_{Ridge})$	Bias Ridge
β_0	0.001033	0.001033	0.001035	0.001035
β_1	0.996575	-0.003425	0.995530	-0.004470
β_2	1.998956	-0.001044	1.996869	-0.003131

Bảng trên cho thấy OLS có độ chệch rất nhỏ ở cả ba hệ số. Các giá trị Bias chỉ dao động quanh 0, phù hợp với kết luận lý thuyết rằng OLS là ước lượng không chệch khi các giả thiết Gauss–Markov được thỏa mãn. Những sai lệch nhỏ còn lại đến từ sai số mô phỏng hữu hạn vì số lần lặp chỉ là 1000 chứ không phải vô hạn.

Đối với Ridge Regression, độ chệch lớn hơn OLS ở các hệ số góc. Điều này xuất phát từ thành phần chính quy hóa L_2 , làm các hệ số bị kéo nhẹ về phía 0. Vì β_1 và β_2 có giá trị thật dương, cơ chế co ngót khiến trung bình ước lượng Ridge nhỏ hơn giá trị thật, dẫn đến Bias âm.



Hình 7: So sánh Bias của OLS và Ridge Regression theo từng hệ số.

Biểu đồ Bias trực quan hóa rõ hơn nhận xét trên. Tại β_0 , hai phương pháp gần như giống nhau vì hệ số chặn không chịu tác động trực tiếp của phạt Ridge. Tại β_1 và β_2 , Ridge có Bias âm lớn hơn OLS, phản ánh đúng bản chất của regularization.

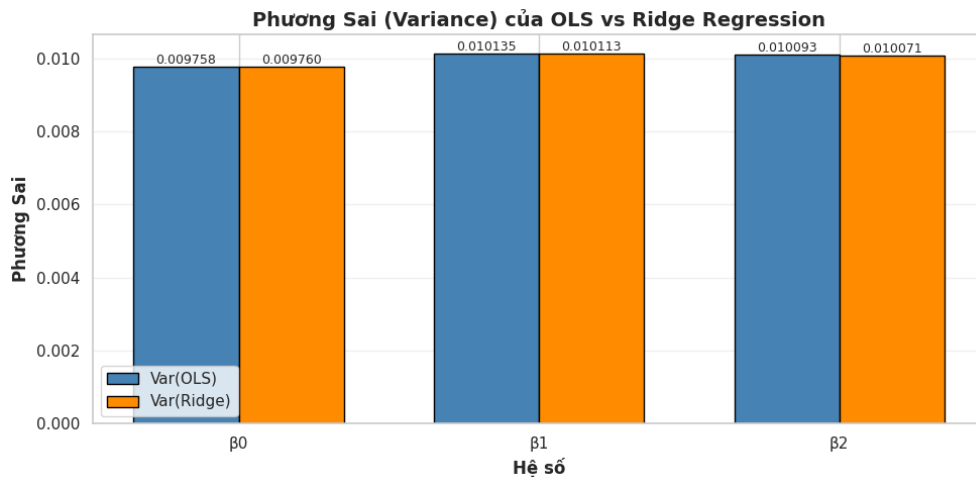
F.5 So sánh phương sai

Phương sai đo mức độ dao động của ước lượng quanh giá trị trung bình của chính nó. Trong bối cảnh Monte Carlo, phương sai càng nhỏ thì hệ số ước lượng càng ổn định qua các mẫu dữ liệu khác nhau.

Bảng 8: So sánh phương sai của OLS và Ridge Regression

Hệ số	Var OLS	Var Ridge	Tỷ lệ $\frac{Var_{OLS}}{Var_{Ridge}}$
β_0	0.0097579772	0.0097600306	0.9998
β_1	0.0101350819	0.0101128773	1.0022
β_2	0.0100929220	0.0100711306	1.0022

Kết quả cho thấy phương sai của Ridge tại β_1 và β_2 nhỏ hơn nhẹ so với OLS. Đây là hệ quả trực tiếp của chính quy hóa: Ridge chấp nhận tạo ra một độ chệch nhỏ để giảm dao động của hệ số. Riêng tại β_0 , phương sai của hai phương pháp gần như bằng nhau vì hệ số chặn không bị phạt.



Hình 8: So sánh phương sai của OLS và Ridge Regression.

Kết quả này không mâu thuẫn với định lý Gauss–Markov. Định lý chỉ khẳng định OLS có phương sai nhỏ nhất trong lớp các ước lượng tuyến tính không chệch. Ridge Regression là một ước lượng có chệch, nên nó không nằm trong lớp so sánh trực tiếp của định lý. Vì vậy, Ridge có thể có phương sai nhỏ hơn OLS nhưng phải trả giá bằng Bias lớn hơn.

F.6 Phân tích Mean Squared Error

MSE kết hợp cả hai nguồn sai số: độ chệch và phương sai. Công thức phân rã là

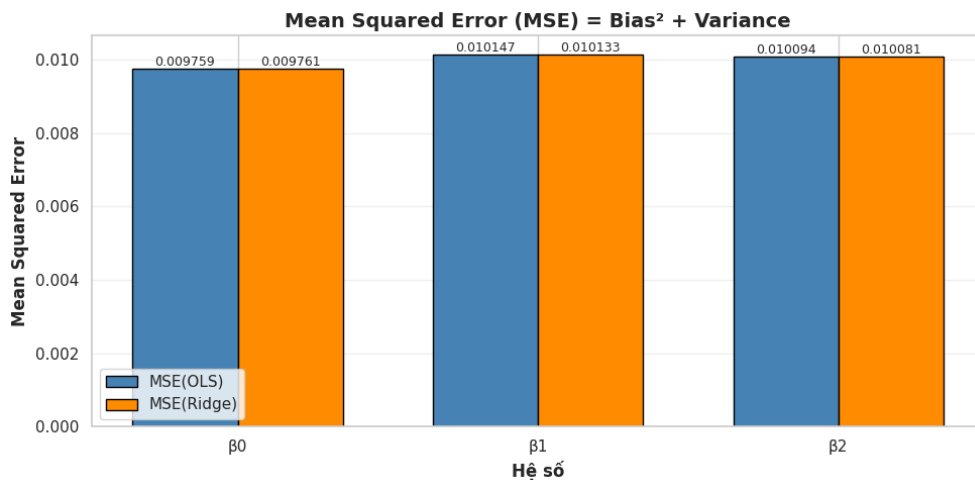
$$\text{MSE}(\hat{\beta}) = \text{Bias}(\hat{\beta})^2 + \text{Var}(\hat{\beta}).$$

Do đó, MSE là thước đo tổng hợp hơn so với chỉ nhìn riêng Bias hoặc Variance.

Bảng 9: So sánh MSE của OLS và Ridge Regression

Hệ số	MSE OLS	MSE Ridge	Bias ² OLS	Bias ² Ridge
β_0	0.0097590449	0.0097611021	0.0000010677	0.0000010715
β_1	0.0101468115	0.0101328602	0.0000117296	0.0000199829
β_2	0.0100940109	0.0100809369	0.0000010889	0.0000098063

Tại β_0 , MSE của OLS và Ridge gần như tương đương. Tại β_1 và β_2 , Ridge có MSE nhỏ hơn nhẹ so với OLS. Điều này cho thấy phần giảm phương sai của Ridge lớn hơn phần tăng thêm do bình phương độ chệch, nên tổng sai số được cải thiện một chút.

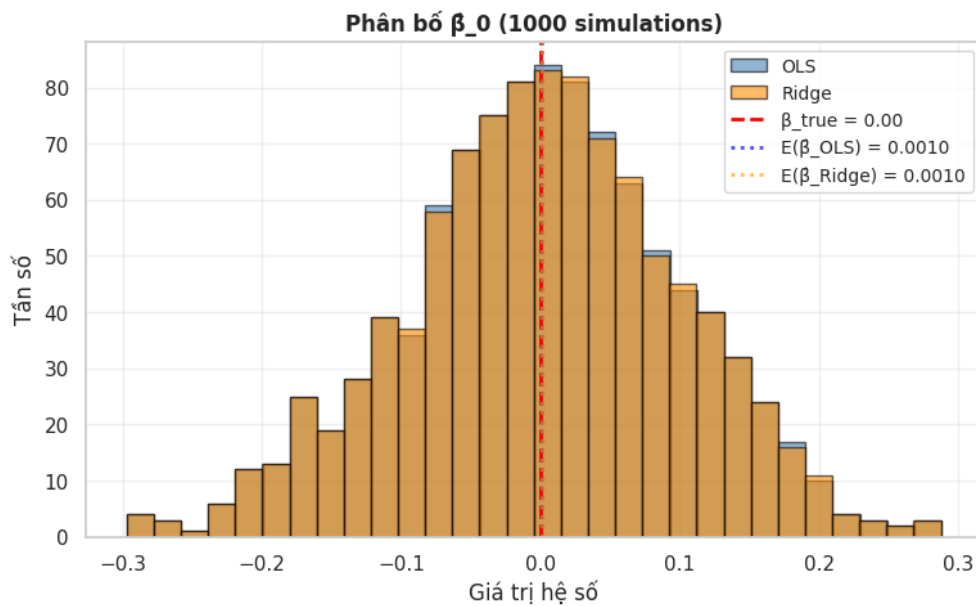


Hình 9: So sánh Mean Squared Error của OLS và Ridge Regression.

Kết quả MSE minh họa một điểm quan trọng: OLS là tốt nhất nếu tiêu chí là phương sai nhỏ nhất trong lớp không chệch, nhưng không nhất thiết luôn có MSE nhỏ nhất nếu nhóm cho phép các ước lượng có chệch. Trong thực hành dự báo, một mô hình có Bias nhỏ nhưng Variance giảm đáng kể có thể hoạt động tốt hơn trên dữ liệu mới.

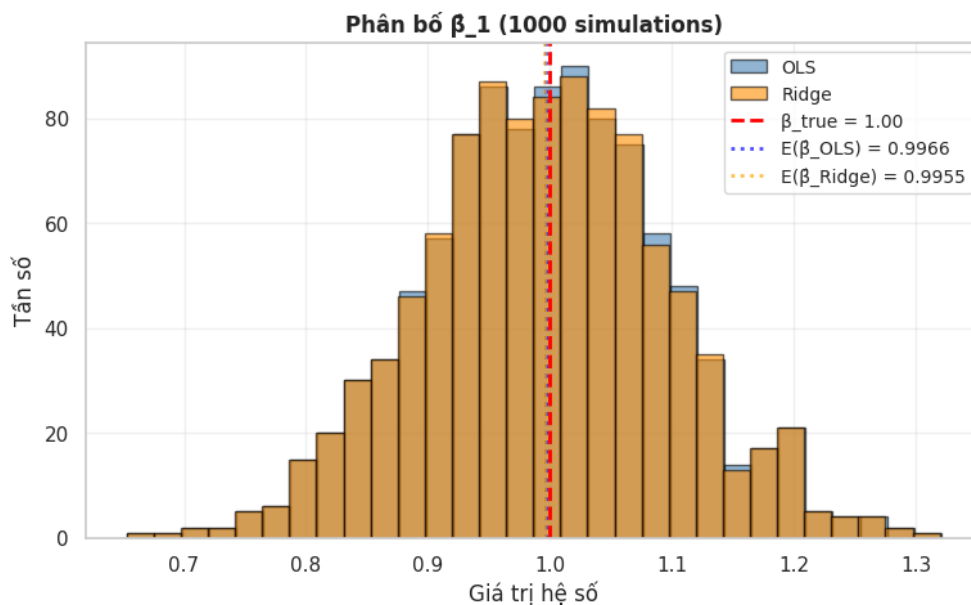
F.7 Trực quan hóa phân bố hệ số ước lượng

Ngoài các bảng số liệu, phân bố của từng hệ số ước lượng qua 1000 lần mô phỏng giúp quan sát trực tiếp mức độ tập trung và dao động của OLS so với Ridge Regression.



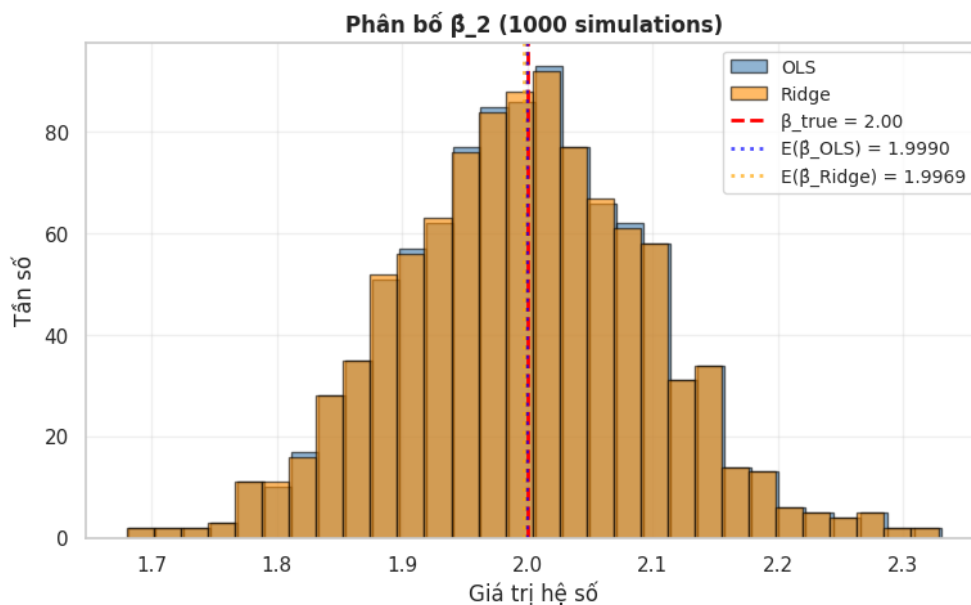
Hình 10: Phân bố ước lượng của hệ số chặn β_0 .

Với β_0 , hai phân bố gần như chồng lên nhau và đều tập trung quanh giá trị thật bằng 0. Điều này phù hợp với bảng Bias và Variance: cả OLS và Ridge đều có Bias rất nhỏ và phương sai gần như giống nhau tại hệ số chặn.



Hình 11: Phân bố ước lượng của hệ số β_1 .

Với β_1 , phân bố của OLS tập trung gần giá trị thật 1, trong khi Ridge dịch nhẹ về phía nhỏ hơn do hiệu ứng shrinkage. Tuy nhiên, Ridge có xu hướng dao động ít hơn một chút, thể hiện qua phương sai thấp hơn.

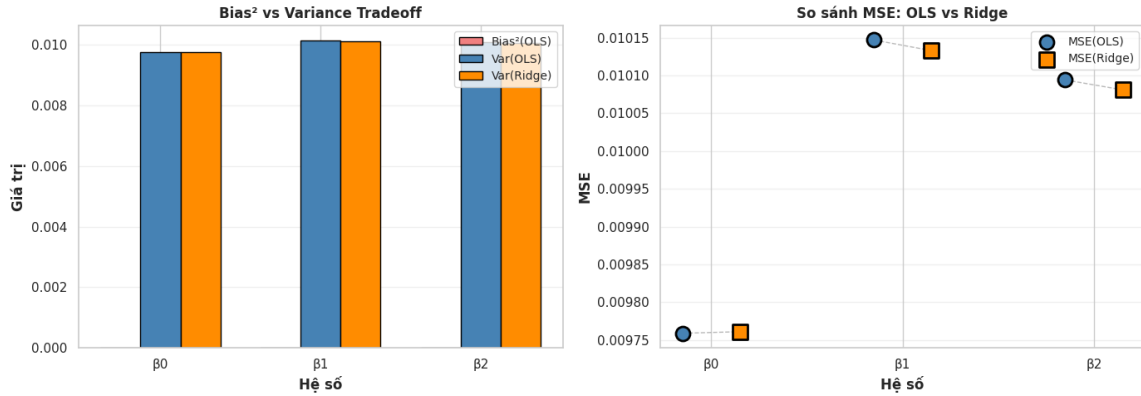


Hình 12: Phân bố ước lượng của hệ số β_2 .

Với β_2 , hiện tượng tương tự xuất hiện. OLS gần như không chệch quanh giá trị thật 2, còn Ridge bị kéo nhẹ về phía 0. Sự dịch chuyển này tạo Bias âm nhưng đồng thời làm giảm nhẹ phương sai của ước lượng.

F.8 Bias–Variance Tradeoff

Đánh đổi Bias–Variance là nguyên lý giải thích vì sao một mô hình có chệch vẫn có thể hữu ích. Khi thêm thành phần phạt $\lambda \|\beta\|_2^2$, Ridge làm nghiệm bớt nhạy cảm với nhiễu trong dữ liệu. Kết quả là phương sai giảm, nhưng hệ số bị kéo về 0 nên Bias tăng.



Hình 13: Minh họa Bias–Variance Tradeoff và so sánh MSE giữa OLS và Ridge.

Biểu đồ bên trái cho thấy thành phần Variance chiếm phần lớn trong MSE, còn Bias² rất nhỏ. Ridge làm Bias² tăng nhẹ nhưng đồng thời giảm Variance ở các hệ số góc. Biểu đồ bên phải cho thấy MSE của Ridge tại β_1 và β_2 thấp hơn OLS một lượng nhỏ. Điều này xác nhận rằng trong thí nghiệm này, lợi ích từ giảm phương sai lớn hơn chi phí do tăng độ chệch.

F.9 Kết luận

Thông qua mô phỏng Monte Carlo, nhóm đã kiểm chứng được các ý chính của Định lý Gauss–Markov và minh họa thêm vai trò của Ridge Regression trong đánh đổi Bias–Variance. OLS có Bias rất gần 0 ở cả ba hệ số, xác nhận tính không chệch khi các giả thiết mô hình tuyến tính được thỏa mãn. Thứ hai, Ridge Regression có Bias lớn hơn do cơ chế chính quy hóa kéo hệ số về phía 0. Thứ ba, Ridge lại có phương sai thấp hơn nhẹ ở các hệ số góc, cho thấy regularization giúp hệ số ổn định hơn trước nhiễu ngẫu nhiên. Cuối cùng, khi xét MSE, Ridge đạt kết quả tốt hơn một chút tại β_1 và β_2 vì mức giảm phương sai lớn hơn mức tăng Bias². Như vậy, OLS vẫn đúng là BLUE trong lớp các ước lượng tuyến tính không chệch, còn Ridge Regression minh họa rằng trong thực hành, nếu mục tiêu là giảm tổng sai số dự báo, nhóm có thể chấp nhận một lượng Bias nhỏ để đổi lấy phương sai thấp hơn và mô hình ổn định hơn.

5 Phần 2: Ứng Dụng Data Fitting vào Dữ Liệu Thực Tế

A Giới thiệu

A.1 Bài toán và Mục tiêu

A.1.1. Bối cảnh bài toán

Điểm số IMDb từ lâu đã trở thành thước đo phổ biến để đánh giá sự thành công của một bộ phim dưới góc nhìn của khán giả. Đặt trong bối cảnh khối lượng dữ liệu điện ảnh ngày càng đồ sộ, câu hỏi đặt ra là liệu chúng ta có thể xác định bằng các phương pháp thống kê những yếu tố cốt lõi nào (như ngân sách, đạo diễn, dàn diễn viên, thời lượng, v.v.) thực sự tác động đến điểm số IMDb hay không.

Đây là một bài toán phân tích hồi quy (regression analysis) điển hình thuộc phạm trù data fitting. Trong đó, điểm IMDb đóng vai trò là biến mục tiêu liên tục (`imdb_score`), còn các đặc trưng của phim đóng vai trò là các biến độc lập giải thích cho sự biến thiên của điểm số.

A.1.2. Mục tiêu nghiên cứu

Thông qua bài toán này, nhóm hướng đến việc hiện thực hóa các lý thuyết data fitting vào một quy trình phân tích dữ liệu chuyên nghiệp, bao gồm các mục tiêu cụ thể:

- Triển khai toàn diện quy trình tiền xử lý, làm sạch và biến đổi một bộ dữ liệu thực tế chứa nhiều nhiễu.
- Xây dựng, chẩn đoán và hiệu chỉnh các mô hình hồi quy tuyến tính (OLS, Ridge) nhằm ước lượng tham số tối ưu.
- Diễn giải ý nghĩa thống kê thực tiễn của các đặc trưng có tác động đến `imdb_score`.

A.2 Quy ước thống kê

A.2.1. Mức ý nghĩa

Trong toàn bộ phần phân tích và đánh giá mô hình tiếp theo, nhóm thiết lập mức ý nghĩa thống kê (significance level) tiêu chuẩn là $\alpha = 0.05$. Ngưỡng này được thống nhất sử dụng làm tiêu chí để đưa ra quyết định bác bỏ hay chấp nhận giả thuyết không (H_0) trong mọi kiểm định (như t-test, F-test, hay Likelihood Ratio Test).

B Hiểu dữ liệu

Mục đích: làm quen với bộ dữ liệu, đánh giá chất lượng dữ liệu ban đầu và khám phá các đặc điểm quan trọng trước khi tiến hành tiền xử lý dữ liệu và xây dựng mô hình.

B.1 Mô tả dữ liệu (Data Description)

B.1.1. Nguồn dữ liệu:

Trong Case Study này, nhóm sử dụng bộ dữ liệu IMDB 5000 Movie Dataset lấy từ nguồn Kaggle:

<https://www.kaggle.com/datasets/carolzhangdc/imdb-5000-movie-dataset>

Bộ dữ liệu bao gồm 28 biến số của 5043 bộ phim, trải dài trong suốt 100 năm tại 66 quốc gia. Có 2399 tên đạo diễn phân biệt và hàng ngàn diễn viên. “imdb_score” là biến đầu ra, 27 biến còn lại là các biến đặc trưng.

B.1.2. Mô tả dữ liệu

Kiểm tra quy mô dữ liệu:

```
1 # (5043, 28)
```

Bộ dữ liệu gồm 5043 quan trắc (bộ phim), 28 đặc trưng (biến) mô tả đặc điểm của từng bộ phim:

- **'color'**: định dạng màu sắc của phim (phim màu hoặc phim trắng đen)
- **'director_name'**: tên đạo diễn
- **'num_critic_for_reviews'**: số lượng nhà phê bình đã viết đánh giá
- **'duration'**: thời lượng phim (tính bằng phút)
- **'director_facebook_likes'**: số lượng thích trang facebook của đạo diễn
- **'actor_3_facebook_likes'**: số lượt thích trang Facebook của diễn viên phụ thứ 3
- **'actor_2_name'**: tên diễn viên phụ thứ 2

- **'actor_1_facebook_likes'**: số lượt thích trang Facebook của diễn viên chính
- **'gross'**: tổng doanh thu của bộ phim
- **'genres'**: thể loại phim (hành động, phiêu lưu, giả tưởng...)
- **'actor_1_name'**: tên diễn viên chính (nam/nữ diễn viên số 1)
- **'movie_title'**: tên bộ phim
- **'num_voted_users'**: số lượng khán giả đã tham gia chấm điểm/bình chọn cho phim
- **'cast_total_facebook_likes'**: tổng số lượt thích trên Facebook của toàn bộ dàn diễn viên
- **'actor_3_name'**: tên diễn viên phụ thứ 3
- **'facenumber_in_poster'**: số lượng khuôn mặt xuất hiện trên poster (áp phích) của phim
- **'plot_keywords'**: các từ khóa chính mô tả cốt truyện
- **'movie_imdb_link'**: đường dẫn (URL) tới trang chủ của phim trên trang web IMDB
- **'num_user_for_reviews'**: số lượng người dùng phổ thông đã viết bài đánh giá
- **'language'**: ngôn ngữ sử dụng trong phim
- **'country'**: quốc gia sản xuất phim
- **'content_rating'**: nhãn phân loại nội dung/độ tuổi khán giả (Ví dụ: PG-13, R, G...)
- **'budget'**: ngân sách sản xuất (tổng chi phí làm phim)
- **'title_year'**: năm phát hành phim
- **'actor_2_facebook_likes'**: số lượt thích trang Facebook của diễn viên phụ thứ 2
- **'imdb_score'**: điểm đánh giá trung bình của phim trên hệ thống IMDB (thang điểm 10)
- **'aspect_ratio'**: tỷ lệ khung hình của phim (Ví dụ: 16:9, 2.35:1)
- **'movie_facebook_likes'**: số lượt thích trên trang Facebook chính thức của bộ phim

⇒ **Chọn biến mục tiêu** (biến đầu ra/ biến đáp ứng) để tìm ra mô hình tốt nhất giải thích cho tổng doanh thu phim: **'imdb_score'**: điểm đánh giá trung bình của phim trên hệ thống IMDB (thang điểm 10).

B.2 Khám phá dữ liệu

B.2.1. Thông tin cơ bản

Kiểm tra sơ liệu về kiểu dữ liệu và loại dữ liệu:

Bảng 10: Thông tin các biến trong bộ dữ liệu trước khi chuẩn hóa (dataset.info)

#	Column	Non-Null Count	Dtype
0	color	5024 non-null	object
1	director_name	4939 non-null	object
2	num_critic_for_reviews	4993 non-null	float64
3	duration	5028 non-null	float64
4	director_facebook_likes	4939 non-null	float64
5	actor_3_facebook_likes	5020 non-null	float64
6	actor_2_name	5030 non-null	object
7	actor_1_facebook_likes	5036 non-null	float64
8	gross	4159 non-null	float64
9	genres	5043 non-null	object
10	actor_1_name	5036 non-null	object
11	movie_title	5043 non-null	object
12	num_voted_users	5043 non-null	int64
13	cast_total_facebook_likes	5043 non-null	int64
14	actor_3_name	5020 non-null	object
15	facenumber_in_poster	5030 non-null	float64
16	plot_keywords	4890 non-null	object
17	movie_imdb_link	5043 non-null	object
18	num_user_for_reviews	5022 non-null	float64
19	language	5029 non-null	object
20	country	5038 non-null	object
21	content_rating	4740 non-null	object
22	budget	4551 non-null	float64
23	title_year	4935 non-null	float64
24	actor_2_facebook_likes	5030 non-null	float64
25	imdb_score	5043 non-null	float64
26	aspect_ratio	4714 non-null	float64
27	movie_facebook_likes	5043 non-null	int64

Bảng 11: Mẫu dữ liệu phim (dataset.head)

movie_title	title_year	imdb_score	gross	director_name
Avatar	2009.0	7.9	760505847.0	James Cameron
Pirates of the Caribbean: At World's End	2007.0	7.1	309404152.0	Gore Verbinski
Spectre	2015.0	6.8	200074175.0	Sam Mendes
The Dark Knight Rises	2012.0	8.5	448130642.0	Christopher Nolan
Star Wars: Episode VII - The Force Awakens	-	7.1	-	Doug Walker

Sau khi kiểm tra sơ liệu về bộ dữ liệu, ta nhận thấy bộ dữ liệu có một số biến sau đây:

- **'color'**: gồm 2 định dạng màu sắc phim: phim màu (Color) và phim trắng đen (Black and White).
- **'genres'**: gồm tất cả thể loại của phim ngăn cách bởi dấu |.
- **'plot_keywords'**: gồm các từ khóa chính mô tả cốt truyện ngăn cách bởi dấu |.

Các biến còn lại đều có kiểu dữ liệu phù hợp.

Để đảm bảo tính chất phân loại của các biến này được đưa vào mô hình một cách đúng nghĩa, giúp các thuật toán hồi quy xử lý dữ liệu chính xác hơn, ta tiến hành chuẩn hóa kiểu dữ liệu của chúng:

```
1 dataset['color'] = dataset['color'].astype('category')
```


Bảng 12: Thông tin các biến trong bộ dữ liệu sau khi chuẩn hóa (dataset.info)

#	Column	Non-Null Count	Dtype
0	color	5024 non-null	category
1	director_name	4939 non-null	object
2	num_critic_for_reviews	4993 non-null	float64
3	duration	5028 non-null	float64
4	director_facebook_likes	4939 non-null	float64
5	actor_3_facebook_likes	5020 non-null	float64
6	actor_2_name	5030 non-null	object
7	actor_1_facebook_likes	5036 non-null	float64
8	gross	4159 non-null	float64
9	genres	5043 non-null	object
10	actor_1_name	5036 non-null	object
11	movie_title	5043 non-null	object
12	num_voted_users	5043 non-null	int64
13	cast_total_facebook_likes	5043 non-null	int64
14	actor_3_name	5020 non-null	object
15	facenumber_in_poster	5030 non-null	float64
16	plot_keywords	4890 non-null	object
17	movie_imdb_link	5043 non-null	object
18	num_user_for_reviews	5022 non-null	float64
19	language	5029 non-null	object
20	country	5038 non-null	object
21	content_rating	4740 non-null	object
22	budget	4551 non-null	float64
23	title_year	4935 non-null	float64
24	actor_2_facebook_likes	5030 non-null	float64
25	imdb_score	5043 non-null	float64
26	aspect_ratio	4714 non-null	float64
27	movie_facebook_likes	5043 non-null	int64

Về giá trị dữ liệu, giá trị của các biến trong bộ dữ liệu có sự chênh lệch lớn giữa các biến.

B.2.2. Kiểm tra chất lượng dữ liệu

Để đánh giá chất lượng dữ liệu, ta tiến hành kiểm tra xem dữ liệu có trùng lặp (duplicate), có khuyết (missing), có nhiễu (noise), có mâu thuẫn (conflict) hay không.

Kiểm tra dữ liệu có trùng lặp (duplicate):

```
# 45
```

Ta thấy bộ dữ liệu có 45 dữ liệu trùng lặp.

Kiểm tra dữ liệu khuyết (missing)

Bảng 13: Thống kê dữ liệu khuyết

Biến số	Số lượng NaN
color	19
director_name	104
num_critic_for_reviews	50
duration	15
director_facebook_likes	104
actor_3_facebook_likes	23
actor_2_name	13
actor_1_facebook_likes	7
gross	884
genres	0
actor_1_name	7
movie_title	0
num_voted_users	0
cast_total_facebook_likes	0
actor_3_name	23
facenumber_in_poster	13
plot_keywords	153
movie_imdb_link	0
num_user_for_reviews	21
language	14
country	5
content_rating	303
budget	492
title_year	108
actor_2_facebook_likes	13
imdb_score	0
aspect_ratio	329
movie_facebook_likes	0

Ta thấy bộ dữ liệu có:

- 19 giá trị khuyết của biến "**color**".
- 104 giá trị khuyết của biến "**director_name**".
- 50 giá trị khuyết của biến "**num_critic_for_reviews**".
- 15 giá trị khuyết của biến "**duration**".
- 104 giá trị khuyết của biến "**director_facebook_likes**".

- 23 giá trị khuyết của biến **"actor_3_facebook_likes"**.
- 13 giá trị khuyết của biến **"actor_2_name"**.
- 7 giá trị khuyết của biến **"actor_1_facebook_likes"**.
- 884 giá trị khuyết của biến **"gross"**.
- 7 giá trị khuyết của biến **"actor_1_name"**.
- 23 giá trị khuyết của biến **"actor_3_name"**.
- 13 giá trị khuyết của biến **"facenumber_in_poster"**.
- 153 giá trị khuyết của biến **"plot_keywords"**.
- 21 giá trị khuyết của biến **"num_user_for_reviews"**.
- 14 giá trị khuyết của biến **"language"**.
- 5 giá trị khuyết của biến **"country"**.
- 303 giá trị khuyết của biến **"content_rating"**.
- 492 giá trị khuyết của biến **"budget"**.
- 108 giá trị khuyết của biến **"title_year"**.
- 13 giá trị khuyết của biến **"actor_2_facebook_likes"**.
- 329 giá trị khuyết của biến **"aspect_ratio"**.

Ta nhận thấy: Tập dữ liệu có tối đa 5043 dòng, tuy nhiên rất nhiều biến bị khuyết dữ liệu. Đặc biệt là hai biến tài chính: "gross" (doanh thu) chỉ có 4159 giá trị và "budget" (ngân sách) chỉ có 4551 giá trị. Điều này cần được xử lý kỹ (điền khuyết hoặc loại bỏ) trước khi đưa vào mô hình.

Kiểm tra dữ liệu nhiễu (noise):

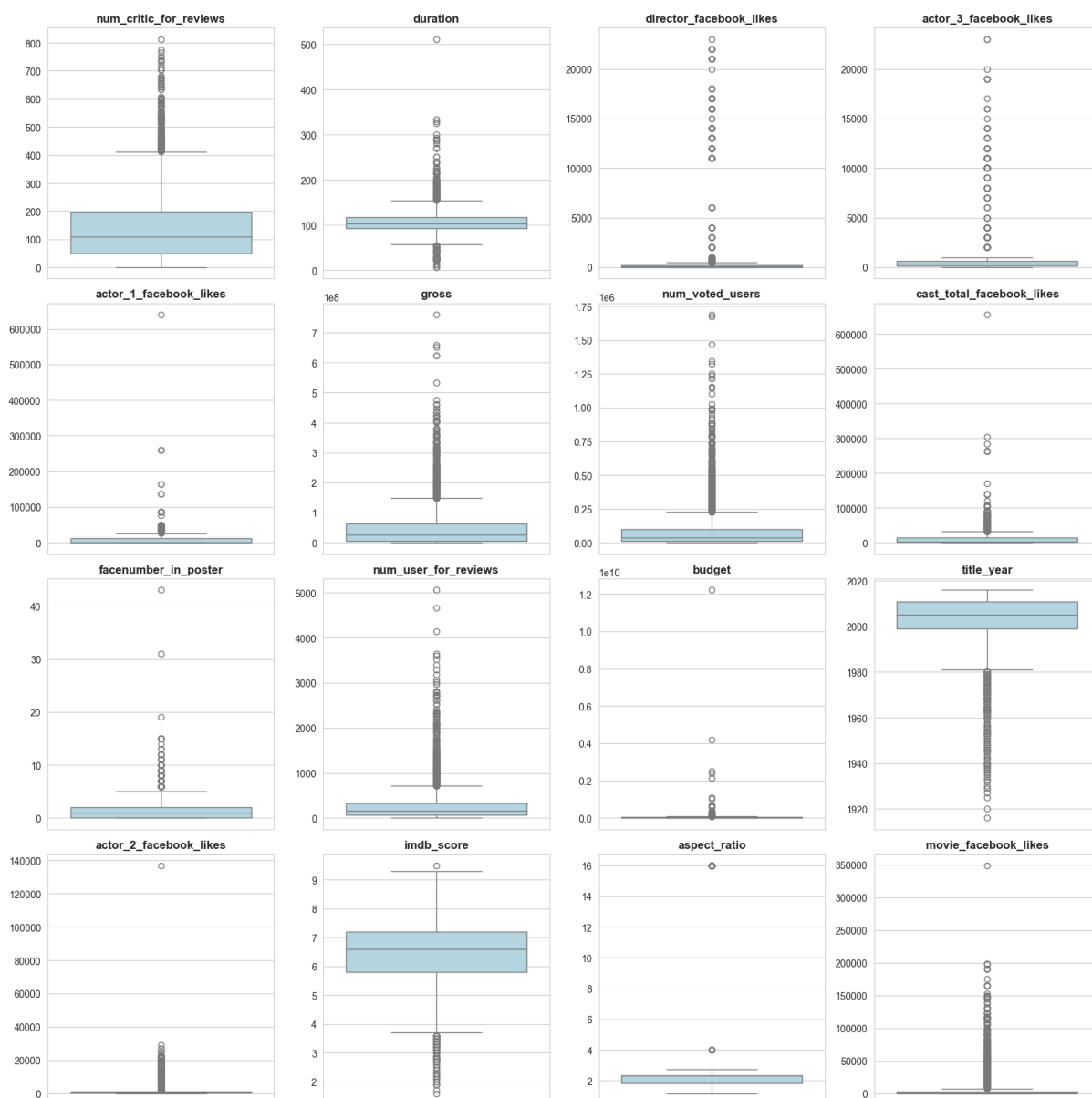
Trước tiên ta xem xét các thống kê mô tả của từng biến numeric trong bộ dữ liệu:

Bảng 14: Thống kê mô tả các biến định lượng (dataset.describe)

Biến số	count	mean	std	min	25%	50%	75%	max
num_critics_for_reviews	4993.0	1.401943e+02	1.216017e+02	1.00	50.00	110.00	195.00	8.130000e+02
duration	5028.0	1.072011e+02	2.519744e+01	7.00	93.00	103.00	118.00	5.110000e+02
director_facebook_likes	4939.0	6.865092e+02	2.813329e+03	0.00	7.00	49.00	194.50	2.300000e+04
actor_3_facebook_likes	5020.0	6.450098e+02	1.665042e+03	0.00	133.00	371.50	636.00	2.300000e+04
actor_1_facebook_likes	5036.0	6.560047e+03	1.502076e+04	0.00	614.00	988.00	11000.00	6.400000e+05
gross	4159.0	4.846841e+07	6.845299e+07	162.00	5340987.50	25517500.00	62309437.50	7.605058e+08
num_voted_users	5043.0	8.366816e+04	1.384853e+05	5.00	8593.50	34359.00	96309.00	1.689764e+06
cast_total_facebook_likes	5043.0	9.699064e+03	1.816380e+04	0.00	1411.00	3090.00	13756.50	6.567300e+05
facenumber_in_poster	5030.0	1.371173e+00	2.013576e+00	0.00	0.00	1.00	2.00	4.300000e+01
num_user_for_reviews	5022.0	2.727708e+02	3.779829e+02	1.00	65.00	156.00	326.00	5.060000e+03
budget	4551.0	3.975262e+07	2.061149e+08	218.00	6000000.00	20000000.00	45000000.00	1.221550e+10
title_year	4935.0	2.002471e+03	1.247460e+01	1916.00	1999.00	2005.00	2011.00	2.016000e+03
actor_2_facebook_likes	5030.0	1.651754e+03	4.042439e+03	0.00	281.00	595.00	918.00	1.370000e+05
imdb_score	5043.0	6.442138e+00	1.125116e+00	1.60	5.80	6.60	7.20	9.500000e+00
aspect_ratio	4714.0	2.220403e+00	1.385113e+00	1.18	1.85	2.35	2.35	1.600000e+01
movie_facebook_likes	5043.0	7.525965e+03	1.932045e+04	0.00	0.00	166.00	3000.00	3.490000e+05

Ta nhận thấy:

- Biến **"duration"** (thời lượng) có min = 7 phút và max = 511 phút. Dù hơi lớn nhưng vẫn nằm trong khoảng hợp lý của thực tế (phim ngắn hoặc các phim tài liệu cực dài).
- Rất nhiều biến như "director_facebook_likes", các biến "actor_facebook_likes", "movie_facebook_likes" và "facenumber_in_poster" có giá trị min = 0. Điều này hợp lý (do không dùng mạng xã hội hoặc poster phong cảnh) nhưng sẽ gây khó khăn nếu cần biến đổi biến.
- Biến **"budget"** có giá trị lớn nhất (max) cực kỳ cao (lên tới 1.22×10^{10} , tức khoảng 12 tỷ). Mức này chênh lệch vô cùng lớn so với phân vị 75% (chỉ 45 triệu). Điều này cho thấy biến này có chứa các giá trị ngoại lai rất nặng, hoặc dữ liệu bị lẫn lộn đơn vị tiền tệ (ví dụ: tiền Won Hàn Quốc, tiền Yên Nhật) thay vì chỉ dùng USD. Ta có thể sẽ xem đó là nhiễu hoặc cần quy đổi lại.
- Biến **"title_year"** có khoảng giá trị [1916, 2016], phản ánh đúng khoảng thời gian phát hành phim hợp lý.
- Biến **"imdb_score"** có giá trị trong khoảng [1.60, 9.50]. Điều này hoàn toàn hợp lý vì điểm IMDB nằm trên thang điểm [0, 10].
- Nhìn chung, tất cả các biến đều có dữ liệu hợp lệ, ngoại trừ việc bị khuyết ở nhiều cột và dấu hiệu sai lệch đơn vị ở biến tài chính thì không phát hiện các giá trị bất thường vi phạm ràng buộc miền giá trị.
- Ta sẽ kiểm tra boxplot của từng biến để xem dữ liệu ngoại lai (outlier) thế nào:



Hình 14: Biểu đồ Boxplot của các biến định lượng.

Từ hình ảnh boxplot, ta thấy tất cả các biến có rất nhiều outlier.

Kiểm tra dữ liệu mâu thuẫn (conflict)

Như đã nhận xét ở phần kiểm tra noise data, các biến đều có dữ liệu hợp lệ, không phát hiện các giá trị bất thường vi phạm ràng buộc miền giá trị của từng biến.

Ta sẽ kiểm tra xem liệu có trường hợp mâu thuẫn về mặt logic thông thường hay không.

- Trường hợp lượt thích trang Facebook của 1 diễn viên chính lại lớn hơn tổng số lượt thích của toàn bộ dàn diễn viên cộng lại:

0

- Trường hợp phim không có khán giả nào bình chọn nhưng lại có điểm đánh giá trung bình:

0

- Trường hợp phim có ngân sách sản xuất (budget) hoặc doanh thu (gross) mang giá trị âm

0

Ta kiểm tra suy rộng hơn một chút:

- Trường hợp phim được đánh giá là kiệt tác (Điểm $\text{imdb_score} \geq 8$) nhưng doanh thu phòng vé lại thảm họa ($\text{Gross} < 50,000$ USD):

Bảng 15: Phim có điểm cao nhưng doanh thu thấp

movie_title	title_year	imdb_score	gross	budget
Raging Bull	1980.0	8.3	23383987.0	18000000.0
Children of Heaven	1997.0	8.3	925402.0	180000.0
A Charlie Brown Christmas	1965.0	8.3	3154800.0	150000.0
Elite Squad	2007.0	8.1	8060.0	4000000.0
Nothing But a Man	1964.0	8.1	12438.0	160000.0

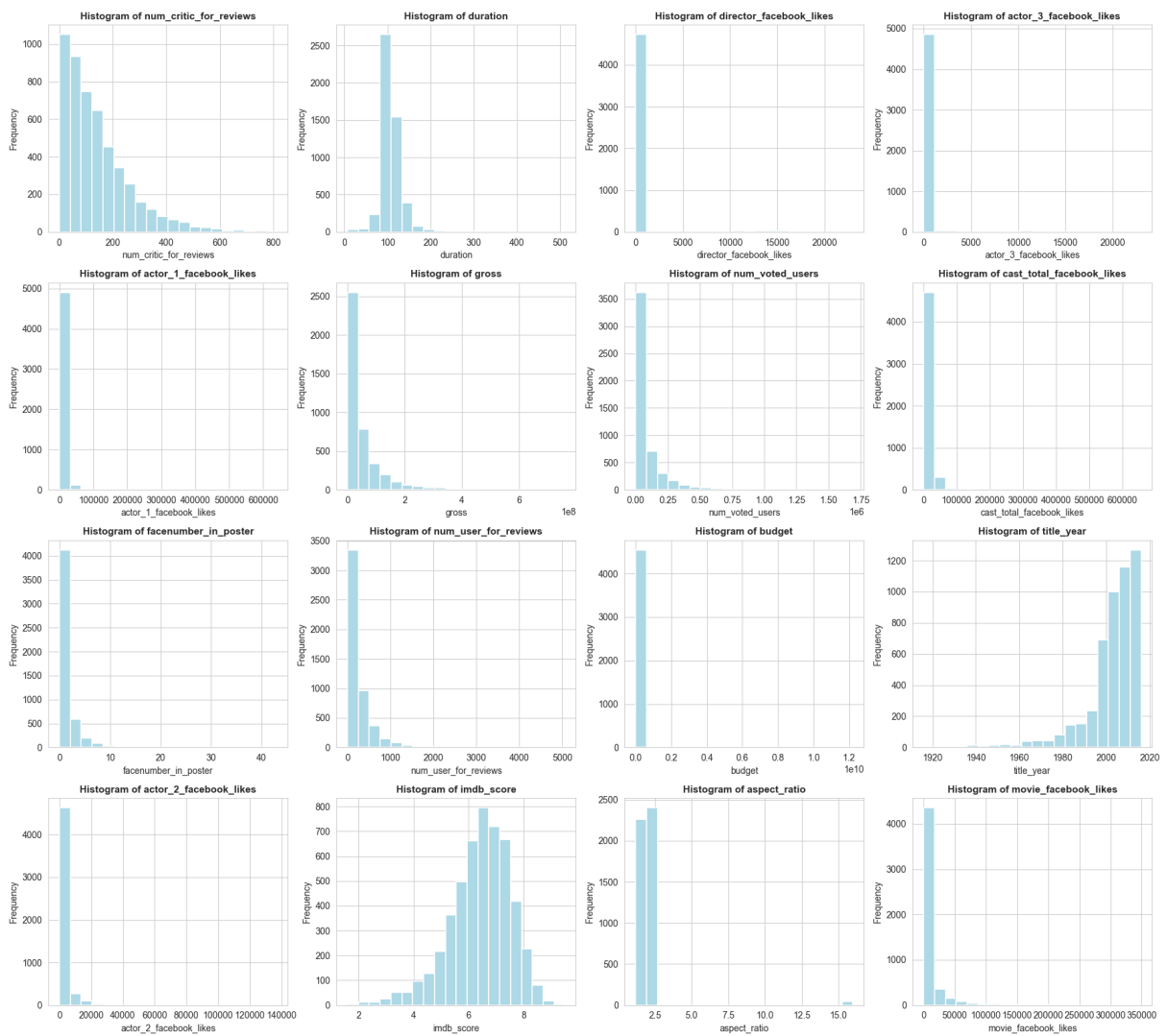
- Trường hợp phim bị khán giả và giới phê bình chê thậm tệ (Điểm imdb_score ≤ 4.5) nhưng doanh thu lại cao kỷ lục (Gross > 100,000,000 USD):

Bảng 16: Phim có điểm thấp nhưng doanh thu cao

movie_title	title_year	imdb_score	gross	budget
Alvin and the Chipmunks: The Squeakquel	2009.0	4.5	219613391.0	75000000.0
Alvin and the Chipmunks: Chipwrecked	2011.0	4.4	133103929.0	75000000.0
Nutty Professor II: The Klumps	2000.0	4.3	123307945.0	65000000.0
The Last Airbender	2010.0	4.2	131564731.0	150000000.0
Fifty Shades of Grey	2015.0	4.1	166147885.0	40000000.0

Ta thấy bộ dữ liệu không có dữ liệu mâu thuẫn. Có 13 quan trắc cho thấy mối quan hệ không phù hợp với logic thông thường của thị trường, chẳng hạn như các bộ phim được đánh giá vô cùng cao nhưng lại thất bại hoàn toàn về doanh thu phòng vé, hoặc ngược lại bị chê tới tấp nhưng lại thành công rực rỡ về mặt thương mại.

B.2.3. Kiểm tra phân phối



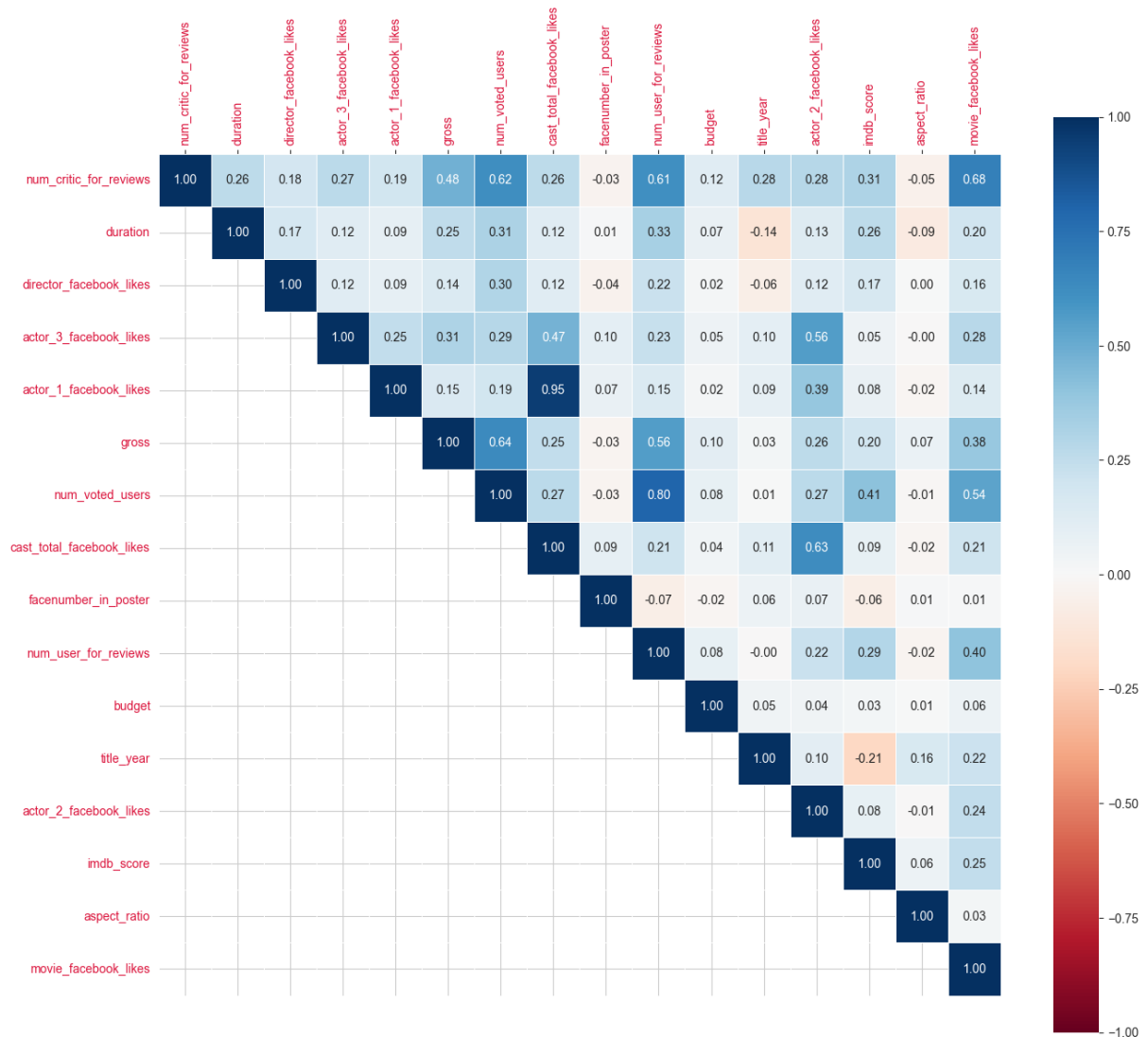
Hình 15: Biểu đồ Histogram của các biến định lượng.

Từ các biểu đồ histogram, ta thấy:

- Biến **imdb_score** có phân phối chuẩn/ gần chuẩn (đồ thị hình quả chuông đối xứng rõ rệt). Biến **duration** cũng có phân phối tập trung quanh giá trị trung bình (khoảng 100 phút) nhưng có đuôi hơi lệch sang phải.
- Hầu hết các biến đếm, tài chính và tương tác mạng xã hội bao gồm **num_critic_for_reviews**, **gross**, **budget**, **num_voted_users**, **num_user_for_reviews**, **facenumber_in_poster** và toàn bộ các biến lượt likes Facebook (**director**, **actor_1/2/3**, **cast_total**, **movie**) đều có phân phối lệch phải rất mạnh (right-skewed). Dữ liệu tập trung ở mức giá trị thấp và kéo đuôi rất dài về bên phải.
- Biến **title_year** có phân phối lệch trái (left-skewed), cho thấy phần lớn các bộ phim trong tập dữ liệu này được sản xuất trong khoảng thời gian gần đây (từ những năm 1990 trở lại đây).
- Biến **aspect_ratio** có phân phối chia thành 1-2 cụm tập trung cao độ biến, điều này cho thấy tỷ lệ khung hình phim chỉ xoay quanh một vài chuẩn điện ảnh phổ biến (như 1.85 hoặc 2.35) chứ không phải là một dải giá trị liên tục.
- Riêng biến **budget** gần như chỉ hiển thị một cột duy nhất. Điều này cho thấy sự tồn tại của các giá trị nhiều vô cùng lớn đã làm biến dạng hoàn toàn thang đo của trục hoành.

Từ phân phối của các biến, dự đoán rằng có lẽ ta cần phải thực hiện phép biến đổi biến trên phần lớn các biến, đặc biệt là nhóm biến lệch phải, để đưa chúng về dạng gần chuẩn hơn. Đồng thời, cần có biện pháp xử lý nhiễu cho biến **budget**.

B.2.4. Phân tích quan hệ giữa các biến



Hình 17: Ma trận tương quan (Heatmap) giữa các biến định lượng.

Từ ma trận tương quan giữa các biến, ta nhận thấy:

- Biến **“imdb_score”** (số điểm) có tương quan thuận tương đối mạnh với các biến tương tác của khán giả là **“num_voted_users”** và **“num_user_for_reviews”** với hệ số tương quan lần lượt là 0.41 và 0.29. Điều này cho thấy mức độ quan tâm và đánh giá của cộng đồng mạng tỷ lệ thuận với số điểm của phim. Qua đây ta cũng dự đoán sự hiện diện của các biến này có vai trò quan trọng trong mô hình dự báo số điểm phim.
- Biến **“actor_1_facebook_likes”** và **“cast_total_facebook_likes”** có hệ số tương quan cực kỳ cao lên tới 0.95 (gần như tuyệt đối). Điều này hoàn toàn logic vì lượt thích của diễn viên chính thường chiếm phần lớn tổng số lượt thích của cả dàn diễn viên. Việc giữ lại cả 2 biến này trong mô hình sẽ rất dễ dẫn đến hiện tượng đa cộng

tuyến (multicollinearity).

- Biến **“num_voted_users”** và **“num_user_for_reviews”** cũng có tương quan thuận và rất chặt với nhau với hệ số tương quan là 0.80. Điều này cũng có thể dẫn đến đa cộng tuyến.
- Các cặp biến liên quan đến diễn viên phụ như **“actor_2_facebook_likes”** – **“cast_total_facebook_likes”** (0.63), hay cặp **“num_critic_for_reviews”** – **“movie_facebook_likes”** (0.68) có hệ số tương quan từ trung bình đến khá, cho thấy chúng có sự đồng biến nhất định.
- Khác với các quy luật thông thường, biến **“budget”** (ngân sách) trong tập dữ liệu này lại có tương quan rất yếu với **“imdb_score”** (chỉ đạt 0.03). Các biến như **“facenumber_in_poster”** (-0.06), **“aspect_ratio”** (0.06) hầu như không có tương quan với **“imdb_score”**. Có thể dự đoán các biến này đóng góp rất ít vào hiệu suất của mô hình dự báo số điểm phim.

Kiểm tra đa cộng tuyến

Bảng 17: Hệ số phóng đại phương sai (VIF) của các biến

Biến số (Variables)	VIF
cast_total_facebook_likes	299.270598
actor_1_facebook_likes	206.201275
actor_2_facebook_likes	16.807017
actor_3_facebook_likes	7.127722
num_voted_users	4.152888
num_user_for_reviews	3.209081
num_critic_for_reviews	2.873496
gross	2.094568
movie_facebook_likes	1.779129
imdb_score	1.400634
duration	1.302716
director_facebook_likes	1.201212
title_year	1.117636
facenumber_in_poster	1.043627
budget	1.035095
aspect_ratio	1.020356

Từ hệ số phóng đại VIF, ta thấy có hiện tượng đa cộng tuyến rất nghiêm trọng, đặc biệt là ở biến **“cast_total_facebook_likes”** (VIF > 299) và **“actor_1_facebook_likes”** (VIF > 206). Ngoài ra, biến **“actor_2_facebook_likes”** cũng vượt mức cho phép (VIF > 16).

C Chuẩn bị dữ liệu

C.1 Tổng quan

Phần chuẩn bị dữ liệu là bước chuyển đổi bộ IMDb 5000 Movie Dataset từ dữ liệu thô sang dạng ma trận số có thể đưa vào mô hình hồi quy tuyến tính. Bộ dữ liệu ban đầu chứa nhiều kiểu biến: biến định lượng như duration, budget, các biến lượt thích Facebook; biến phân loại như color, language, country, content_rating; và các biến định danh hoặc văn bản như tên đạo diễn, tên diễn viên, từ khóa phim, đường dẫn IMDb. Nếu sử dụng trực tiếp, dữ liệu có thể gây sai lệch mô hình do missing values, duplicated records, outliers, skewed distributions, multicollinearity, high-cardinality categorical variables và data leakage.

Mục tiêu của mục C là thiết kế một quy trình tiền xử lý có tính tái lập, trong đó mọi tham số thống kê chỉ được học từ tập huấn luyện rồi áp dụng sang tập kiểm tra. Nguyên tắc này đặc biệt quan trọng vì nó ngăn thông tin từ test set rò rỉ vào quá trình huấn luyện. Trong báo cáo này, phần mô tả phương pháp và lý do lựa chọn được trình bày trước; các đoạn mã chỉ đóng vai trò minh họa cách nhóm hiện thực hóa từng quyết định xử lý.

C.1.1. Tóm tắt kết quả chính

Bảng 18: Tóm tắt các mốc dữ liệu quan trọng trong mục C.

Giai đoạn	Train	Test	Ghi chú
Sau chọn biến	5043 dòng, 12 cột		Giữ các biến phù hợp cho mô hình dự đoán
Sau chia dữ liệu	4034	1009	Tỷ lệ 80%–20%, random_state=42
Sau xóa trùng lặp	3970	–	Train có 64 dòng trùng lặp được loại bỏ
Sau xử lý missing và encoding	3885	981	Dữ liệu sạch, không còn missing values
Ma trận cuối cùng	(3885, 14)	(981, 14)	Ma trận X có 14 đặc trưng

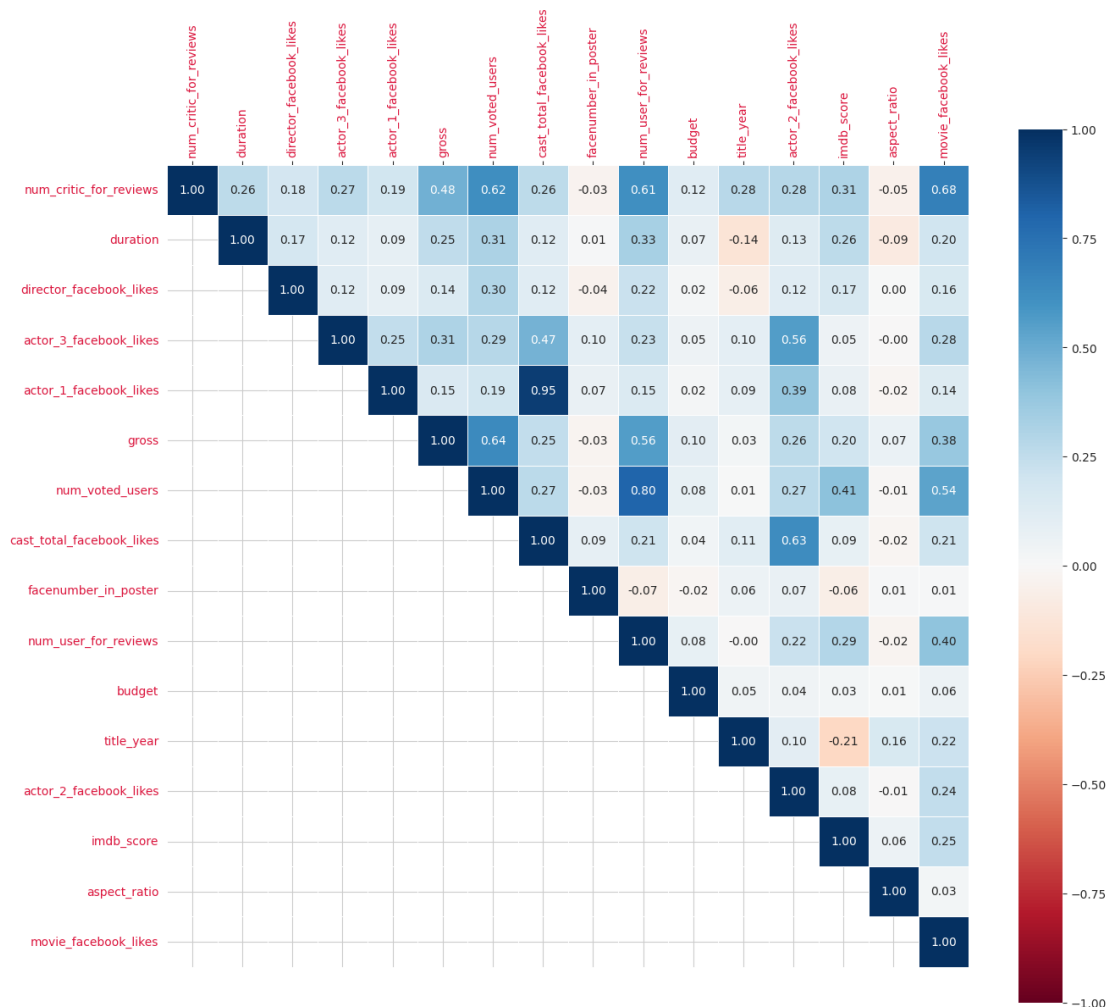
C.1.2. Luồng xử lý tổng quát

1. Chọn biến dựa trên lý thuyết, tương quan, VIF, khả năng leakage và cardinality.
2. Chia dữ liệu thành train/test trước khi fit bất kỳ tham số tiền xử lý nào.
3. Làm sạch dữ liệu: xử lý duplicate, conflict, outlier và missing values.
4. Biến đổi phân phối bằng Box-Cox để giảm lệch phải và ổn định mô hình OLS.
5. Gom nhóm nhãn hiếm và one-hot encoding biến phân loại.
6. Tách X, y và đóng gói toàn bộ quy trình bằng DataPipeline.

C.2 Lựa chọn biến

C.2.1. Cơ sở thống kê: tương quan và VIF

Trước khi chọn biến, nhóm phân tích ma trận tương quan và hệ số phóng đại phương sai VIF. Hình 18 cho thấy một số nhóm biến có tương quan tuyến tính mạnh, đặc biệt là nhóm lượt thích của diễn viên và tổng lượt thích của dàn diễn viên. Kết quả VIF cho thấy `cast_total_facebook_likes` có VIF xấp xỉ 299.27, `actor_1_facebook_likes` xấp xỉ 206.20 và `actor_2_facebook_likes` xấp xỉ 16.81. Đây là dấu hiệu đa cộng tuyến nghiêm trọng.



Hình 18: Ma trận tương quan giữa các biến định lượng trước khi chọn biến.

Về lý thuyết, đa cộng tuyến xảy ra khi một biến giải thích có thể được biểu diễn gần đúng bằng tổ hợp tuyến tính của các biến khác. Trong OLS, hệ số được ước lượng bởi:

$$\hat{\beta} = (X^T X)^{-1} X^T y.$$

Khi các cột của X phụ thuộc tuyến tính mạnh, $X^T X$ gần suy biến, làm nghiệm OLS kém ổn định. Vì vậy, loại bỏ các biến gây đa cộng tuyến là cần thiết để tăng độ tin cậy của hệ số hồi quy.

C.2.2. Quy tắc loại bỏ và giữ lại biến

Bảng 19: Nhóm biến bị loại bỏ và lý do xử lý.

Tiêu chí	Biến bị loại bỏ	Lý do loại bỏ
Đa cộng tuyến / trùng thông tin	cast_total_facebook_likes num_user_for_reviews	Các biến này trùng hoặc gần trùng thông tin với những biến tương tác khác, làm tăng VIF và khiến ước lượng OLS kém ổn định.
Tương quan tuyến tính yếu	facenumber_in_poster aspect_ratio	Hai biến này không thể hiện quan hệ tuyến tính rõ ràng với imdb_score, nên đóng góp dự báo thấp trong mô hình tuyến tính.
Rò rỉ dữ liệu tương lai	gross num_voted_users num_critic_for_reviews movie_facebook_likes	Đây là các thông tin thường chỉ có sau khi phim công chiếu; nếu giữ lại, mô hình có thể “nhìn thấy tương lai” và đánh giá quá lạc quan.
Cardinality cao	director_name, genres tên diễn viên plot_keywords movie_imdb_link	Các biến này có quá nhiều nhãn; mã hóa trực tiếp sẽ tạo nhiều cột dummy thừa, làm tăng nguy cơ overfitting và ma trận suy biến.

Sau bước chọn lọc, nhóm giữ lại 12 đặc trưng: color, duration, director_facebook_likes, actor_3_facebook_likes, actor_1_facebook_likes, language, country, content_rating, budget, title_year, actor_2_facebook_likes và imdb_score. Kích thước dữ liệu lúc này là (5043, 12). Cách giải quyết của nhóm là loại bỏ các cột không phù hợp trước, sau đó mới chia dữ liệu và xử lý missing/outlier; đoạn mã dưới đây chỉ minh họa thao tác thực thi quyết định này.


```

1 if 'movie_title' in dataset.columns:
2     dataset['movie_title'] = dataset['movie_title'].str.strip()
3     dataset = dataset.set_index('movie_title')
4
5 cols_to_drop = [
6     'cast_total_facebook_likes', 'num_user_for_reviews',
7     'facenumber_in_poster', 'aspect_ratio',
8     'gross', 'num_voted_users', 'num_critic_for_reviews',
9     'movie_facebook_likes', 'director_name', 'genres',
10    'actor_1_name', 'actor_2_name', 'actor_3_name',
11    'plot_keywords', 'movie_imdb_link'
12 ]
13
14 dataset = dataset.drop(columns=[c for c in cols_to_drop if c in
    dataset.columns])

```

Listing 9: Mã minh họa chọn biến và đặt tên phim làm index.

C.3 Chia tập dữ liệu

Dữ liệu được chia theo tỷ lệ 80%–20% với `random_state=42`. Kết quả là train set có 4034 dòng và test set có 1009 dòng. Điểm quan trọng là việc chia dữ liệu được thực hiện trước các bước fit tham số tiền xử lý. Điều này đảm bảo median, mode, ngưỡng Winsorization, tham số KNN Imputer, hệ số Box-Cox và cấu trúc one-hot encoding đều chỉ được học từ train set.

Cách giải quyết là chia dữ liệu ngay sau khi chọn biến để mọi tham số tiền xử lý về sau chỉ được fit trên train set. Đoạn mã sau chỉ minh họa bước chia dữ liệu.

```

1 from sklearn.model_selection import train_test_split
2
3 train_dataset, test_dataset = train_test_split(
4     dataset,
5     test_size=0.2,
6     random_state=42
7 )

```

Listing 10: Mã minh họa chia dữ liệu thành train/test trước khi fit preprocessing.

Nếu tính toán các tham số tiền xử lý trên toàn bộ dữ liệu trước khi chia, thông tin từ test set sẽ gián tiếp ảnh hưởng đến mô hình. Đây là lỗi *data leakage*, làm kết quả đánh giá trên test set trở nên lạc quan giả tạo.

C.4 Làm sạch dữ liệu

C.4.1. Dữ liệu trùng lặp

Nhóm phát hiện 64 dòng trùng lặp trên train set. Các dòng này được loại bỏ thông qua hàm `drop_duplicates(keep='first')` trên cả tập train và test. Sau bước này, train set giảm từ (4034, 12) xuống (3970, 12).

Dữ liệu trùng lặp làm một số phim xuất hiện nhiều lần trong quá trình tối ưu. Với OLS, điều này tương đương tăng trọng số cho các quan sát lặp lại, khiến đường hồi quy có thể bị kéo lệch về các mẫu đó.

C.4.2. Dữ liệu mâu thuẫn

Nhóm kiểm tra các quy tắc logic sau:

- `duration` phải lớn hơn 0.
- `budget` không được âm.
- `title_year` không được lớn hơn 2016.

Kết quả trên train set đều bằng 0 trường hợp vi phạm. Tuy nhiên, quy trình vẫn định nghĩa hàm xử lý `conflict` để chuyển các giá trị mâu thuẫn thành NaN. Cách làm này tốt hơn xóa dòng ngay lập tức vì các lỗi logic được đưa về cùng nhóm missing values để xử lý nhất quán.

Cách giải quyết là không xóa ngay các dòng lỗi logic, mà chuyển giá trị bất hợp lệ thành missing value để xử lý nhất quán ở bước sau. Đoạn mã dưới đây minh họa quy tắc này.

```

1 def handle_conflict(df):
2     df.loc[df['duration'] <= 0, 'duration'] = np.nan
3     df.loc[df['budget'] < 0, 'budget'] = np.nan
4     df.loc[df['title_year'] > 2016, 'title_year'] = np.nan
5     return df
6
7 train_dataset = handle_conflict(train_dataset)
8 test_dataset = handle_conflict(test_dataset)

```

Listing 11: Mã minh họa chuẩn hóa dữ liệu mâu thuẫn thành missing values.

C.4.3. Dữ liệu nhiễu và ngoại lệ

Dữ liệu phim có nhiều biến lệch phải, đặc biệt là budget và các biến Facebook likes. Một số phim có ngân sách hoặc lượng tương tác rất lớn, tạo ra outliers. Với OLS, outliers có ảnh hưởng mạnh vì mô hình tối thiểu hóa tổng bình phương sai số:

$$RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

Sai số lớn bị bình phương, nên một vài điểm cực đoan có thể kéo đường hồi quy lệch khỏi xu hướng chính.

Nhóm sử dụng Winsorization theo phân vị 1% và 99%. Các ngưỡng này được học trên train set rồi áp dụng cho cả train và test.

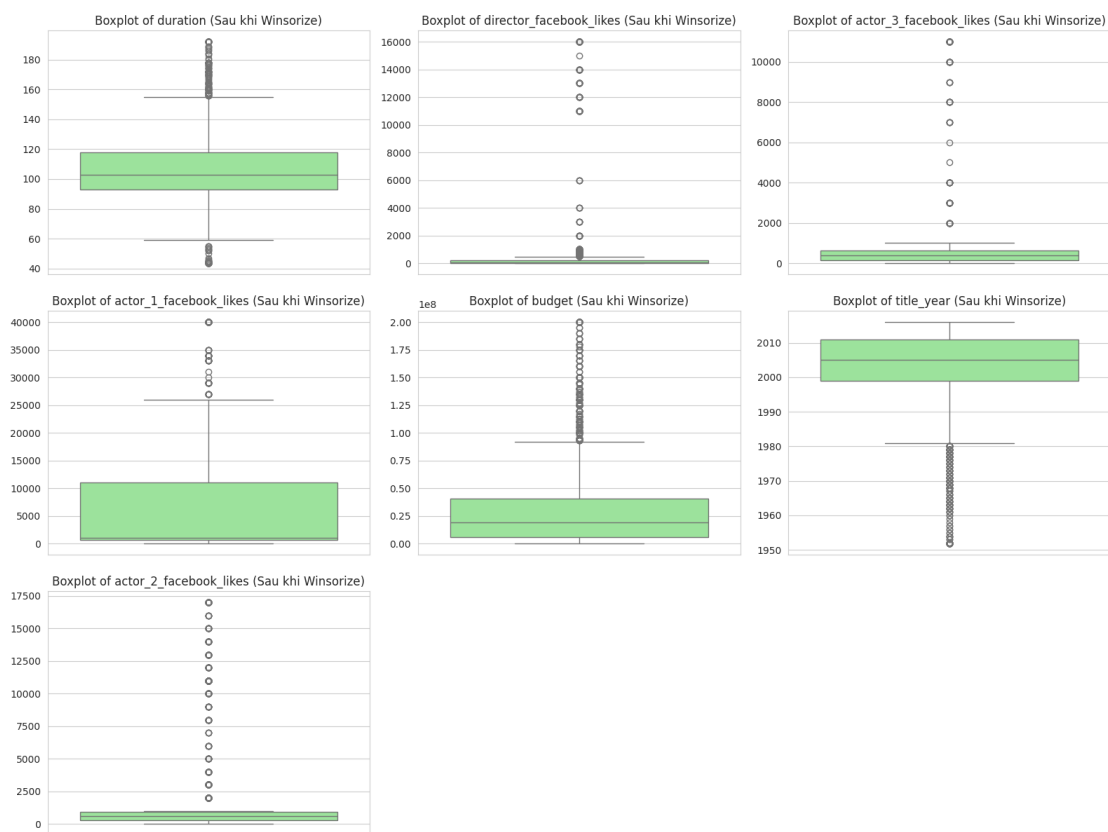
Cách giải quyết là học ngưỡng cắt đuôi từ train set, rồi áp dụng cùng ngưỡng cho cả train và test. Đoạn mã dưới đây minh họa cách triển khai Winsorization.

```

1 num_cols = train_dataset.select_dtypes(include=[np.number]).
  columns
2 lower_bounds, upper_bounds = {}, {}
3
4 for col in num_cols:
5     if col != 'imdb_score':
6         lower_bounds[col] = train_dataset[col].quantile(0.01)
7         upper_bounds[col] = train_dataset[col].quantile(0.99)
8
9 for col in num_cols:
10    if col != 'imdb_score':
11        train_dataset[col] = train_dataset[col].clip(lower_bounds[col], upper_bounds[col])
12        test_dataset[col] = test_dataset[col].clip(lower_bounds[col], upper_bounds[col])

```

Listing 12: Mã minh họa Winsorization dựa trên phân vị học từ train set.



Hình 19: Boxplot các biến định lượng sau khi Winsorization.

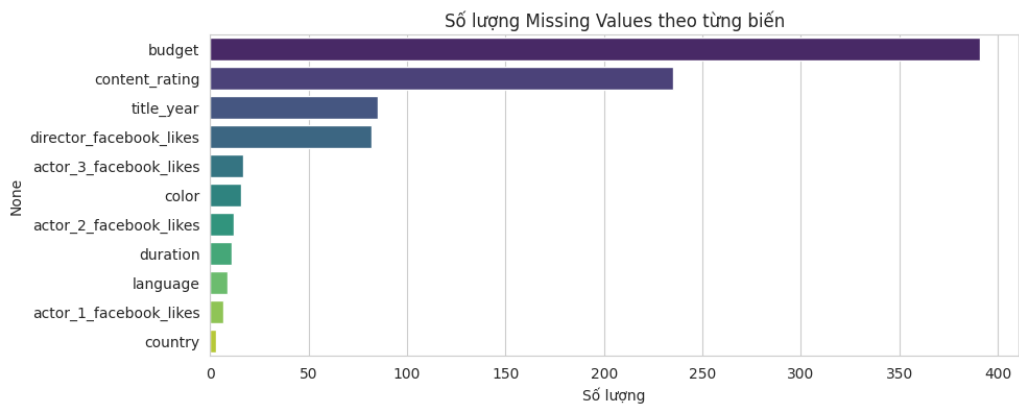
Hình 19 cho thấy các râu boxplot được thu gọn đáng kể. Điều này giúp giảm ảnh hưởng của giá trị cực đoan mà không làm mất quan sát.

C.4.4. Dữ liệu khuyết

Nhóm thống kê missing values trên train set như Bảng 20.

Bảng 20: Số lượng missing values trên train set.

Cột	Thiếu	Cột	Thiếu
budget	391	actor_2_facebook_likes	12
content_rating	235	duration	11
title_year	85	language	9
director_facebook_likes	82	actor_1_facebook_likes	7
actor_3_facebook_likes	17	country	3
color	16		



Hình 20: Biểu đồ số lượng missing values theo từng biến.

Chiến lược xử lý được thiết kế theo ý nghĩa từng biến:

Bảng 21: Chiến lược xử lý missing values.

Nhóm biến	Phương pháp	Lý do
imdb_score, title_year	Listwise deletion	Thiếu target thì không huấn luyện được; thiếu năm phát hành để làm mất ý nghĩa thời gian.
content_rating	Gán Unrated	Missing ở biến này có ý nghĩa riêng, không nên ép về mode.
Biến phân loại còn lại	Điền mode học từ train	Mode là nhân hợp lệ và đại diện cho nhóm phổ biến nhất.
Biến số thiếu ít	Điền median học từ train	Median bền vững với phân phối lệch và outliers.
budget	KNN Imputer, $k = 5$	Budget thiếu nhiều; KNN tận dụng phim tương đồng thay vì tạo đỉnh ảo tại median.

Sau bước này, tổng missing values ở cả train và test đều bằng 0.

Cách giải quyết là chọn chiến lược riêng cho từng nhóm biến thay vì dùng một quy tắc chung cho toàn bộ dữ liệu. Đoạn mã dưới đây minh họa trình tự xử lý missing values.

```

1 from sklearn.impute import KNNImputer
2
3 train_dataset = train_dataset.dropna(subset=['imdb_score', '
    title_year'])
4 test_dataset = test_dataset.dropna(subset=['imdb_score', '
    title_year'])
5
6 train_dataset['content_rating'] = train_dataset['content_rating'].
    fillna('Unrated')
7 test_dataset['content_rating'] = test_dataset['content_rating'].
    fillna('Unrated')
8
9 cat_cols = train_dataset.select_dtypes(exclude=[np.number]).
    columns.tolist()
10 num_cols = train_dataset.select_dtypes(include=[np.number]).
    columns.tolist()
11
12 for col in cat_cols:
13     if col != 'content_rating':
14         mode_val = train_dataset[col].mode()[0]
15         train_dataset[col] = train_dataset[col].fillna(mode_val)
16         test_dataset[col] = test_dataset[col].fillna(mode_val)
17
18 for col in num_cols:
19     if col not in ['imdb_score', 'budget']:
20         median_val = train_dataset[col].median()
21         train_dataset[col] = train_dataset[col].fillna(median_val)
22         test_dataset[col] = test_dataset[col].fillna(median_val)
23
24 knn_imputer = KNNImputer(n_neighbors=5)
25 knn_imputer.fit(train_dataset[num_cols])
26 train_dataset[num_cols] = knn_imputer.transform(train_dataset[
    num_cols])
27 test_dataset[num_cols] = knn_imputer.transform(test_dataset[
    num_cols])

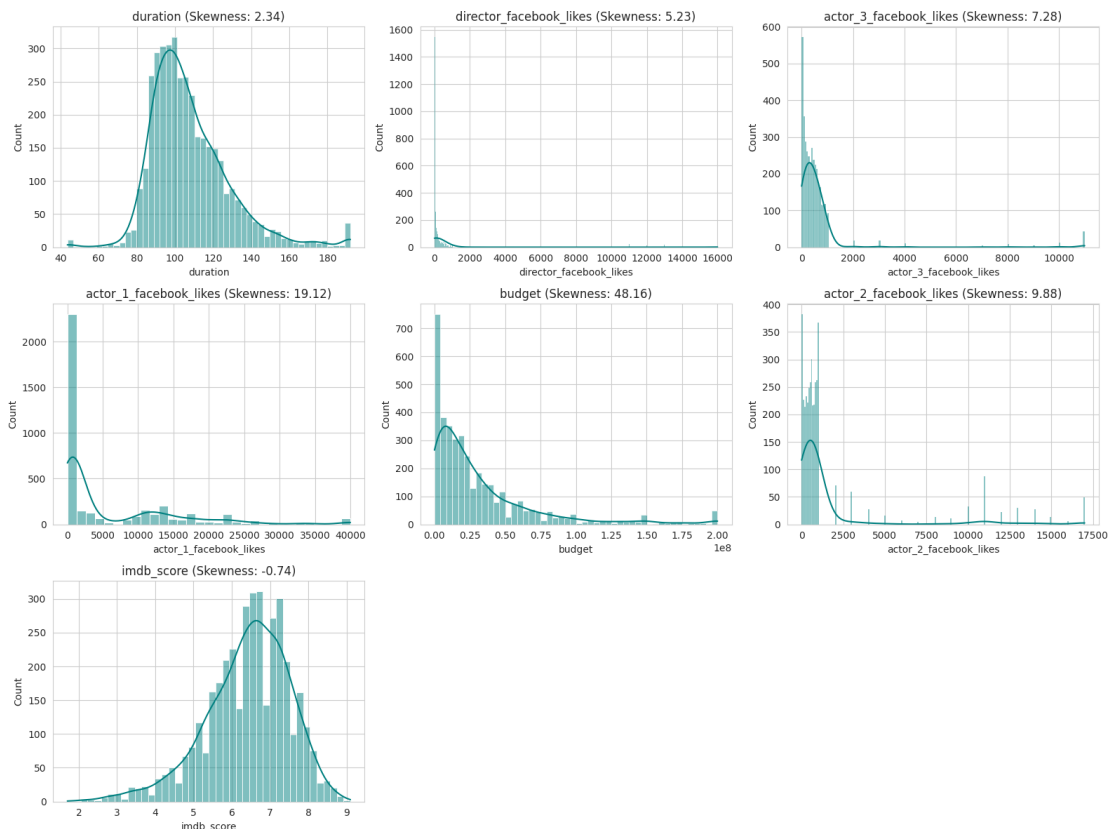
```

Listing 13: Mã minh họa xử lý missing values theo từng nhóm biến.

C.5 Xây dựng dữ liệu và biến đổi phân phối

C.5.1. Phân tích độ lệch bằng histogram

Hình 21 cho thấy các biến như `budget`, `director_facebook_likes`, `actor_1_facebook_likes`, `actor_2_facebook_likes` và `actor_3_facebook_likes` có phân phối lệch phải rõ rệt. Đây là dạng thường gặp ở dữ liệu tài chính và mạng xã hội: phần lớn quan sát có giá trị nhỏ, một số ít quan sát có giá trị rất lớn.



Hình 21: Histogram các biến định lượng trước khi biến đổi Box-Cox.

Phân phối lệch phải làm tăng ảnh hưởng của các điểm ở đuôi và có thể làm quan hệ giữa biến đầu vào với `imdb_score` kém tuyến tính. Do đó, nhóm sử dụng Box-Cox để nén đuôi phải và đưa phân phối các biến đầu vào về dạng ổn định hơn. Riêng `imdb_score` là biến mục tiêu nên không được đưa vào bước biến đổi Box-Cox; biến này chỉ được tách riêng làm vector y ở giai đoạn chuẩn bị ma trận cuối cùng.

C.5.2. Box-Cox và Likelihood Ratio Test

Phép biến đổi Box-Cox có dạng:

$$y^{(\lambda)} = \begin{cases} \frac{y^\lambda - 1}{\lambda}, & \lambda \neq 0, \\ \ln(y), & \lambda = 0. \end{cases}$$

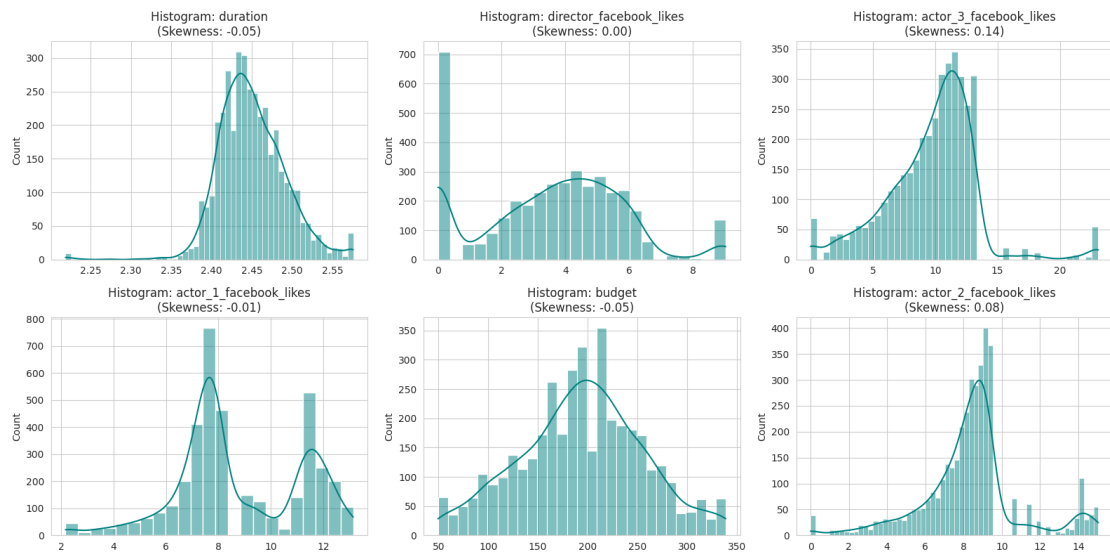
Vì Box-Cox yêu cầu dữ liệu dương, nhóm cộng thêm 1 trước khi kiểm định. Hệ số λ được chọn bằng cực đại hóa log-likelihood. Sau đó, nhóm dùng Likelihood Ratio Test để kiểm tra hai giả thuyết: $H_0 : \lambda = 1$ (không cần biến đổi) và $H_0 : \lambda = 0$ (log transformation là tối ưu).

Bảng 22: Hệ số λ Box-Cox học từ train set cho các biến đặc trưng lệch phải.

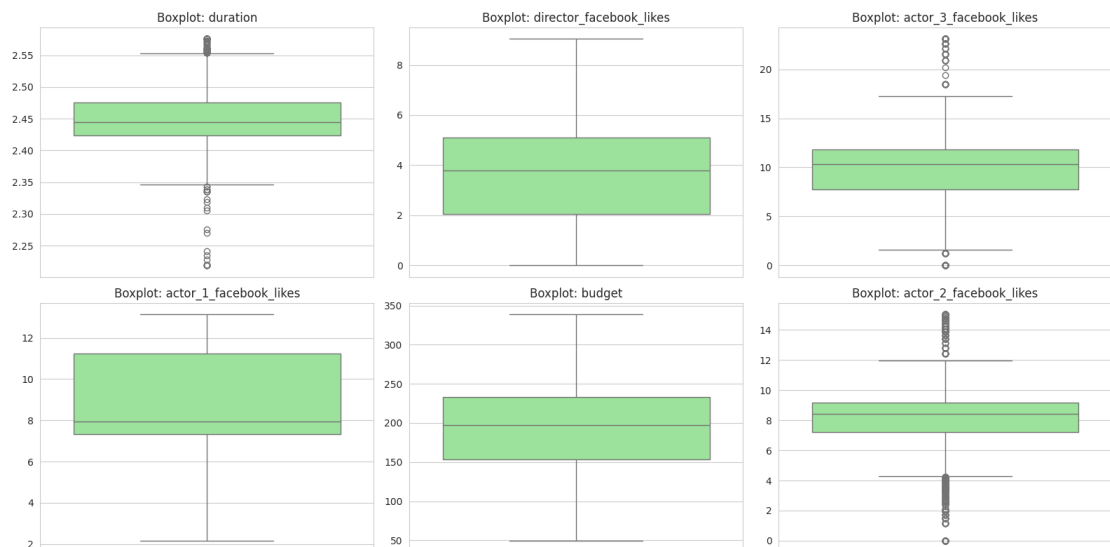
Biến	λ tối ưu	Diễn giải
duration	-0.3138	Cần biến đổi, không chỉ log
director_facebook_likes	-0.0144	Gần log nhưng LR test vẫn chọn Box-Cox
actor_3_facebook_likes	0.1726	Nén đuôi phải rõ rệt
actor_1_facebook_likes	0.0393	Gần log, vẫn cần Box-Cox
budget	0.2281	Cần biến đổi mạnh do lệch phải
actor_2_facebook_likes	0.0836	Cần nén đuôi phải

Trong phạm vi chuẩn bị dữ liệu cho mô hình hồi quy, nhóm áp dụng Box-Cox cho các biến đầu vào có phân phối lệch phải như budget, duration và các biến Facebook likes. Biến mục tiêu imdb_score được tách riêng làm vector y để phục vụ huấn luyện và đánh giá mô hình, nên không được xem là đặc trưng đầu vào trong bước biến đổi này.

C.5.3. Đánh giá sau biến đổi



Hình 22: Histogram các biến sau khi biến đổi Box-Cox.



Hình 23: Boxplot các biến sau khi biến đổi Box-Cox.

Sau Box-Cox, histogram trở nên cân đối hơn và boxplot giảm đáng kể các điểm cực đoan. Điều này giúp OLS ổn định hơn, giảm ảnh hưởng của các quan sát ở đuôi phải và cải thiện khả năng mô tả quan hệ tuyến tính.

C.6 Mã hóa biến phân loại

C.6.1. Vấn đề cardinality và ma trận thưa

Các biến phân loại cần được chuyển sang dạng số trước khi đưa vào OLS. Tuy nhiên, số nhãn duy nhất trên train set khá khác nhau: `color` có 2 nhãn, `language` có 41 nhãn, `country` có 60 nhãn và `content_rating` có 15 nhãn. Nếu one-hot encoding trực tiếp toàn bộ `language` và `country`, dữ liệu sẽ sinh thêm gần 100 cột dummy, phần lớn rất thưa.

Ma trận thưa làm tăng nguy cơ overfitting và có thể khiến ma trận thiết kế mất ổn định. Vì vậy, nhóm gom nhóm các nhãn hiếm trước khi mã hóa.

C.6.2. Gom nhóm và one-hot encoding

Cách giải quyết là giữ lại các nhãn phổ biến và gom các nhãn hiếm vào nhóm `Other`, qua đó giảm số chiều sau one-hot encoding. Cụ thể, nhóm không gom nhóm tùy ý mà dựa trên tần suất xuất hiện và ý nghĩa thực tế của từng biến:

Bảng 23: Quy tắc gom nhóm biến phân loại trước khi one-hot encoding.

Biến	Nhóm giữ lại	Cách gom nhóm và lý do
color	Giữ nguyên các mức hiện có	Biến này chỉ có 2 nhãn nên không cần gom nhóm. Sau drop_first=True, mô hình chỉ cần một dummy color_Color; mức còn lại đóng vai trò nhóm tham chiếu.
language	English	Tiếng Anh chiếm tỷ trọng áp đảo trong dữ liệu. Tất cả ngôn ngữ khác được gom thành Other để tránh tạo hàng chục cột dummy rất thưa. Sau khi dùng drop_first=True, dummy còn lại là language_Other, so sánh phim không dùng tiếng Anh với nhóm tham chiếu English.
country	USA, UK	Hai quốc gia này xuất hiện nhiều nhất và có ý nghĩa đại diện cho thị trường phim lớn trong bộ dữ liệu. Các quốc gia còn lại như France, Canada, Germany,... được gom thành Other. Sau mã hóa, nhóm tham chiếu là Other, còn hai dummy được giữ là country_UK và country_USA.
content_rating	R, PG-13, PG	Đây là ba nhãn phổ biến nhất và có ý nghĩa rõ ràng về phân loại độ tuổi. Các nhãn hiếm hoặc ít ổn định như G, NC-17, TV-MA, Unrated,... được gom thành Other. Sau mã hóa, nhóm tham chiếu là Other, còn các dummy được giữ là content_rating_PG, content_rating_PG-13, content_rating_R.

Như vậy, sau gom nhóm, số mức cần mã hóa giảm mạnh: language từ 41 mức còn 2 mức, country từ 60 mức còn 3 mức, và content_rating từ 15 mức còn 4 mức. Nhờ đó, one-hot encoding chỉ tạo thêm một số cột dummy có ý nghĩa thống kê rõ ràng, thay vì tạo nhiều cột hiếm chứa hầu hết giá trị 0. Đoạn mã dưới đây chỉ minh họa hàm gom nhóm.

```

1 def group_categories(df):
2     df_grouped = df.copy()
3     if 'language' in df_grouped.columns:
4         df_grouped['language'] = df_grouped['language'].apply(
5             lambda x: x if x == 'English' else 'Other'
6         )
7     if 'country' in df_grouped.columns:
8         df_grouped['country'] = df_grouped['country'].apply(
9             lambda x: x if x in ['USA', 'UK'] else 'Other'
10        )
11    if 'content_rating' in df_grouped.columns:
12        df_grouped['content_rating'] = df_grouped['content_rating']
13        .apply(
14            lambda x: x if x in ['R', 'PG-13', 'PG'] else 'Other'
15        )
16    return df_grouped

```

Listing 14: Mã minh họa gom nhóm các nhãn hiếm trước khi one-hot encoding.

Sau gom nhóm, `pd.get_dummies(..., drop_first=True)` được dùng để mã hóa. Tham số `drop_first=True` loại bỏ một nhóm cơ sở, tránh *dummy variable trap*. Nếu một biến có k mức mà tạo đủ k dummy cùng với intercept, tổng các dummy luôn bằng 1, gây cộng tuyến hoàn hảo. Với quy tắc trên, các cột dummy cuối cùng gồm `color_Color`, `language_Other`, `country_UK`, `country_USA`, `content_rating_PG`, `content_rating_PG-13` và `content_rating_R`.

Sau khi gom nhóm, nhóm mới thực hiện one-hot encoding và căn chỉnh cột để train/test có cùng cấu trúc. Đoạn mã sau minh họa bước mã hóa.

```

1 train_encoded = pd.get_dummies(
2     train_set_grouped,
3     columns=cat_cols,
4     drop_first=True,
5     dtype=float
6 )
7
8 test_encoded = pd.get_dummies(
9     test_set_grouped,
10    columns=cat_cols,
11    drop_first=True,
12    dtype=float
13 )
14
15 train_encoded, test_encoded = train_encoded.align(
16     test_encoded,
17     join='left',
18     axis=1,
19     fill_value=0.0
20 )

```

Listing 15: Mã minh họa one-hot encoding và căn chỉnh cột train/test.

Sau encoding, train set có kích thước (3885, 15) và test set có kích thước (981, 15). Trong đó, 15 cột bao gồm imdb_score và 14 đặc trưng.

C.7 Chuẩn bị ma trận đặc trưng và biến mục tiêu

Sau khi mã hóa, nhóm tách dữ liệu thành ma trận đặc trưng X và vector mục tiêu y :

$$y = \text{imdb_score}, \quad X = \text{các biến còn lại sau tiền xử lý.}$$

Kết quả cuối cùng được trình bày trong Bảng 24.

Bảng 24: Kích thước ma trận cuối cùng.

Đối tượng	Kích thước
X_{train}	(3885, 14)
y_{train}	(3885,)
X_{test}	(981, 14)
y_{test}	(981,)

Các đặc trưng cuối cùng gồm 7 biến số đã xử lý và 7 biến dummy: duration, director_facebook_likes, actor_3_facebook_likes, actor_1_facebook_likes, budget, title_year, actor_2_facebook_likes, color_Color, language_Other, country_UK, country_USA, content_rating_PG, content_rating_PG-13, content_rating_R.

C.8 Đóng gói quy trình bằng DataPipeline

File data_pipeline.py đóng gói toàn bộ quy trình thành class DataPipeline. Đây là bước quan trọng để chuyển quy trình phân tích thủ công của nhóm sang quy trình có thể tái sử dụng và triển khai lại trên dữ liệu mới.

Bảng 25: Vai trò các thành phần chính trong DataPipeline.

Thành phần	Vai trò
modes_, medians_	Lưu mode/median học từ train để điền khuyết cho dữ liệu mới.
knn_	Lưu KNN Imputer đã fit trên train để xử lý budget.
lower_bounds_, upper_bounds_	Lưu ngưỡng Winsorization theo phân vị 1% và 99%.
lambdas_	Lưu hệ số Box-Cox học từ train.
encoded_columns_	Lưu cấu trúc cột sau one-hot encoding để align test/new data.
process()	Chạy toàn bộ quy trình từ raw dataset đến $X_{train}, y_{train}, X_{test}, y_{test}$.

Sau khi đã xác định toàn bộ quy trình, nhóm đóng gói các bước vào DataPipeline để tái sử dụng. Đoạn mã dưới đây chỉ minh họa cách gọi pipeline, không thay thế phần giải thích phương pháp ở trên.

```

1 from data_pipeline import DataPipeline
2
3 raw_dataset = pd.read_csv('movie_metadata.csv')
4 pipeline = DataPipeline(test_size=0.2, random_state=42)
5
6 X_train, y_train, X_test, y_test = pipeline.process(raw_dataset)
7
8 print(X_train.shape, y_train.shape)
9 print(X_test.shape, y_test.shape)

```

Listing 16: Mã minh họa sử dụng class DataPipeline cho dữ liệu thô.

Kết quả thực nghiệm của nhóm cho thấy pipeline trả về đúng kích thước: $X_{train} = (3885, 14)$, $y_{train} = (3885,)$, $X_{test} = (981, 14)$ và $y_{test} = (981,)$. Điều này chứng minh pipeline đã tái hiện được quy trình xử lý thủ công, đồng thời đảm bảo train/test có cùng cấu trúc đặc trưng.

C.9 Kết luận

Phần chuẩn bị dữ liệu đã chuyển bộ IMDb 5000 Movie Dataset từ dạng thô, nhiều nhiễu và nhiều kiểu biến thành một bộ dữ liệu sạch, nhất quán và sẵn sàng cho hồi quy tuyến tính. Các quyết định xử lý đều dựa trên cơ sở thống kê rõ ràng: loại đa cộng tuyến để ổn định ma trận $X^T X$, loại biến leakage để đảm bảo tính dự đoán thực tế, dùng Winsorization để giảm ảnh hưởng ngoại lệ, dùng KNN/median/mode theo bản chất missing values, dùng Box-Cox để xử lý phân phối lệch, và dùng one-hot encoding có gom nhóm để tránh ma trận thưa.

Kết quả cuối cùng là một pipeline chuyên nghiệp, có khả năng tái lập và hạn chế rò rỉ dữ liệu. Đây là nền tảng cần thiết để các bước xây dựng, đánh giá và diễn giải mô hình hồi quy tuyến tính ở các phần sau đạt độ tin cậy cao hơn.

D Xây dựng mô hình (Modeling)

Sau khi hoàn thành bước chuẩn bị dữ liệu ở mục C, nhóm sử dụng ma trận đặc trưng cuối cùng để xây dựng và so sánh ba mô hình hồi quy: OLS cơ bản, OLS chọn biến và Ridge Regression. Các đoạn mã trong notebook được chuyển thành công thức, bảng kết quả và phần nhận xét để báo cáo tập trung vào diễn giải thống kê thay vì trình bày code.

D.1 Mô hình OLS cơ bản

Khai báo mô hình OLS cơ bản:

$$\begin{aligned} \text{imdb_score} = & \beta_0 + \beta_1 \cdot \text{actor_2_facebook_likes} + \beta_2 \cdot \text{actor_3_facebook_likes} + \beta_3 \cdot \text{content_rating_PG-13} \\ & + \beta_4 \cdot \text{content_rating_R} + \beta_5 \cdot \text{actor_1_facebook_likes} + \beta_6 \cdot \text{country_USA} \\ & + \beta_7 \cdot \text{content_rating_PG} + \beta_8 \cdot \text{country_UK} + \beta_9 \cdot \text{language_Other} + \beta_{10} \cdot \text{budget} \\ & + \beta_{11} \cdot \text{duration} + \beta_{12} \cdot \text{title_year} + \beta_{13} \cdot \text{color_Color} + \beta_{14} \cdot \text{director_facebook_likes} + \varepsilon. \end{aligned}$$

Trong đó, `imdb_score` là biến phụ thuộc; β_0 là hệ số tự do; β_1 đến β_{14} là các hệ số hồi quy tương ứng với từng biến độc lập; và ε là sai số ngẫu nhiên, đại diện cho những yếu tố ảnh hưởng đến điểm IMDb nhưng không được đưa vào mô hình.

Bảng 26: Tóm tắt kết quả mô hình OLS cơ bản.

Chỉ số	Giá trị	Chỉ số	Giá trị
R^2	0.2199	$Adj R^2$	0.2170
F-statistic	77.90	Prob(F-statistic)	0.0
Log-Likelihood	-5413.56	No. Observations	3885
AIC	10857.11	BIC	10951.08

Bảng 27: Bảng hệ số hồi quy chi tiết của mô hình OLS cơ bản.

Variable	coef	std err	t	$P > t $	[0.025, 0.975]
const	10.7629	3.3546	3.2084	0.0013	[4.1878, 17.3380]
duration	8.2329	0.4308	19.1120	0.000e+00	[7.3886, 9.0772]
director_facebook_likes	0.0447	0.0072	6.2508	4.526e-10	[0.0307, 0.0587]
actor_3_facebook_likes	-0.0285	0.0079	-3.6159	3.031e-04	[-0.0439, -0.0130]
actor_1_facebook_likes	0.0748	0.0099	7.5690	4.663e-14	[0.0554, 0.0941]
budget	-0.0015	3.223e-04	-4.5070	6.770e-06	[-0.0021, -8.209e-04]
title_year	-0.0122	0.0015	-8.1379	4.441e-16	[-0.0151, -0.0092]
actor_2_facebook_likes	0.0016	0.0124	0.1258	0.8999	[-0.0228, 0.0260]
color_Color	-0.4985	0.0830	-6.0071	2.063e-09	[-0.6611, -0.3358]
language_Other	0.6976	0.0809	8.6218	0.000e+00	[0.5391, 0.8562]
country_UK	0.3845	0.0723	5.3201	1.096e-07	[0.2429, 0.5262]
country_USA	0.0343	0.0536	0.6400	0.5222	[-0.0708, 0.1394]
content_rating_PG	-0.1250	0.0653	-1.9130	0.0558	[-0.2530, 0.0031]
content_rating_PG-13	-0.1880	0.0600	-3.1316	0.0018	[-0.3057, -0.0703]
content_rating_R	0.0110	0.0540	0.2044	0.8381	[-0.0948, 0.1169]

Bảng 28: Kiểm tra đa cộng tuyến bằng VIF cho mô hình OLS cơ bản.

Biến số độc lập	VIF	Biến số độc lập	VIF
actor_2_facebook_likes	4.22	country_UK	1.73
actor_3_facebook_likes	3.34	budget	1.63
content_rating_PG-13	3.05	language_Other	1.61
content_rating_R	2.92	duration	1.30
actor_1_facebook_likes	2.28	title_year	1.26
country_USA	2.15	color_Color	1.11
content_rating_PG	2.11	director_facebook_likes	1.07

D.1.1. Kiểm định ý nghĩa toàn phần của mô hình (F-test)

Để đánh giá ý nghĩa thống kê của mô hình tổng thể, ta thực hiện kiểm định:

- $H_0 : \beta_0 = \beta_1 = \beta_2 = \dots = \beta_k = 0$, tức mô hình không có giá trị dự báo.
- $H_A : \exists \beta_i \neq 0$, tức có ít nhất một biến độc lập ảnh hưởng đến `imdb_score`.

Với mức ý nghĩa $\alpha = 0.05$, kết quả hồi quy cho thấy $F = 77.90$ và $Prob(F\text{-statistic}) = 0.0 < 0.05$. Do đó, có đủ bằng chứng để bác bỏ H_0 . Mô hình OLS cơ bản có ý nghĩa thống kê tổng thể và phù hợp để phân tích mối quan hệ giữa các biến đầu vào với điểm IMDb.

D.1.2. Đánh giá độ khớp của mô hình R-squared

Chỉ số $R^2 = 0.2199$ cho thấy mô hình giải thích được khoảng 22.0% sự biến động của điểm số IMDb. Chỉ số $Adj R^2 = 0.2170$ sau khi hiệu chỉnh theo số lượng biến độc lập vẫn duy trì ở mức tương đối ổn định, cho thấy mô hình chưa xuất hiện hiện tượng thêm biến dư thừa quá mức. Mặc dù mức độ giải thích chưa cao, đây vẫn là kết quả có ý nghĩa thực tiễn đối với dữ liệu phim ảnh, nơi điểm IMDb chịu ảnh hưởng bởi nhiều yếu tố cảm tính như thị hiếu khán giả, xu hướng thị trường và giá trị nghệ thuật.

D.1.3. Kiểm định ý nghĩa từng biến (t-test)

Để xác định các biến thực sự ảnh hưởng đến điểm IMDb, ta kiểm định từng hệ số hồi quy với $H_0 : \beta_i = 0$ và $H_A : \beta_i \neq 0$ ở mức ý nghĩa $\alpha = 0.05$.

Các biến có ý nghĩa thống kê ($p < 0.05$) gồm `duration`, `director_facebook_likes`, `actor_1_facebook_likes`, `actor_3_facebook_likes`, `budget`, `title_year`, `color_Color`, `language_Other`, `country_UK` và `content_rating_PG-13`. Trong đó, `duration` có hệ số dương rất lớn ($coef = 8.2329$), cho thấy thời lượng phim có mối liên hệ cùng chiều với điểm IMDb trong dữ liệu đã tiền xử lý. `director_facebook_likes` và `actor_1_facebook_likes` cũng mang hệ số dương, phản ánh mức độ nổi tiếng của đạo diễn và diễn viên chính có liên hệ tích cực với điểm IMDb.

Ngược lại, `budget` ($coef = -0.0015$) và `title_year` ($coef = -0.0122$) mang dấu âm, cho thấy ngân sách lớn hoặc phim phát hành gần đây hơn không nhất thiết đi kèm điểm IMDb cao hơn trong mô hình hiện tại. Các biến không có ý nghĩa thống kê gồm `actor_2_facebook_likes` ($p = 0.900$), `country_USA` ($p = 0.522$) và

content_rating_R ($p = 0.838$). Riêng content_rating_PG có $p = 0.056$, lớn hơn nhẹ so với 0.05 nên chỉ ở trạng thái cận ý nghĩa thống kê.

D.1.4. Chẩn đoán đa cộng tuyến (Multicollinearity)

Mặc dù condition number của mô hình khá lớn, khoảng 4.32×10^5 , nhưng khi đánh giá bằng hệ số VIF thì toàn bộ biến độc lập đều nằm trong ngưỡng an toàn. Các biến có VIF lớn nhất là actor_2_facebook_likes (4.22), actor_3_facebook_likes (3.34) và content_rating_PG-13 (3.05), đều nhỏ hơn ngưỡng cảnh báo nghiêm trọng $VIF = 5$.

Kết luận là hiện tượng đa cộng tuyến nghiêm trọng không xảy ra trong mô hình OLS cơ bản. Condition number lớn chủ yếu xuất phát từ sự khác biệt thang đo giữa các biến định lượng và sự xuất hiện của biến giả sau encoding.

D.2 Mô hình OLS chọn biến

Từ kết quả kiểm định của mô hình đầy đủ biến OLS cơ bản, mô hình tổng thể có ý nghĩa thống kê rất cao vì:

$$Prob(F\text{-statistic}) = 0.000 < 0.05.$$

Điều này cho thấy tồn tại ít nhất một biến độc lập ảnh hưởng đáng kể đến điểm IMDb. Về đa cộng tuyến, mặc dù condition number lớn (4.32×10^5), toàn bộ các biến độc lập đều có $VIF < 5$, trong đó biến có VIF lớn nhất là actor_2_facebook_likes với VIF bằng 4.22. Vì vậy, mô hình không gặp vấn đề đa cộng tuyến nghiêm trọng.

Tuy nhiên, dựa trên kiểm định t-test, ba biến có $p\text{-value} > 0.05$ gồm actor_2_facebook_likes ($p = 0.900$), country_USA ($p = 0.522$) và content_rating_R ($p = 0.838$). Việc giữ lại các biến này có thể làm mô hình phức tạp hơn, tăng sai số chuẩn và giảm tính tinh gọn. Do đó, nhóm xây dựng mô hình OLS chọn biến bằng cách loại bỏ ba biến trên.

Ngoài ra, biến content_rating_PG tuy chưa đạt ý nghĩa thống kê trong OLS cơ bản ($p = 0.056$), nhưng sau khi tái xây dựng mô hình rút gọn thì trở nên có ý nghĩa với $p = 0.006 < 0.05$. Vì vậy, biến này tiếp tục được giữ lại.

Công thức hồi quy của mô hình OLS chọn biến có dạng:

$$\begin{aligned} \text{imdb_score} = & \beta_0 + \beta_1 \cdot \text{duration} + \beta_2 \cdot \text{director_facebook_likes} + \beta_3 \cdot \text{actor_3_facebook_likes} \\ & + \beta_4 \cdot \text{actor_1_facebook_likes} + \beta_5 \cdot \text{budget} + \beta_6 \cdot \text{title_year} + \beta_7 \cdot \text{color_Color} \\ & + \beta_8 \cdot \text{language_Other} + \beta_9 \cdot \text{country_UK} + \beta_{10} \cdot \text{content_rating_PG} + \beta_{11} \cdot \text{content_rating_PG-13} + \varepsilon. \end{aligned}$$

Bảng 29: Tóm tắt kết quả mô hình OLS chọn biến.

Chỉ số	Giá trị	Chỉ số	Giá trị
R^2	0.2198	$Adj R^2$	0.2175
F-statistic	99.17	Prob(F-statistic)	0.0
Log-Likelihood	-5413.79	No. Observations	3885
AIC	10851.58	BIC	10926.76

Bảng 30: Bảng hệ số hồi quy chi tiết của mô hình OLS chọn biến.

Variable	coef	std err	t	$P > t $	[0.025, 0.975]
const	10.8476	3.2996	3.2876	0.0010	[4.3804, 17.3148]
duration	8.2369	0.4281	19.2411	0.000e+00	[7.3979, 9.0760]
director_facebook_likes	0.0449	0.0071	6.2989	3.334e-10	[0.0309, 0.0589]
actor_3_facebook_likes	-0.0274	0.0058	-4.7263	2.368e-06	[-0.0387, -0.0160]
actor_1_facebook_likes	0.0756	0.0088	8.5748	0.000e+00	[0.0583, 0.0929]
budget	-0.0015	3.212e-04	-4.5173	6.451e-06	[-0.0021, -8.215e-04]
title_year	-0.0122	0.0015	-8.2736	2.220e-16	[-0.0151, -0.0093]
color_Color	-0.4954	0.0825	-6.0018	2.130e-09	[-0.6572, -0.3336]
language_Other	0.6701	0.0700	9.5664	0.000e+00	[0.5328, 0.8074]
country_UK	0.3548	0.0554	6.3997	1.742e-10	[0.2462, 0.4635]
content_rating_PG	-0.1330	0.0488	-2.7285	0.0064	[-0.2286, -0.0375]
content_rating_PG-13	-0.1962	0.0384	-5.1139	3.308e-07	[-0.2714, -0.1210]

Bảng 31: Kiểm tra đa cộng tuyến bằng VIF cho mô hình OLS chọn biến.

Biến số độc lập	VIF	Biến số độc lập	VIF
actor_1_facebook_likes	1.82	title_year	1.23
actor_3_facebook_likes	1.81	language_Other	1.20
budget	1.62	content_rating_PG	1.18
duration	1.28	color_Color	1.10
content_rating_PG-13	1.24	director_facebook_likes	1.06
		country_UK	1.02

D.2.1. Đánh giá về ý nghĩa mô hình

F-statistic bằng 99.17 cho thấy mô hình có mức ý nghĩa thống kê rất cao. Giá trị xác

suất đi kèm kiểm định F gần bằng 0 và nhỏ hơn nhiều so với $\alpha = 0.05$. Do đó, ta bác bỏ giả thuyết $H_0 : \beta_1 = \beta_2 = \dots = \beta_k = 0$ và chấp nhận đối thuyết $H_A : \exists \beta_i \neq 0$. Mô hình OLS chọn biến có ý nghĩa thống kê tổng thể.

D.2.2. Đánh giá về khả năng giải thích của mô hình

Chỉ số $R^2 = 0.2198$ và $Adj R^2 = 0.2175$ cho thấy mô hình giải thích được khoảng 21.98% sự biến động của `imdb_score`. Mặc dù mức độ giải thích chưa quá cao, đây là kết quả phù hợp với dữ liệu điện ảnh và giải trí, nơi điểm IMDb còn chịu ảnh hưởng bởi cảm nhận người xem, nội dung nghệ thuật, xu hướng truyền thông và chất lượng kịch bản.

Giá trị $Adj R^2$ gần bằng R^2 cho thấy việc loại bỏ biến không cần thiết không làm suy giảm đáng kể khả năng giải thích. Điều này chứng minh mô hình đạt được sự tinh gọn nhưng vẫn duy trì hiệu suất ổn định.

D.2.3. Diễn giải các hệ số hồi quy

Phần lớn biến trong mô hình đều có mức ý nghĩa thống kê rất cao với $p\text{-value} < 0.01$. `duration` ($coef = 8.2369$) là biến có mức ý nghĩa mạnh nhất, cho thấy phim có thời lượng dài hơn thường có xu hướng đạt điểm IMDb cao hơn.

`director_facebook_likes` ($coef = 0.0449$) và `actor_1_facebook_likes` ($coef = 0.0756$) đều mang hệ số dương, phản ánh mức độ nổi tiếng của đạo diễn và diễn viên chính có ảnh hưởng tích cực đến đánh giá phim.

`actor_3_facebook_likes` ($coef = -0.0274$) có hệ số âm nhưng vẫn đạt ý nghĩa thống kê cao, cho thấy mức độ nổi tiếng của diễn viên phụ không đồng biến với điểm IMDb trong tập dữ liệu hiện tại. `budget` ($coef = -0.0015$) và `title_year` ($coef = -0.0122$) cũng mang dấu âm, cho thấy ngân sách lớn hoặc năm phát hành mới hơn chưa chắc làm điểm IMDb cao hơn. Đáng chú ý, `content_rating_PG` ($coef = -0.1330$, $p = 0.0064$) và `content_rating_PG-13` ($coef = -0.1962$) đều có ý nghĩa thống kê trong mô hình rút gọn.

D.2.4. Kiểm tra hiện tượng đa cộng tuyến

Dựa trên hệ số VIF, toàn bộ biến độc lập trong mô hình đều có $VIF < 2$. Hai biến có VIF lớn nhất là `actor_1_facebook_likes` (1.82) và `actor_3_facebook_likes` (1.81), thấp hơn nhiều so với ngưỡng cảnh báo $VIF = 5$. Do đó, mô hình OLS chọn biến không gặp hiện tượng đa cộng tuyến nghiêm trọng và có mức ổn định thống kê tốt.

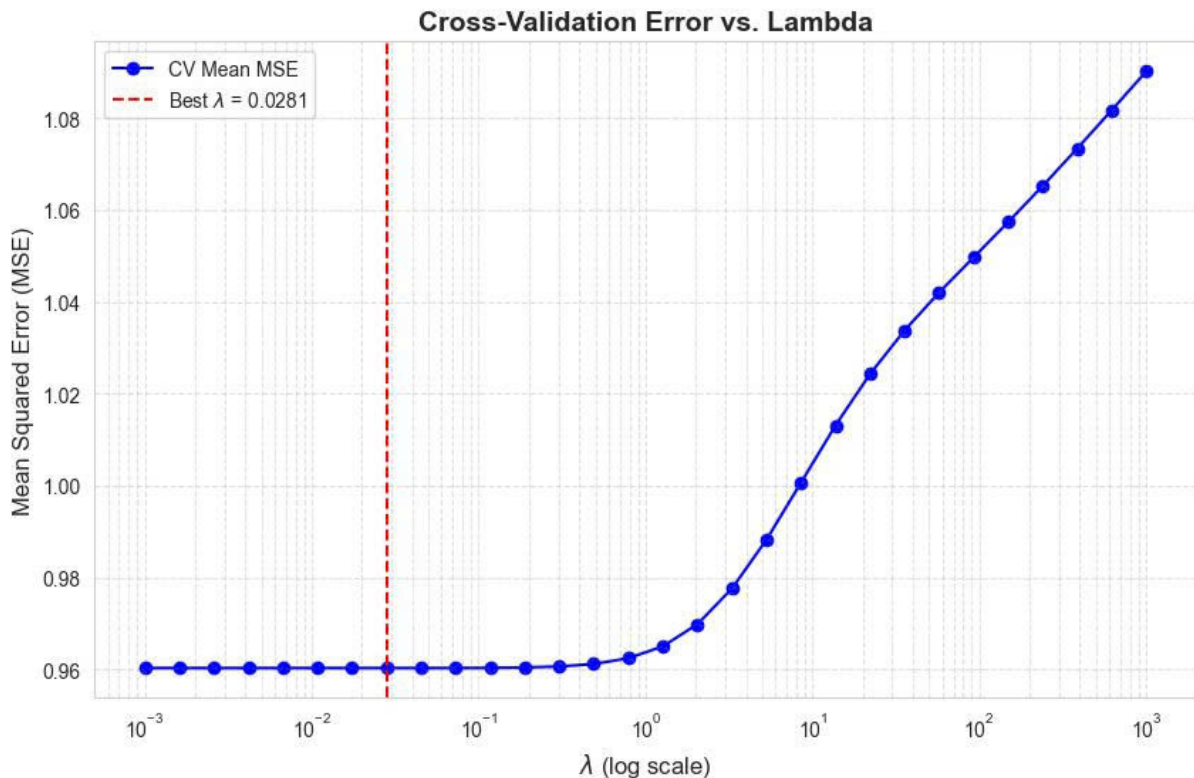
D.2.5. Kết luận

Sau khi loại bỏ các biến không có ý nghĩa thống kê trong OLS cơ bản, mô hình OLS chọn biến đạt được sự tinh gọn cần thiết nhưng vẫn duy trì hiệu suất giải thích gần tương đương mô hình ban đầu. Số biến độc lập giảm từ 14 xuống 11, R^2 gần như không thay đổi, đa số biến còn lại có ý nghĩa thống kê cao và toàn bộ $VIF < 2$. Kết quả này cho thấy mô hình phù hợp để phân tích và diễn giải mối quan hệ giữa đặc trưng phim với điểm IMDb.

D.3 Mô hình Ridge/Lasso

Trong notebook, nhóm tập trung triển khai Ridge Regression bằng cách kiểm thử nhiều giá trị λ trên thang logarit từ 10^{-3} đến 10^3 và chọn hệ số điều chuẩn tối ưu thông qua 5-Fold Cross-Validation. Kết quả cho thấy mô hình Ridge đạt giá trị tối ưu tại:

$$\lambda = 0.0281.$$



Hình 24: Cross Validation Error vs. Lambda

Bảng 32: Tóm tắt kết quả kiểm định chéo của Ridge Regression.

Thành phần	Kết quả
Miền giá trị λ khảo sát	10^{-3} đến 10^3 , chia theo thang logarit với 30 giá trị.
Phương pháp chọn mô hình	5-Fold Cross-Validation.
Giá trị tối ưu	$\lambda = 0.0281$.
Sai số vùng tối ưu	Mean MSE ≈ 0.960 .
Sai số khi $\lambda = 10^3$	Mean MSE ≈ 1.09 , cho thấy mô hình bắt đầu underfitting.

Phân tích mô hình Ridge Regression

Để cải thiện tính ổn định của mô hình hồi quy tuyến tính và hạn chế ảnh hưởng của đa cộng tuyến, nhóm xây dựng Ridge Regression kết hợp với 5-Fold Cross-Validation nhằm lựa chọn λ tối ưu.

Dựa trên đồ thị thực nghiệm giữa sai số kiểm thử chéo và tham số λ , đường cong Mean MSE thể hiện ba vùng biến thiên đặc trưng. Ở vùng tiệm cận OLS ($\lambda \leq 10^{-2}$), thành phần điều chuẩn $\lambda \|\beta\|_2^2$ gần như không đáng kể so với tổng bình phương phần dư RSS. Vì vậy, Ridge hoạt động gần tương đương OLS cơ bản, khiến sai số Cross-Validation duy trì ổn định quanh mức thấp nhất, xấp xỉ Mean MSE ≈ 0.960 . Ở giai đoạn này, hệ số hồi quy gần như chưa bị co hẹp, mô hình vẫn học mạnh từ dữ liệu, nhưng độ nhạy với nhiễu và đa cộng tuyến vẫn còn tương tự OLS.

Tại điểm tối ưu hội tụ, quá trình 5-Fold Cross-Validation xác định $\lambda = 0.02807216203941177$, làm tròn thành 0.0281. Đây là vị trí sai số kiểm thử chéo đạt giá trị nhỏ nhất trên toàn bộ miền khảo sát, cho thấy mô hình cân bằng tốt giữa độ chệch và phương sai. Ở mức λ này, các hệ số được co hẹp vừa đủ để tăng ổn định, hiện tượng dao động mạnh do đa cộng tuyến được kiểm soát tốt hơn, nhưng khả năng giải thích dữ liệu không bị suy giảm đáng kể.

Ở vùng underfitting ($\lambda > 10^{-1}$), khi λ tiếp tục tăng lớn, đặc biệt từ $\lambda \geq 1$, đường cong Mean MSE bắt đầu tăng nhanh. Tại $\lambda = 10^3$, sai số kiểm thử chéo tăng lên khoảng Mean MSE ≈ 1.09 . Nguyên nhân là thành phần điều chuẩn trở nên quá mạnh, khiến mô hình ưu tiên thu nhỏ vector hệ số β thay vì tối thiểu hóa sai số dự báo. Khi đó, nhiều quan hệ tuyến tính quan trọng bị suy yếu và mô hình rơi vào trạng thái underfitting.

Về mặt toán học, trong OLS truyền thống, đa cộng tuyến khiến $X^T X$ có thể gần suy biến, làm hệ số hồi quy thiếu ổn định, phương sai ước lượng tăng cao và mô hình nhạy

cảm với nhiễu. Ridge Regression khắc phục bằng cách điều chỉnh nghiệm OLS thành:

$$\hat{\beta}_{Ridge} = (X^T X + \lambda I)^{-1} X^T y.$$

Trong đó, I là ma trận đơn vị và λI đóng vai trò điều chuẩn hóa hệ thống tuyến tính. Việc cộng thêm λI giúp tăng tính khả nghịch của ma trận, ổn định hóa quá trình nghịch đảo, giảm phụ thuộc tuyến tính giữa các biến đầu vào và hạn chế hiện tượng hệ số hồi quy phình to.

Kết quả thực nghiệm từ Cross-Validation cho thấy Ridge Regression với $\lambda = 0.0281$ đạt hiệu suất tổng quát hóa tốt nhất trong miền khảo sát. Mô hình vừa kiểm soát đa cộng tuyến, vừa ổn định hệ số hồi quy, vừa giảm độ nhạy với nhiễu dữ liệu.

D.4 So sánh các mô hình trên

D.4.1. Phân tích điểm tương đồng và khác biệt giữa các mô hình

Về điểm tương đồng, khả năng giải thích biến động điểm IMDb của các mô hình đều dao động quanh ngưỡng 22%. Điều này cho thấy dù thay đổi thuật toán hoặc cắt giảm biến, giới hạn trần thông tin tuyến tính khai thác được từ bộ dữ liệu hiện tại đã gần đạt trạng thái bão hòa. Xu hướng tác động của các biến cốt lõi như `duration`, `director_facebook_likes` và `actor_1_facebook_likes` luôn duy trì chiều tác động tích cực với mức ý nghĩa thống kê rất cao ($p\text{-value} < 0.001$) trong các mô hình OLS. Ngược lại, `budget` và `title_year` luôn mang dấu âm ổn định.

Bảng 33: Tóm tắt điểm tương đồng giữa các mô hình.

Khía cạnh	Nhận xét chung
Năng lực giải thích	R^2 quanh 22%, phản ánh giới hạn của thông tin tuyến tính hiện có.
Biến dương ổn định	<code>duration</code> , <code>director_facebook_likes</code> , <code>actor_1_facebook_likes</code> .
Biến âm ổn định	<code>budget</code> , <code>title_year</code> .
Ý nghĩa thực nghiệm	Kỹ thuật mô hình hóa giúp tăng tính ổn định nhưng không tạo bước nhảy lớn về năng lực giải thích.

Về điểm khác biệt, OLS cơ bản chọn giải pháp thụ động bằng cách giữ lại cả các biến không có ý nghĩa như `actor_2_facebook_likes` ($p = 0.900$), `country_USA` ($p =$

0.522) và `content_rating_R` ($p = 0.838$). OLS chọn biến áp dụng cơ chế loại bỏ các biến không có ý nghĩa thống kê ($p \geq 0.05$). Trong khi đó, Ridge Regression giữ lại các biến nhưng dùng phạt $L2$ để triệt tiêu bớt quyền lực của biến nhiễu, ép hệ số hồi quy nhỏ lại gần 0.

Về mức độ ổn định, OLS cơ bản chịu rủi ro lớn nhất do condition number cao và sự khác biệt thang đo giữa các biến. OLS chọn biến giải quyết bằng cách dập tắt nguồn gốc gây nhiễu trong mô hình. Ridge Regression giải quyết theo hướng toán học hơn, tức thay đổi cấu trúc ma trận hiệp biến thông qua việc cộng thêm thành phần λI .

Bảng 34: Khác biệt trong cách xử lý biến nhiễu và độ ổn định.

Mô hình	Xử lý biến nhiễu	Độ ổn định
OLS cơ bản	Giữ toàn bộ 14 biến, kể cả biến có p -value cao.	Có condition number cao; VIF vẫn dưới 5 nhưng mô hình còn cồng kềnh.
OLS chọn biến	Loại <code>actor_2_facebook_likes</code> , <code>country_USA</code> , <code>content_rating_R</code> .	Ổn định hơn, toàn bộ $VIF < 2$.
Ridge Regression	Giữ biến nhưng dùng phạt $L2$ để co nhỏ hệ số.	Ổn định nhờ nghiệm $(X^T X + \lambda I)^{-1} X^T y$.

D.4.2. Đánh giá về các mô hình

Bảng 35: Đánh giá điểm mạnh và điểm yếu của từng mô hình.

Mô hình	Điểm mạnh	Điểm yếu
OLS cơ bản	Cung cấp cái nhìn toàn diện, nguyên bản về toàn bộ thuộc tính thu thập được từ dữ liệu phim ảnh mà không qua bộ lọc định kiến.	Cồng kềnh, chứa nhiều biến dư thừa ít giá trị thống kê; hệ số dễ nhạy cảm với biến động nhỏ hoặc nhiễu trong dữ liệu huấn luyện.
OLS chọn biến	Đạt độ tinh gọn toán học cao; $Adj R^2$ đạt mức cao nhất 21.75%; $VIF < 2$ khẳng định đa cộng tuyến được kiểm soát tốt; mô hình dễ diễn giải.	Cơ chế loại bỏ biến dựa thuần túy trên p -value có thể làm mất thông tin biên hoặc quan hệ tương tác phức tạp giữa các cụm biến giả.
Ridge Regression	Tổng quát hóa an toàn nhất nhờ tối ưu sai số bằng 5-Fold Cross-Validation; giải quyết tốt bài toán ma trận gần suy biến; phù hợp khi cần dự báo dữ liệu mới liên tục.	Không triệt tiêu hoàn toàn biến nào về 0, nên vẫn duy trì cả 14 đặc trưng và khó giải thích tường minh hơn OLS chọn biến.

D.4.3. Kết luận

Lựa chọn tối ưu cho mục đích dự báo dữ liệu mới là Ridge Regression với $\lambda = 0.0281$. Mô hình này vượt trội nhờ khả năng kiểm soát phương sai ước lượng, được bảo chứng bằng quy trình kiểm định chéo trực quan. Ridge giúp hạn chế rủi ro quá khớp của OLS cơ bản và giữ sai số kiểm định chéo ổn định ở vùng thấp nhất.

Lựa chọn tối ưu cho mục đích diễn giải chính sách là OLS chọn biến. Nếu mục tiêu nghiên cứu là tìm kiếm sự tường minh, tinh gọn để báo cáo hoặc phân tích xu hướng hành vi, OLS chọn biến cung cấp bộ tham số tối giản, có ý nghĩa thống kê đồng đều, loại bỏ gần như hoàn toàn đa cộng tuyến ($VIF < 2$) mà vẫn giữ năng lực giải thích dữ liệu tương đương mô hình đầy đủ.

Bảng 36: Khuyến nghị lựa chọn mô hình theo mục tiêu sử dụng.

Mục tiêu	Mô hình khuyến nghị	Lý do
Dự báo dữ liệu mới	Ridge Regression, $\lambda = 0.0281$	Ổn định nhất, kiểm soát phương sai tốt và được lựa chọn bằng 5-Fold Cross-Validation.
Diễn giải chính sách	OLS chọn biến	Tinh gọn, dễ giải thích, các biến còn lại có ý nghĩa thống kê và toàn bộ $VIF < 2$.
Thăm dò ban đầu	OLS cơ bản	Hữu ích để quan sát toàn bộ biến đầu vào trước khi chọn biến hoặc điều chuẩn.

E Đánh giá trên tập test

E.1 Chạy trên bộ test và chọn mô hình tốt nhất

Sau khi huấn luyện thành công, chúng ta tiến hành đưa bộ dữ liệu kiểm tra X_{test} (dữ liệu hoàn toàn mới mà các mô hình chưa từng được tiếp cận) vào để chạy **Dự đoán**. Hiệu suất dự đoán của 3 mô hình sẽ được đo lường khách quan qua 3 chỉ số: R^2 , RMSE, và MAE.

Nguyên tắc: Tập Test được giữ **hoàn toàn độc lập** trong suốt quá trình huấn luyện và chọn hyperparameter. Đây là bước đánh giá **khách quan cuối cùng** phản ánh khả năng tổng quát hóa (generalization) của từng mô hình.

Các độ đo được dùng để so sánh:

- **Sai số tuyệt đối trung bình (MAE):** Đo lường mức độ sai số trung bình tuyệt đối, phản ánh thực tế sai lệch điểm IMDb.
- **Sai số bình phương trung bình căn (RMSE):** Đo lường độ lệch trung bình giữa giá trị dự đoán và thực tế, phạt nặng các sai số lớn.
- **Hệ số xác định (R^2):** Đo lường tỷ lệ phương sai của biến mục tiêu được giải thích bởi mô hình.

$$\text{MAE} = \frac{1}{n_{\text{test}}} \sum_i |y_i - \hat{y}_i|, \quad \text{RMSE} = \sqrt{\frac{1}{n_{\text{test}}} \sum_i (y_i - \hat{y}_i)^2}, \quad R^2_{\text{test}} = 1 - \frac{\text{RSS}_{\text{test}}}{\text{TSS}_{\text{test}}}$$

```

1 import numpy as np
2 import pandas as pd
3 from summary_utils import dataframe_to_matrix, ridge_predict
4 from statsmodels.api import add_constant
5 from model_comparison import model_comparison
6
7 # Du lieu cho OLS (Them hang so)
8 X_test_df = test_encoded.drop(columns=['imdb_score'])
9 X_test_const = np.array(dataframe_to_matrix(add_constant(X_test_df
10     )))
11 X_test_sel_const = np.array(dataframe_to_matrix(add_constant(
12     test_encoded[selected_features])))
13
14 # Du lieu cho Ridge
15 X_test_raw = X_test_df.values.tolist()
16 y_test_raw = [float(x) for x in test_encoded['imdb_score'].tolist
17     ()]
18
19 # Lay beta
20 beta_full = model_results['beta_hat']
21 beta_short = selected_model_results['beta_hat']
22 beta_ridge = raw_beta_ridge
23
24 def predict_ols(X_matrix, beta):
25     return np.dot(X_matrix, np.array(beta)).flatten().tolist()
26
27 # Tinh y pred
28 y_pred_full = predict_ols(X_test_const, beta_full)
29 y_pred_short = predict_ols(X_test_sel_const, beta_short)
30 y_pred_ridge = ridge_predict(X_test_raw, beta_ridge, fit_intercept
31     =True)
32
33 # So sanh cac mo hinh
34 comparison_table = model_comparison([
35     {"model_name": "OLS Full", "y_pred": y_pred_full, "
36     y_true": y_test_raw},
37     {"model_name": "OLS Chon Bien", "y_pred": y_pred_short, "
38     y_true": y_test_raw},
39     {"model_name": "Ridge Regression", "y_pred": y_pred_ridge, "
40     y_true": y_test_raw},
41 ])
42
43 print(comparison_table.to_string(index=True))

```

	Mô hình	MAE	RMSE	R-squared (Test)
0	OLS Full	0.7781	1.0551	0.1753
1	OLS Chọn Biến	0.7775	1.0549	0.1757
2	Ridge Regression	0.7781	1.0552	0.1752

E.1.1. Nhận xét

Cả 3 mô hình cho kết quả rất sát nhau trên toàn bộ 3 chỉ số, cho thấy không có mô hình nào vượt trội rõ rệt. Đây là hiện tượng phổ biến khi dữ liệu có nhiều nhiễu và mối quan hệ tuyến tính giữa các đặc trưng với biến mục tiêu là tương đối yếu.

Về MAE:

$$MAE_{\text{OLS Chọn Biến}} = 0.7775 < MAE_{\text{OLS Full}} = MAE_{\text{Ridge}} = 0.7781$$

Sai số tuyệt đối trung bình của cả 3 mô hình đều dưới 0.78 điểm IMDb, hoàn toàn hợp lý về mặt thực tế. OLS Chọn Biến thấp hơn 0.0006 so với hai mô hình còn lại.

Về RMSE:

$$RMSE_{\text{OLS Chọn Biến}} = 1.0549 < RMSE_{\text{OLS Full}} = 1.0551 < RMSE_{\text{Ridge}} = 1.0552$$

RMSE lớn hơn MAE ở cả 3 mô hình, phản ánh sự tồn tại của một số điểm ngoại lệ. OLS Chọn Biến vẫn dẫn đầu, dù biên độ chênh lệch chỉ là 0.0003.

Về R^2 :

$$R^2_{\text{OLS Chọn Biến}} = 0.1757 > R^2_{\text{OLS Full}} = 0.1753 > R^2_{\text{Ridge}} = 0.1752$$

Cả 3 mô hình chỉ giải thích được khoảng 17.5% phương sai của IMDb score — con số thấp nhưng phù hợp với đặc thù bài toán, vì điểm IMDb phụ thuộc nhiều vào yếu tố chủ quan của người xem mà các đặc trưng định lượng không thể nắm bắt hoàn toàn.

E.1.2. Kết luận

Mô hình được chọn: OLS Chọn Biến

Mặc dù mức chênh lệch giữa 3 mô hình là rất nhỏ và không có ý nghĩa thống kê đáng kể, OLS Chọn Biến được chọn vì:

- Tốt nhất trên cả 3 chỉ số: MAE thấp nhất (0.7775), RMSE thấp nhất (1.0549), R^2 cao nhất (0.1757).
- Nguyên tắc parsimony (*Occam's Razor*): Chỉ giữ lại các biến có ý nghĩa thống kê, loại bỏ nhiều so với OLS Full — mô hình đơn giản hơn mà hiệu suất không giảm.
- Khả năng diễn giải tốt hơn: Ít biến hơn giúp dễ phân tích ý nghĩa của từng hệ số $\hat{\beta}$.
- Rủi ro overfitting thấp hơn OLS Full: Việc loại bỏ biến không có ý nghĩa giúp mô hình tổng quát hóa tốt hơn trên dữ liệu mới.

E.2 Kiểm chứng với phần dư (Residual Analysis)

Sau khi lựa chọn **OLS Chọn Biến** là mô hình tốt nhất, ta tiến hành phân tích phần dư để kiểm chứng các giả định cơ bản của hồi quy tuyến tính OLS:

Bảng 37: Các giả định kiểm tra thông qua biểu đồ phần dư

Biểu đồ	Giả định kiểm tra
(1) Residuals vs Fitted	Linearity — phần dư không có xu hướng hệ thống theo $\hat{y}$
(2) Normal Q-Q Plot	Normality — phần dư xấp xỉ phân phối chuẩn $\mathcal{N}(0, \sigma^2)$
(3) Scale-Location	Homoscedasticity — phương sai phần dư đồng đều (không đổi theo $\hat{y}$)
(4) Cook's Distance	Influential points — phát hiện quan sát có ảnh hưởng bất thường đến $\hat{\beta}$

```

1 from partlutils.residual_analysis import residual_plots
2 import numpy as np
3 from summary_utils import dataframe_to_matrix
4 from statsmodels.api import add_constant
5
6 # Chuẩn bị đầu vào
7 X_train_sel_const = np.array(
8     dataframe_to_matrix(add_constant(train_encoded[
9         selected_features])))
10 y_train_raw = [float(x) for x in train_encoded['imdb_score'].
11                 tolist()]
12
13 # Flatten beta_short về 1D nếu đang là 2D
14 beta_flat = [

```


(1) Residuals vs Fitted Values — Kiểm tra Linearity & Homoscedasticity

Biểu đồ này kiểm tra hai giả định quan trọng: **tính tuyến tính** và **đồng phương sai**. Nếu mô hình phù hợp, các điểm phần dư phải phân bố ngẫu nhiên quanh đường $e = 0$, không tạo thành dạng cong hoặc dạng sóng; đồng thời độ rộng phân tán của phần dư phải tương đối ổn định dọc theo trục $\hat{y}$, không tạo hình phễu.

Quan sát:

- Đường LOWESS trend có hình chữ **S rõ rệt** — đi từ giá trị dương ($\approx +2.5$) tại $\hat{y} \approx 4$, cắt đường $e = 0$ tại $\hat{y} \approx 6$, rồi xuống âm khi $\hat{y} > 8$.
- Mức uốn cong của LOWESS **lớn và có hệ thống**, khác hẳn với dao động ngẫu nhiên.
- Độ phân tán của phần dư thu hẹp dần khi $\hat{y}$ tăng, tạo thành dạng **hình phễu ngược**.

Kết luận: Vi phạm cả hai giả định:

- **Linearity:** LOWESS có xu hướng phi tuyến hình chữ S — mô hình tuyến tính chưa nắm bắt được toàn bộ cấu trúc của dữ liệu IMDb score.
- **Homoscedasticity:** Độ phân tán phần dư không đồng đều theo $\hat{y}$ — có dấu hiệu của hiện tượng **heteroscedasticity**.

(2) Normal Q-Q Plot — Kiểm tra Normality

Biểu đồ này kiểm tra giả thuyết phần dư chuẩn hóa có phân phối gần chuẩn hay không. Nếu phần dư tuân theo phân phối chuẩn, các điểm dữ liệu sẽ nằm gần đường tham chiếu. Khi các điểm lệch mạnh ở hai đuôi, phần dư có thể có đuôi dày hoặc chứa các outliers.

Quan sát:

- Vùng trung tâm $[-2, +2]$: các điểm bám **khá sát** đường tham chiếu màu đỏ → phần lớn phần dư ở vùng giữa tuân thủ tốt phân phối chuẩn lý thuyết.
- Đuôi trái ($q < -2$): các điểm **lệch xuống dưới** đường tham chiếu — đuôi trái dày hơn chuẩn.
- Đuôi phải ($q > 2$): các điểm **lệch lên trên** đường tham chiếu — đuôi phải dày hơn chuẩn.

- Xuất hiện **cấu trúc dạng chữ S** ở hai đuôi.

Kết luận: Giả định Normality được thỏa mãn ở mức chấp nhận được. Phần dư có hiện tượng **heavy-tailed** (leptokurtic) — hai đuôi dày hơn so với phân phối chuẩn lý tưởng, chủ yếu do một số outliers có độ lệch lớn. Tuy nhiên, sai lệch này chỉ xuất hiện ở vùng đuôi và không phá vỡ cấu trúc chính của phân phối, nên các kiểm định thống kê vẫn có thể thực hiện với độ tin cậy hợp lý.

(3) Scale-Location Plot — Kiểm tra Homoscedasticity

Biểu đồ Scale-Location nhạy hơn biểu đồ (1) trong việc phát hiện phương sai thay đổi. Nếu mô hình có đồng phương sai, đường LOWESS nên tương đối phẳng và độ phân tán của các điểm không nên tăng hoặc giảm có hệ thống theo $\hat{y}$.

Quan sát:

- Đường LOWESS dốc **giảm mạnh và có hệ thống** từ trái sang phải: $\sqrt{|r_i|} \approx 1.7$ tại $\hat{y} \approx 4$ xuống còn ≈ 0.5 tại $\hat{y} \approx 9$, chênh lệch khoảng **1.2** — lớn hơn nhiều so với mức dao động ngẫu nhiên.
- Độ phân tán của các điểm cũng **thu hẹp rõ rệt** khi $\hat{y}$ tăng.
- Biểu đồ có dạng **hình phễu ngược** — trái rộng, phải hẹp.

Kết luận: Vi phạm rõ ràng giả định **Homoscedasticity**. Khác với dữ liệu mô phỏng trong báo cáo lý thuyết — nơi LOWESS chỉ dốc nhẹ với biên độ ≈ 0.37 và không đủ mạnh để kết luận phương sai thay đổi — ở đây biên độ thay đổi lên đến ≈ 1.2 , là bằng chứng rõ ràng của hiện tượng **heteroscedasticity**. Điều này có thể ảnh hưởng đến tính hiệu quả của ước lượng OLS.

(4) Cook's Distance — Kiểm tra Influential Points

Cook's Distance xác định các quan sát có ảnh hưởng lớn đến hệ số ước lượng $\hat{\beta}$. Một quan sát có D_i lớn thường vừa có phần dư lớn, vừa có leverage cao. Ngưỡng tham chiếu được sử dụng là $D_i > 4/n$.

Quan sát:

- Ngưỡng $4/n = 0.0010$ — rất thấp do tập train có $n \approx 4000$ quan sát.

- Có **nhiều quan sát vượt ngưỡng** $4/n$, nổi bật là: **388, 1023, 778, 3394, 2371,...** với D_i cao nhất đạt ≈ 0.014 .
- Tuy nhiên, **không có quan sát nào** đạt mức ngưỡng nghiêm trọng $D_i > 0.5$ hoặc $D_i > 1.0$.

Kết luận: Có nhiều điểm vượt ngưỡng tương đối $4/n$, nhưng tương tự như nhận xét trong báo cáo lý thuyết, giá trị D_i lớn nhất vẫn thấp xa so với các ngưỡng nghiêm trọng. Do đó, các điểm này chỉ nên được xem là **điểm cần chú ý**, không phải điểm làm méo mó hệ thống mô hình. Mô hình OLS Chọn Biến được đánh giá là **ổn định**, các hệ số β không bị chi phối bởi một điểm dữ liệu đơn lẻ.

Tổng kết

Bảng 38: Tổng kết đánh giá các giả định thống kê của mô hình OLS

Giả định	Biểu đồ	Kết quả
Linearity	(1) Residuals vs Fitted	Vi phạm — LOWESS có cấu trúc phi tuyến hình chữ S
Normality	(2) Normal Q-Q Plot	Chấp nhận được — heavy-tailed nhẹ ở hai đuôi
Homoscedasticity	(3) Scale-Location	Vi phạm — phương sai giảm mạnh theo y
Influential points	(4) Cook's Distance	Nhiều điểm vượt $4/n$ nhưng không đạt ngưỡng nghiêm trọng

Nhận xét chung: Mô hình OLS Chọn Biến vi phạm hai giả định quan trọng là **Linearity** và **Homoscedasticity**. Điều này phù hợp với $R^2 \approx 0.1757$ — mô hình tuyến tính đơn thuần chưa đủ khả năng mô tả toàn bộ mối quan hệ giữa các đặc trưng và IMDb score. Giả định **Normality** được thỏa mãn ở mức chấp nhận được, và không có điểm dữ liệu đơn lẻ nào gây ảnh hưởng nghiêm trọng đến mô hình. Để cải thiện, có thể xem xét **biến đổi biến mục tiêu**, bổ sung **đặc trưng phi tuyến**, hoặc chuyển sang các mô hình phức tạp hơn trong các nghiên cứu tiếp theo.

6 Kết luận

A Tóm tắt kết quả đạt được

Trải qua quá trình nghiên cứu và thực hiện đồ án, nhóm đã hoàn thành đầy đủ các mục tiêu đề ra với các kết quả cụ thể như sau:

- **Về mặt lý thuyết và cơ sở toán học:** Nghiên cứu và cài đặt thành công từ đầu (*from scratch*) các thuật toán hồi quy tuyến tính (OLS, Ridge) cùng hệ thống đánh giá toàn diện gồm các hệ số (R^2 , Adjusted R^2), kiểm định giả thuyết (F-test, t-test), hiện tượng đa cộng tuyến (VIF) và kiểm định chéo (K-Fold Cross-Validation).
- **Về phân tích và trực quan hóa lý thuyết:** Thực nghiệm thành công mô phỏng Monte Carlo để kiểm chứng định lý Gauss-Markov và trực quan hóa rõ nét hiện tượng đánh đổi độ chệch và phương sai (Bias-Variance Tradeoff) giữa OLS và Ridge Regression. Đồng thời, xây dựng hệ thống biểu đồ chẩn đoán phần dư (Residual Diagnostic Plots) để đánh giá các giả định cơ bản.
- **Về ứng dụng thực tế:** Xây dựng hoàn chỉnh một *Data Pipeline* chuẩn mực trên bộ dữ liệu thực tế IMDb 5000 Movie Dataset. Thực hiện xử lý triệt để các vấn đề của dữ liệu thô: lọc bỏ giá trị khuyết, chuẩn hóa outliers, biến đổi phân phối bằng Box-Cox và mã hóa One-hot cho các biến phân loại.
- **Về đánh giá mô hình thực tế:** Lựa chọn và đánh giá thành công các mô hình OLS cơ bản, OLS chọn biến và Ridge Regression trên tập kiểm thử độc lập. Phân tích sâu sắc ưu nhược điểm của từng mô hình qua các độ đo (MAE, RMSE, R^2) và đưa ra lựa chọn tối ưu (OLS Chọn Biến) dựa trên nguyên tắc tính nguyên sơ (Parsimony) và khả năng diễn giải thực tế.

B Bài học rút ra

- **Tầm quan trọng của việc hiểu bản chất toán học:** Cài đặt thuật toán thủ công giúp nhóm không sử dụng thư viện học máy như một "hộp đen", mà thực sự hiểu sâu sắc cách hệ số β được tối ưu hóa, cũng như các giả định (Linearity, Normality, Homoscedasticity) phía sau một mô hình tuyến tính.
- **Tác động của tiền xử lý dữ liệu:** Trong bài toán thực tế, chất lượng dữ liệu và các bước *Feature Engineering* đóng vai trò cốt lõi. Việc khử đa cộng tuyến và biến đổi

biến về phân phối chuẩn (Box-Cox) đã giúp mô hình OLS ổn định hơn hẳn so với việc học trực tiếp trên dữ liệu thô.

- **Tính thực tế của các mô hình và giới hạn của độ đo:** Việc hệ số R^2 trên tập thực tế chỉ đạt mức tương đối thấp ($\sim 17.5\%$) phản ánh đúng tính chất chủ quan của bài toán đánh giá phim. Qua đó, nhóm học được cách đánh giá khách quan một kết quả thống kê thay vì chỉ chạy theo việc tối đa hóa độ đo một cách mù quáng.

C Hướng mở rộng

- **Thử nghiệm các mô hình phi tuyến tính:** Qua phân tích phần dư bằng biểu đồ LOWESS, dữ liệu vẫn chứa nhiều mối quan hệ phi tuyến mà OLS không nắm bắt được. Hướng đi tiếp theo là áp dụng các mô hình phi tuyến mạnh mẽ hơn như Random Forest hoặc Gradient Boosting (XGBoost, LightGBM).
- **Khai thác dữ liệu văn bản (NLP):** Tập dữ liệu IMDb chứa nhiều trường thông tin ngôn ngữ tự nhiên cực kỳ có giá trị (plot keywords, genres). Việc sử dụng các mô hình nhúng từ (Word2Vec, BERT) có thể trích xuất thêm các đặc trưng mạnh mẽ cho mô hình.
- **Phân tích tương tác giữa các biến (Interaction Terms):** Khám phá sâu hơn tác động kết hợp của các đặc trưng (ví dụ: một thể loại phim cụ thể kết hợp với diễn viên hạng A có làm tăng điểm IMDb không) bằng cách bổ sung các đa thức tương tác vào mô hình.

7 Tài liệu tham khảo

Tài liệu

- [1] Chuxin Chen (Kaggle). *IMDB 5000 Movie Dataset*.
<https://www.kaggle.com/datasets/carolzhangdc/imdb-5000-movie-dataset>
- [2] Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani. *An Introduction to Statistical Learning (ISLR)*.
<https://www.statlearning.com/>
- [3] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
<https://www.microsoft.com/en-us/research/uploads/prod/2006/01/Bishop-Pattern-Recognition-and-Machine-Learning-2006.pdf>
- [4] Skipper Seabold, Josef Perktold. *statsmodels: Econometric and statistical modeling with python*.
<https://www.statsmodels.org/stable/index.html>
- [5] F. Pedregosa et al. *Scikit-learn: Machine Learning in Python*. JMLR 12, pp. 2825-2830, 2011.
<https://scikit-learn.org/stable/>
- [6] Jake VanderPlas. *Python Data Science Handbook*. O'Reilly Media.
<https://jakevdp.github.io/PythonDataScienceHandbook/>